



Papers fundacionales de la IA

Artificial Intelligence Evolution Program · eje transversal

16 hitos · de Rosenblatt (1958) a los sistemas agentic · actualizado 2026-08-16

Este documento enlaza a las fuentes primarias; no las reproduce. Los notebooks ejecutables viven en el repositorio.

1. P01 — El perceptrón: un modelo probabilístico de almacenamiento y organización de información en el cerebro (1958)
2. P02 — Aprender representaciones retropropagando errores (1986)
3. P03 — Memoria larga de corto plazo (1997)
4. P04 — Clasificación de ImageNet con redes neuronales convolucionales profundas (2012)
5. P05 — Estimación eficiente de representaciones de palabras en un espacio vectorial (2013)
6. P06 — Aprendizaje de secuencia a secuencia con redes neuronales (2014)
7. P07 — Traducción automática neuronal aprendiendo conjuntamente a alinear y traducir (2014)
8. P08 — La atención es todo lo que necesitas (2017)
9. P09 — BERT: preentrenamiento de Transformers bidireccionales profundos para comprensión del lenguaje (2018)
10. P10 — Los modelos de lenguaje son aprendices con pocos ejemplos (2020)
11. P11 — Generación aumentada por recuperación para tareas de PLN intensivas en conocimiento (2020)
12. P12 — Entrenar modelos de lenguaje para seguir instrucciones con retroalimentación humana (2022)
13. P13 — ReAct: sinergia entre razonar y actuar en modelos de lenguaje (2022)
14. P14 — Toolformer: los modelos de lenguaje pueden enseñarse a sí mismos a usar herramientas (2023)
15. P15 — Optimización directa de preferencias: tu modelo de lenguaje ya es un modelo de recompensa (2023)

16. P16 — Sistemas agentic contemporáneos: memoria, reflexión, multiagente e interoperabilidad
(2023)

📖 Eje de papers fundacionales ## **16 hitos · 24 notebooks ejecutables · de Rosenblatt (1958) a los sistemas agentic**
La historia de la IA contada por los papers que la movieron — no como una colección de PDFs, sino como una cadena de problemas resueltos que cada estudiante puede ejecutar, romper e interpretar.
📖 Índice de papers · 📖 Ruta y niveles · 🌐 Fuentes y venues · 📖 Cómo leer un paper · 🔄 5 pasadas · 📖 Glosario | 📄 Papers | 📖 Notebooks | 🧪 Motores | 🎓 Niveles | 🔗 Clases enlazadas | |:---:|:---:| |:---:|:---:| | **16** | **24** | **16** | **L0-L5** | **22** |

💡 La idea

Un paper leído es una anécdota. Un paper **ejecutado, roto e interpretado** es conocimiento.

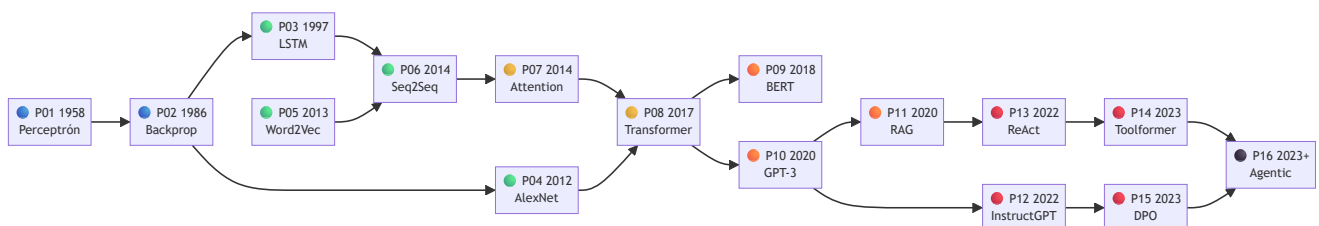
Este eje convierte cada hito en una experiencia con la misma secuencia siempre:

problema histórico → propuesta → intuición → matemática mínima → implementación → experimento → interpretación → limitaciones → siguiente hito

La última flecha es la importante. **Cada paper existe porque el anterior dejó algo sin resolver.** El perceptrón no separa XOR → backpropagation entrena capas ocultas → el gradiente se desvanece en secuencias largas → la LSTM lo controla → un vector fijo no sostiene frases largas → la atención lo elimina → si la atención basta, sobra la recurrencia → Transformer. Estudiado así, no hay nada que memorizar: hay una cadena que se sigue.

[!IMPORTANT] Este eje **no redistribuye papers**. Enlaza a la fuente primaria de cada uno, con autoría, año, venue, URL y fecha de consulta. Los notebooks implementan **miniaturas** del mecanismo en Python estándar: no reproducen los experimentos originales y lo declaran en cada salida.

🗺️ La ruta mínima



#	Paper	Año	Nivel	Lo que desbloqueó
P01	Perceptrón	1958	L1	Una máquina ajusta sus pesos con ejemplos
P02	Backpropagation	1986	L2	Se pueden entrenar capas ocultas
P03	LSTM	1997	L2	Memoria a través de cientos de pasos
P04	AlexNet	2012	L3	El deep learning se vuelve corriente principal
P05	Word2Vec	2013	L2	Las palabras tienen geometría
P06	Seq2Seq	2014	L3	Secuencia variable → secuencia variable
P07	Attention (Bahdanau)	2014	L3	Muere el cuello de botella del vector fijo
P08	Transformer	2017	L4	Se elimina la recurrencia; todo paraleliza
P09	BERT	2018	L3	Preentrenar y ajustar como norma
P10	GPT-3	2020	L3	Aprendizaje en contexto sin tocar pesos
P11	RAG	2020	L3	Conocimiento actualizable y citable
P12	InstructGPT / RLHF	2022	L3	De completar texto a seguir instrucciones
P13	ReAct	2022	L2	El modelo controla un bucle que observa y actúa
P14	Toolformer	2023	L3	El uso de herramientas se autosupervisa
P15	DPO	2023	L4	Alineación sin modelo de recompensa ni RL
P16	Sistemas agentic	2023+	L5	El agente pasa de bucle a sistema



Tratamiento especial: *Attention Is All You Need*

El paper que sostiene casi todo lo que vino después se desmonta pieza por pieza en ocho miniaturas independientes, además de su ficha completa:

Miniatura	Qué aísla
To1	Por qué había que quitar la recurrencia
To2	Q, K, V y el producto escalar escalado
To3	Softmax, escala $\sqrt{d_k}$ y saturación
To4	Self-attention y máscara causal
To5	Multi-head attention
To6	Codificación posicional
To7	Residual, layer norm y feed-forward
To8	Encoder–decoder, complejidad y qué no dice el título



Cómo se usa

```
pip install -e .

ai-evolution papers          # catálogo del eje
ai-evolution paper P08       # ficha resumida de un paper
ai-evolution paper-lab P08 --seed 7 # ejecuta la miniatura
python scripts/generate_papers.py # regenera índice, notebooks y manifiesto
```

Con los notebooks:

```
jupyter lab notebooks/papers/
```

Empieza por `P01_perceptron.ipynb` y sigue el orden de la ruta. **Antes de ejecutar cada celda, escribe tu predicción** en la sección 7 — ese paso no es decorativo: es lo que se evalúa.

Estructura del eje

```
papers/
├─ README.md          ← este archivo
├─ ROADMAP.md         ← niveles L0-L5 y plan de estudio
├─ manifest.json      ← inventario con SHA-256 (generado)
├─ guides/            ← cómo leer, 5 pasadas, plantilla, glosario, fuentes
├─ catalog/
│   └─ papers.json    ← fuente de verdad estructurada
│   └─ sources.yaml   ← venues y repositorios primarios
│   └─ PAPERS_INDEX.md ← índice legible (generado)
└─ foundational/PXX_slug/ ← una ficha de 18 secciones por paper

notebooks/papers/     ← 16 + 8 notebooks (generados)
instructor/papers/    ← plan de sesión por paper (generado)
student/papers/       ← ficha de estudio y bitácora (generado)
assessments/papers/   ← evaluación con rúbrica por paper (generado)
prompts/              ← prompts reutilizables del eje
```

Cómo se conecta con el resto del programa

El eje **no sustituye** a las 183 clases: las ancla en su origen documental.

Este eje	Clases del programa
P01, P02	049 Perceptrón · 050 MLP y backpropagation
P03, P06	054 RNN, LSTM y secuencias
P04	053 CNN y aprendizaje espacial
P05	066 Embeddings semánticos
P07, P08	055 Atención y arquitectura Transformer
P09, P10	074 Objetivos de preentrenamiento
P11	105 RAG básico con citas
P12, P15	078 RLHF, RLAIIF y DPO
P13, P14	114 Ciclo ReAct · 080 Tool calling
P16	124 Workflow, subagente y multiagente · 132 MCP

Y con la clase que enseña precisamente esta competencia: 010 · Cómo leer papers, benchmarks y claims de IA.

 Reglas del eje (verificadas automáticamente)

1. **Contexto antes que tecnicismo.** Nadie ve una ecuación antes de saber qué problema resuelve.
2. **Progresión antes que saturación.** Un hito por vez, en orden.
3. **Interpretación antes que ejecución mecánica.** Predecir, ejecutar, explicar.
4. **Implementación pequeña y explicable** antes que frameworks.
5. **Nunca atribuir al paper ideas posteriores.** Sección 11 de cada ficha.
6. **Nunca inventar** autores, fechas, datasets, métricas ni citas.
7. **Siempre distinguir** hecho documentado, simplificación didáctica, inferencia y práctica moderna.
8. **Siempre registrar** URL, autoría, año, venue y fecha de consulta.
9. **Sin APIs pagadas** como requisito de aprendizaje.
10. **Sin redistribuir** material con restricciones de copyright.

 Verificación

```
python scripts/generate_papers.py --check
python -m unittest tests.test_papers -v
python scripts/validate_repository.py --strict
```

Se comprueba: JSON y YAML válidos, las 18 secciones de cada ficha en orden, los 17 momentos de cada notebook, `nbformat` correcto, que cada motor exista y sea determinista, que las clases enlazadas existan, ausencia de rutas absolutas y coherencia de los SHA-256 del manifiesto.

Ruta del eje de papers

Cómo recorrer los 16 hitos: qué nivel exige cada uno, en qué orden, con qué dedicación y qué evidencia deja cada tramo.

Los seis niveles

Nivel	Nombre	Qué se hace	Evidencia que produce
L0	Orientación	Entender qué es un paper, cómo se publica y cómo se cita	Una cita completa y bien formada
L1	Fundamentos	Leer la idea y su contexto histórico, sin ejecutar nada	Ficha de lectura de pasada 1–2
L2	Implementación	Escribir o ejecutar la miniatura del mecanismo	Notebook ejecutado con predicción previa
L3	Análisis	Comparar variantes, medir e interpretar	Experimento controlado con tres semillas
L4	Reproducción parcial	Replicar una figura, tabla o ablación a escala reducida	Figura reproducida + límites declarados
L5	Investigación	Leer la frontera con fecha y fuente, proponer preguntas abiertas	Nota de linaje fechada

Lo no está asignado a ningún paper: es la puerta de entrada. Se cubre con las tres guías (cómo leer, 5 pasadas, fuentes) y con la clase 010 del programa.

Plan de cuatro fases

● Fase 1 — Cómo aprende una máquina (P01–P04)

Duración sugerida: 2 semanas · **Niveles:** L1–L3

De la neurona que corrige su error a la red profunda que gana un certamen. Al terminar sabes por qué el conexionismo tuvo un invierno y qué lo sacó de él.

Paper	Nivel	Pregunta que responde
P01 Perceptrón	L1	¿Puede una máquina ajustar sus propios parámetros?
P02 Backpropagation	L2	¿Cómo se entrena lo que no se observa directamente?
P03 LSTM	L2	¿Cómo se recuerda a través del tiempo?
P04 AlexNet	L3	¿Qué hizo falta para que la profundidad escalara?

Entregable de fase: un informe de 2 páginas explicando el invierno de las redes neuronales usando **solo** evidencia de P01 y P02, sin narrativa retrospectiva.

● Fase 2 — Del vector a la secuencia (P05–P07)

Duración sugerida: 2 semanas · **Niveles:** L2–L3

Las palabras adquieren geometría, las secuencias se transforman en secuencias, y el cuello de botella del vector fijo aparece y se elimina.

Paper	Nivel	Pregunta que responde
Po5 Word2Vec	L2	¿Cómo se representa el significado sin definirlo?
Po6 Seq2Seq	L3	¿Cómo se mapea longitud variable a longitud variable?
Po7 Attention	L3	¿Cómo se deja de comprimir la entrada?

Entregable de fase: medir experimentalmente el cuello de botella (Po6) y demostrar que la atención lo elimina (Po7), con tres semillas y una conclusión que no exceda los datos.

● **Fase 3 — El bloque que lo cambió todo (P08–P11)**

Duración sugerida: 3 semanas · **Niveles:** L3–L4

El Transformer y sus dos descendencias, más la separación entre conocimiento y razonamiento. Es la fase más densa y la más rentable.

Paper	Nivel	Pregunta que responde
Po8 Transformer + To1–To8	L4	¿Y si quitamos la recurrencia por completo?
Po9 BERT	L3	¿Por qué mirar a ambos lados cambia la comprensión?
P10 GPT-3	L3	¿Qué aparece al escalar el decoder?
P11 RAG	L3	¿Cómo se cita lo que un modelo afirma?

Entregable de fase: las ocho miniaturas To1–To8 ejecutadas, más un texto de una página titulado «*qué NO dice el título del paper*».

● **Fase 4 — Del modelo al agente (P12–P16)**

Duración sugerida: 3 semanas · **Niveles:** L3–L5

Alineación, herramientas y sistemas. Aquí el eje entra en territorio con menos consenso, y eso se declara.

Paper	Nivel	Pregunta que responde
P12 InstructGPT	L3	¿Cómo se alinea un modelo con una intención?
P13 ReAct	L2	¿Cómo se ancla el razonamiento en observaciones reales?
P14 Toolformer	L3	¿Cómo se aprende <i>cuándo</i> usar una herramienta?
P15 DPO	L4	¿Hace falta RL para alinear?
P16 Sistemas agentic	L5	¿Qué convierte un bucle en un sistema?

Entregable de fase: una nota de linaje fechada sobre un tema de `frontier/current-topics.yaml`, con fuentes primarias y una declaración explícita de qué no está consolidado.

Dedicación estimada

Perfil	Ritmo	Alcance
Curioso	2 h/semana	Pasada 1–2 de todos los papers; miniaturas de Po1, Po7, Po8
Estudiante	6 h/semana	Ruta completa a nivel L2–L3, todas las miniaturas ejecutadas
Profesional	10 h/semana	L4 en Po8 y P15; reproducción parcial de dos figuras
Investigador	continuo	L5 permanente: pasada 5 con fecha en su subárea

Definición de terminado

Un estudiante ha terminado el eje cuando, **sin abrir los papers**, puede:

- [] ubicar cada hito en la evolución de la IA y decir qué lo precede y qué lo sigue;
- [] explicar qué problema resolvió cada uno y por qué el anterior no bastaba;
- [] ejecutar la miniatura del mecanismo e interpretar su salida;
- [] señalar al menos un límite del paper que sus autores **no** declararon;
- [] diferenciar el paper original de las prácticas modernas que se le atribuyen;
- [] reconocer un claim mal formulado y pedir los cinco datos que le faltan (tarea, dataset, métrica, línea base, condiciones);
- [] citar cualquiera de los 16 con autoría, año, venue, URL y fecha de consulta.

Trabajo pendiente del eje

Lo que aún no está y se declara como tal, en vez de fingir completitud:

Pendiente	Estado
Páginas HTML del eje dentro del sitio PWA	 no iniciado
PDF imprimible del eje completo	 no iniciado
Fichas de segunda línea (GloVe, ELMo, T5, Chinchilla, CLIP, difusión)	 no iniciado
Reproducción parcial guiada de una figura real de Po8	 no iniciado
Traducción de las fichas a inglés	 no iniciado

Los papers de frontera **no** se añaden a `foundational/`: se registran en `frontier/current-topics.yaml` hasta que se consoliden.



Cómo leer un paper de IA

Leer un paper no es leer un texto de principio a fin. Es un **interrogatorio**: entras con preguntas, buscas dónde se responden y decides si la respuesta te conviene.

Esta guía enseña qué preguntarle a un paper. El método en 5 pasadas enseña en qué **orden** hacerlo.



Anatomía de un paper y qué esconde cada parte

Sección	Qué dice	Qué NO dice	Trampa habitual
Título	La consigna del trabajo	No es un teorema	<i>Attention Is All You Need</i> no significa que baste la atención
Abstract	La versión más optimista	Las condiciones	Los resultados aparecen sin su línea base
Introducción	El problema y la promesa	Los fracasos previos de los autores	Presenta como nuevo lo que ya existía
Trabajo relacionado	Genealogía según los autores	Lo que ignoraron	Se citan a sí mismos con generosidad
Método	Lo que hay que reproducir	Los trucos no documentados	Un hiperparámetro decisivo en una nota al pie
Experimentos	Evidencia	Los experimentos que no salieron	Comparar contra una línea base débil
Ablaciones	Qué componente aporta qué	—	La sección más honesta; se salta casi siempre
Limitaciones	Los límites que los autores admiten	Los que no vieron	Puede faltar por completo
Apéndice	Los detalles que importan	—	Aquí vive lo que hace falta para reproducir

[!TIP] Si tienes 10 minutos, léete el **abstract**, mira las **figuras** y ve directo a las **ablaciones**. Ese recorrido te dice más que leer las 4 primeras páginas seguidas.



Las siete preguntas

Ninguna es sobre «qué dice el paper». Todas son sobre qué demuestra.

1. **¿Qué se hacía antes y por qué no bastaba?** Si no puedes responderla, no entiendes el paper: entiendes su solución fuera de contexto.
2. **¿Cuál es la afirmación central, en una frase?** Escríbela sin usar el vocabulario del paper. Si no puedes, aún no la entendiste.

3. **¿Qué evidencia la sostiene?** Tarea, dataset, métrica, línea base, número de ejecuciones, semillas, cómputo. Faltar cualquiera de estos seis es una debilidad, no un descuido de formato.
4. **¿Contra qué se compara?** Una mejora sobre una línea base mal ajustada no es una mejora. Comprueba si la línea base recibió el mismo esfuerzo de ajuste que el método propuesto.
5. **¿Qué se rompe si quito una pieza?** Es la pregunta de las ablaciones. Si no hay ablaciones, no sabes qué componente explica el resultado — y probablemente los autores tampoco.
6. **¿Qué NO demuestra?** Lo más valioso que puedes escribir sobre un paper. Casi siempre queda fuera del abstract.
7. **¿Qué haría falta para refutarlo?** Si no existe ningún resultado imaginable que lo contradiga, no es una afirmación empírica.

🚩 Señales de alarma

- **Métricas sin línea base.** «Alcanzamos 92 % de exactitud» no significa nada solo.
- **Una sola ejecución.** Sin varianza ni semillas, la diferencia puede ser ruido.
- **Benchmark contaminado.** Si el test pudo estar en el corpus de preentrenamiento, la métrica mide memorización, no capacidad.
- **Comparación entre desiguales.** Distinto cómputo, distintos datos, distinto ajuste.
- **Cherry-picking cualitativo.** Cinco ejemplos preciosos elegidos entre mil.
- **Ausencia de sección de limitaciones.** No significa que no las haya.
- **El código no existe, o existe y no reproduce las tablas.**
- **Un número que solo aparece en el abstract** y no se puede rastrear a ninguna tabla.

🕒 El error específico de leer papers históricos

Es el error que más penaliza este eje: **atribuir a un paper ideas que llegaron después.**

Se dice...	Pero en realidad...
«La LSTM tiene puerta de olvido desde 1997»	La añadieron Gers, Schmidhuber y Cummins en 1999/2000
«Rumelhart inventó la retropropagación»	La popularizó para redes; la diferenciación en modo reverso es de Linnainmaa (1970) y Werbos (1974)
«AlexNet inventó las CNN»	LeNet es de 1998; AlexNet demostró que escalaban
«El Transformer eliminó todo menos la atención»	También usa FFN, residuales, layer norm y codificación posicional
«GPT-3 aprende de los ejemplos del prompt»	Se condiciona ; no hay actualización de pesos

Un paper se lee **con la información que existía cuando se escribió**. La narrativa retrospectiva —«ya se veía venir»— es cómoda y casi siempre falsa.

Tres categorías que hay que separar siempre

Al escribir sobre un paper, cada frase pertenece a una de estas cajas. Etiquétalas:

Categoría	Ejemplo
Hecho documentado	«El paper reporta la configuración base con $N=6$ capas y $h=8$ cabezas (sección 3).»
Simplificación didáctica	«Q es lo que preguntas, K la etiqueta y V el contenido.»
Inferencia propia	«Probablemente la escala $\sqrt{d_k}$ importa más al crecer d_k .» ← verificable, pero mía
Práctica moderna posterior	«Hoy se usa RMSNorm y pre-norm en vez de post-norm.» ← no está en el paper

Mezclarlas es lo que convierte un resumen en desinformación con buena intención.

Cómo leer la matemática sin bloquearse

1. **Identifica los símbolos antes que las operaciones.** ¿Qué es un vector, qué una matriz, qué un escalar, qué un índice?
2. **Comprueba las dimensiones.** Si las formas no cuadran, has entendido mal algo. Es el depurador más rápido que existe.
3. **Evalúa el caso trivial.** ¿Qué pasa con $n=1$? ¿Con $d=1$? ¿Si el peso es 0? ¿Si es 1?
4. **Busca el caso límite.** ¿Qué ocurre cuando la secuencia es muy larga, la dimensión muy grande o la probabilidad se acerca a 0?
5. **Implementa la versión de 10 líneas.** La ecuación se entiende cuando corre.

Ese paso 5 es exactamente lo que hacen los notebooks de este eje.

Producto mínimo de una lectura

No has leído un paper hasta que puedes producir esto, sin volver a abrirlo:

- [] El problema anterior, en dos frases.
- [] La propuesta, en una.
- [] La ecuación o el diagrama central, dibujado de memoria.
- [] Una evidencia concreta con su tabla o figura de origen.
- [] Una limitación que el paper admite.
- [] Una limitación que el paper **no** admite.
- [] Una idea que hoy se le atribuye y llegó después.
- [] El hito siguiente que este paper hizo posible.

Esa lista es, literalmente, el esqueleto de la plantilla de ficha.

Referencia externa recomendada: S. Keshav, *How to Read a Paper* (ACM SIGCOMM CCR, 2007), origen del método de tres pasadas que este eje extiende a cinco. DOI



Dónde vive la investigación de IA

Regla del eje: una afirmación sobre un paper se sostiene con la **fuentes primaria**. Un blog, un hilo o un resumen generado por IA son pistas para encontrar el paper, nunca sustituto de haberlo abierto.

El error más frecuente al empezar es confundir **dónde se publica** con **dónde se busca**. Google Scholar no publica nada: indexa. arXiv no revisa nada: aloja. Saber qué hace cada sitio cambia cómo se cita y cuánta confianza merece lo que encuentras.



El mapa en una tabla

Sitio	Qué es realmente	Revisión por pares	Cuándo usarlo
arXiv	Repositorio de preprints	✗ No	Ver la investigación antes de que se publique formalmente
NeurIPS	Conferencia	✓ Sí	ML e IA en sentido amplio
ICML	Conferencia	✓ Sí	ML avanzado, teoría, optimización
ICLR	Conferencia	✓ Sí	Deep learning, representaciones, arquitecturas
ACL Anthology	Archivo abierto de ACL/NAACL/EMNLP	✓ Sí	PLN, LLM, traducción, evaluación lingüística
OpenReview	Plataforma de revisión abierta	✓ Sí (visible)	Leer las objeciones de los revisores y las réplicas
Semantic Scholar	Índice y grafo de citas	—	Rastrear qué papers citan a este
Google Scholar	Índice de citas	—	Encontrar versiones alternativas
Papers with Code	Enlace paper ↔ implementación	—	Buscar código, con reservas



arXiv — el primer lugar donde mirar

Es donde aparece primero casi toda la investigación relevante de IA, a menudo meses antes de la conferencia. Las categorías que importan para este programa:

Categoría	Contenido	Enlace
cs.AI	Inteligencia artificial en general	listado
cs.LG	Machine learning (la más activa)	listado
cs.CL	Lenguaje computacional: NLP, LLM, traducción	listado
cs.CV	Visión por computador	listado
stat.ML	Machine learning desde la estadística	listado

[!WARNING] Un **preprint no ha pasado revisión por pares**. Puede contener errores, ser retirado o cambiar entre versiones (v1 , v2 , v3). Cita siempre la versión que leíste. Un paper muy citado en arXiv que nunca fue aceptado en ningún venue es una señal a investigar, no a ignorar — pero es una señal.

Conferencias — el filtro de la comunidad

En IA, la publicación de referencia es la **conferencia**, no la revista. Es una diferencia cultural con otras disciplinas y explica los ciclos rápidos del campo.

- **NeurIPS** — la más grande y transversal. Publicó AlexNet (2012), el Transformer (2017), GPT-3, RAG, InstructGPT, Toolformer y DPO.
- **ICML** — machine learning avanzado; más peso en teoría y optimización.
- **ICLR** — nació en torno al deep learning y las representaciones. Usa OpenReview, así que todo su proceso editorial es público.
- **ACL / NAACL / EMNLP** — el mundo del lenguaje. Su archivo, **ACL Anthology**, es de acceso abierto y permanente: si un paper de PLN está ahí, esa es la cita canónica.

OpenReview — el sitio infravalorado

Es el único lugar donde se ve el paper y **su discusión**: qué objetaron los revisores, qué respondieron los autores, qué cambió entre versiones y por qué se aceptó o rechazó.

Para aprender a leer críticamente vale tanto como el paper. Ejercicio de nivel L3 de este eje: **buscar una revisión negativa de un paper aceptado y decidir si la objeción quedó realmente resuelta o solo respondida con elegancia**.

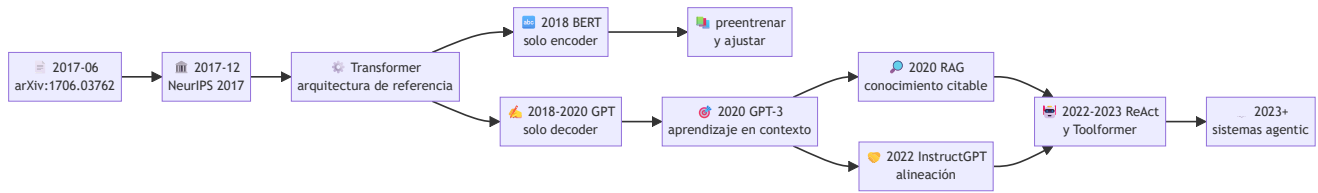
Buscadores — para rastrear, no para citar

Semantic Scholar y **Google Scholar** sirven para reconstruir linajes: quién citó a quién, qué vino antes y qué vino después. Semantic Scholar además expone una **API pública** que permite automatizar la vigilancia.

[!CAUTION] El número de citas mide **atención**, no calidad ni corrección. Papers muy citados han sido posteriormente matizados o refutados; papers poco citados han resultado fundamentales años después.

La ruta histórica, contada como circulación entre fuentes

Así viajó la idea que sostiene casi todo lo que usas hoy:



Cada flecha del diagrama es un paper concreto con su ficha en este eje: índice completo.

Higiene de citación (obligatoria en este eje)

1. Registra **autores, año, título, venue, URL y fecha de consulta**.
2. Cita la **versión** que leíste (arXiv:1706.03762v5 , no solo arXiv:1706.03762).
3. Prefiere el venue revisado si existe; usa arXiv cuando no exista o para la versión extendida.
4. **No cites un paper que no abriste**. Se detecta preguntando por el número de una figura.
5. No conviertas un número de un resumen ajeno en un dato tuyo sin verlo en la tabla original.
6. Distingue siempre: **hecho documentado** / **simplificación didáctica** / **inferencia propia**.

Copyright y acceso

Este repositorio **enlaza** a las fuentes; no aloja PDFs de terceros con licencia restrictiva.

Los tres papers más antiguos del eje (Po1 Rosenblatt, Po2 Rumelhart y Po3 Hochreiter) están en revistas de suscripción. Vías legítimas: la versión de acceso abierto del autor, el repositorio institucional de su universidad, o una biblioteca. La versión en ACL Anthology siempre es abierta cuando existe.

Los datos de esta guía están también en formato legible por máquina en `catalog/sources.yaml` , con fecha de revisión.

 Eje de papers ·  Cómo leer un paper ·  Método en 5 pasadas ·  Plantilla de ficha ·  Glosario

Glosario del eje de papers

Términos que aparecen **leyendo papers**: vocabulario de publicación, de evaluación y de los mecanismos de la ruta mínima. Para el vocabulario general del programa (agentes, harness, context engineering, MLOps...) usa el glosario principal.

Cada entrada marca su tipo:  concepto técnico ·  proceso editorial ·  evaluación ·  trampa.

A

Ablación  — Quitar un componente y medir cuánto empeora el resultado. Es la única forma de saber qué pieza explica la mejora. Un paper sin ablaciones afirma que su método funciona, pero no por qué.

Abstract  — Resumen de 150–250 palabras. Contiene la versión más favorable de los resultados: casi nunca menciona la línea base ni las condiciones.

Alineación (alignment)  — Hacer que el comportamiento del modelo coincida con la intención humana. Ver P12 y P15.

Anacronismo de atribución  — Adjudicar a un paper una idea que apareció después. El error más penalizado en este eje. Ejemplo: la puerta de olvido de la LSTM.

arXiv  — Repositorio de preprints. **Sin revisión por pares.** Ver fuentes y venues.

Atención  — Mecanismo que calcula pesos que suman 1 sobre un conjunto de elementos y devuelve su combinación ponderada. Introducida para traducción en PO7.

Aprendizaje en contexto (in-context learning)  — Adaptación al condicionar con ejemplos en el prompt, **sin actualizar pesos.** Ver P10.

Autorregresivo  — Modelo que genera el elemento t condicionado a los anteriores: $p(x_t \mid x_{<t})$. La familia GPT.

B

Benchmark  — Conjunto estandarizado de tareas y métricas. Útil para comparar; peligroso cuando se optimiza directamente contra él (ley de Goodhart).

BLEU  — Métrica de traducción basada en solapamiento de n-gramas con referencias humanas. Correlaciona con calidad de forma imperfecta y no es comparable entre papers que usan distinta tokenización.

Bradley-Terry  — Modelo estadístico de comparaciones por pares: $p(a > b) = \sigma(r(a) - r(b))$. Base del modelo de recompensa en RLHF.

C

Camino máximo  — Número de pasos que debe recorrer una señal entre dos posiciones de una secuencia. $O(n)$ en un RNN, $O(1)$ en self-attention. Explica por qué el Transformer entrena mejor en dependencias largas.

Carrusel de error constante (CEC)  — Ruta aditiva de la celda LSTM por la que el gradiente viaja sin multiplicarse por activaciones. Ver P03.

Contaminación de benchmark  — Que los datos de test hayan aparecido en el corpus de entrenamiento. Convierte la métrica en una medida de memorización.

Cross-attention  — Atención donde las consultas vienen de una secuencia y las claves y valores de otra. Es lo que conecta decoder con encoder.

D

DPO (Direct Preference Optimization)  — Alinear con preferencias sin modelo de recompensa ni RL. Ver P15.

DOI  — Identificador persistente de una publicación. Preferible a una URL cuando existe.

E

Embedding  — Representación vectorial densa. Ver P05.

Entropía de la atención  — Cuánto se reparten los pesos α . Baja = atención concentrada; alta = repartida. Un softmax saturado tiene entropía casi nula y gradientes casi nulos.

Escalado (scaling laws)  — Relación empírica entre cómputo, datos, parámetros y pérdida. Kaplan et al. (2020) y Hoffmann et al. (2022).

F

Few-shot / one-shot / zero-shot  — Protocolos de evaluación según cuántos ejemplos se incluyen en el prompt. **No** son formas de entrenamiento.

Fine-tuning  — Ajustar los pesos de un modelo preentrenado sobre una tarea concreta. Distinto de condicionar con un prompt.

FFN por posición  — Red densa aplicada de forma independiente a cada posición dentro de un bloque Transformer. Concentra la mayoría de los parámetros del bloque.

G

Gradiente desvaneciente  — El gradiente se hace exponencialmente pequeño al propagarse por muchas capas o pasos temporales, y las primeras capas dejan de aprender.

GLUE  — Colección de tareas de comprensión del lenguaje usada para evaluar BERT y sucesores.

H

Hipótesis distribucional 🧱 — «Una palabra se caracteriza por la compañía que mantiene» (Firth, 1957; Harris, 1954). Fundamento de los embeddings.

Hoja de ruta del hito 🧱 — En este eje, la cadena problema → propuesta → límite → siguiente paper. Es lo que evita estudiar los papers como una lista inconexa.

K

KL (divergencia) 🧱 — Medida de cuánto se aleja una distribución de otra. En RLHF penaliza que la política se separe del modelo base; en DPO aparece implícita en el log-ratio.

L

Layer normalization 🧱 — Normaliza cada vector de activación a media 0 y varianza 1. Sin su epsilon, un vector constante produce división por cero.

Línea base (baseline) 🧱 — El método contra el que se compara. **Un resultado sin línea base no es un resultado.** Comprueba si recibió el mismo esfuerzo de ajuste.

M

Máscara causal 🧱 — Impide que la posición i atienda a posiciones futuras. Se aplica **antes** del softmax, poniendo los scores a $-\infty$.

Memoria paramétrica / no paramétrica 🧱 — Lo que el modelo sabe en sus pesos frente a lo que consulta en un índice externo. La separación que propone RAG.

MLM (Masked Language Modeling) 🧱 — Predecir tokens enmascarados usando contexto bidireccional. Objetivo de preentrenamiento de BERT.

O

OpenReview 🏛️ — Plataforma donde el proceso de revisión es público: revisiones, réplicas y decisión. La mejor escuela de lectura crítica disponible gratis.

Overclaiming ⚠️ — Afirmar más de lo que la evidencia sostiene. Forma más común: convertir una mejora en un benchmark en una afirmación sobre «comprensión» o «razonamiento».

P

Peer review (revisión por pares) 🏛️ — Evaluación por investigadores del área antes de publicar. arXiv no la tiene; NeurIPS, ICML, ICLR y ACL sí.

Preprint 🏛️ — Versión previa a la revisión formal. Citable, pero declarando que lo es.

Producto escalar escalado  — $QK^T / \sqrt{d_k}$. La división evita que el softmax se sature al crecer la dimensión.

Positional encoding  — Información de orden que se **suma** al embedding, porque la atención por sí sola es indiferente a las permutaciones.

R

RAG  — Recuperar documentos y generar condicionando en ellos. Ver P11.

Reproducibilidad  — Que otra persona, con el paper y el código, obtenga el mismo resultado. Requiere semillas, versiones, datos y cómputo declarados.

Residual (conexión)  — $x + \text{Sublayer}(x)$. Camino identidad que permite apilar decenas o cientos de capas sin que la señal se apague.

Reward hacking  — La política maximiza el modelo de recompensa explotando un atajo (por ejemplo, ser más larga) sin mejorar la calidad real.

RLHF  — Ajuste con retroalimentación humana en tres etapas: SFT, modelo de recompensa y RL con penalización KL. Ver P12.

S

Self-attention  — Atención de una secuencia sobre sí misma.

Semilla (seed)  — Valor que fija la aleatoriedad. Reportar resultados sin semilla ni número de ejecuciones impide distinguir mejora de ruido.

Softmax  — Convierte un vector de scores en una distribución de probabilidad. Se implementa restando el máximo para evitar desbordamiento.

SOTA (state of the art)  — «Estado del arte». Caduca. Sin fecha y benchmark concretos, la palabra no informa de nada.

T

Tool use  — Que el modelo invoque funciones externas. Aprendido de forma autosupervisada en P14.

Transformer  — Arquitectura basada solo en atención, FFN, residuales, layer norm y codificación posicional. Ver Po8.

V

Venue  — Dónde se publicó: conferencia, revista o repositorio. Parte obligatoria de una cita.

Varianza entre ejecuciones  — Diferencia de resultados al cambiar solo la semilla. Si la mejora reportada es menor que la varianza, no hay mejora demostrada.

Método de lectura en 5 pasadas

Origen y honestidad de la fuente: el método de **tres pasadas** es de S. Keshav, *How to Read a Paper* (ACM SIGCOMM CCR, 2007) — DOI. Las **pasadas 4 y 5** son una extensión pedagógica de este programa, no del paper de Keshav. Se marcan como tales para no atribuirle algo que no escribió.

La idea central: **nunca leas un paper una sola vez de principio a fin**. Haz varias pasadas, cada una con un objetivo distinto, y decide después de cada una si merece la siguiente. La mayoría de los papers no pasan de la pasada 1, y eso está bien.

El método de un vistazo

Pasada	Tiempo	Objetivo	Salida	¿Sigo?
1	10 min	¿De qué va y me sirve?	5 frases	Si no me sirve, paro
2	1 h	¿Qué propone y con qué evidencia?	Ficha secciones 1–9	Si no lo voy a usar ni evaluar, paro
3	4–5 h	¿Podría reimplementarlo?	Miniatura ejecutable	Si no necesito el detalle, paro
4	1–2 días	¿Se sostiene al replicarlo?	Figura o ablación reproducida	Solo para papers que van a fundamentar trabajo propio
5	continuo	¿Dónde encaja hoy?	Nota de linaje con fecha	Solo para el núcleo de tu área

1 Pasada 1 — Reconocimiento (10 minutos)

Lee: título, abstract, introducción, encabezados de sección, figuras, conclusiones, referencias (solo por encima). **No leas:** método, ecuaciones, tablas de resultados.

Al terminar responde las **cinco C**:

1. **Categoría** — ¿es un método nuevo, un análisis, un dataset, una evaluación, una posición?
2. **Contexto** — ¿con qué trabajos dialoga?
3. **Corrección** — ¿los supuestos parecen razonables a primera vista?
4. **Contribución** — ¿qué aporta, en una frase?
5. **Claridad** — ¿está bien escrito? (mala escritura correlaciona con trabajo confuso)

Criterio de parada: si no responde a una pregunta que tienes, cierra. Guarda la referencia y sigue. Leer todo lo que llega no es rigor: es falta de criterio.

2 Pasada 2 — Comprensión (1 hora)

Lee: todo el cuerpo con atención a figuras y tablas. **Ignora** las demostraciones.

Objetivos concretos:

- Escribir la **afirmación central** sin usar el vocabulario del paper.
- Identificar **tarea, dataset, métrica y línea base** de cada resultado principal.
- Marcar las referencias que necesitarás leer.
- Anotar cada punto que no entendiste (no lo disimules: la lista es tu plan de estudio).

Al terminar deberías poder **describir el paper a otra persona** con sus evidencias principales. Con esto ya puedes rellenar las secciones 1 a 9 de la ficha.

[!WARNING] Si al final de esta pasada no puedes nombrar la línea base, no has entendido el resultado: has memorizado un número.

3 Pasada 3 — Reimplementación mental (4–5 horas)

El objetivo: poder reconstruir el trabajo con los mismos supuestos.

Método: haz de nuevo el paper. Asume lo que asumen los autores y recrea el trabajo. Comparar tu reconstrucción con la real revela no solo las innovaciones del paper, sino sus supuestos ocultos y sus fallos.

En este eje, la pasada 3 tiene una salida obligatoria y concreta: **la miniatura ejecutable**, que ya está construida en `notebooks/papers/`. Tu trabajo es predecir la salida antes de ejecutarla y explicar cualquier diferencia.

Preguntas de esta pasada:

- ¿Qué hiperparámetro decide el resultado, y está documentado?
- ¿Qué pasaría si cambio este componente por el más simple posible?
- ¿Puedo derivar la ecuación central en una servilleta?

4 Pasada 4 — Reproducción parcial (*extensión de este programa*)

No siempre es posible reproducir un paper: el cómputo original puede costar cientos de miles de dólares. Pero casi siempre es posible reproducir **algo**:

- una **figura** con datos sintéticos o un subconjunto público;
- una **ablación** a escala reducida;
- una **tendencia** (que la métrica suba con el tamaño, aunque no llegues a sus valores).

Regla de honestidad: una tendencia reproducida a escala 1/1000 es evidencia de que entendiste el mecanismo, **no** de que el resultado del paper sea correcto. Decláralo así.

Salida: un notebook con semilla fija, la figura reproducida y una frase sobre qué parte del resultado original queda fuera de tu alcance y por qué.

5 Pasada 5 — Linaje y estado del arte (*extensión de este programa*)

Un paper no es un punto: es un nodo. Esta pasada lo sitúa.

1. Busca en Semantic Scholar qué papers lo citan.
2. Identifica: quién lo **extendió**, quién lo **contradijo** y quién lo **reemplazó**.
3. Comprueba si su resultado principal **sigue vigente** o fue matizado.
4. Anota la **fecha de consulta**. Esta pasada caduca.

Salida: una nota de linaje de 5 líneas con fecha, que en este repositorio vive en la sección 13 de cada ficha y, para lo aún no consolidado, en `frontier/current-topics.yaml`.

Cuánto invertir, según para qué

Tu objetivo	Pasada suficiente
Saber si existe algo relevante	1
Citarlo en un informe	2
Explicarlo en clase	3
Construir sobre él	4
Investigar en su área	5

Autoevaluación del método

- ☐ He parado en la pasada 1 al menos una vez esta semana (señal de tener criterio).
- ☐ Puedo nombrar la línea base de los últimos tres papers que leí.
- ☐ Tengo una lista escrita de lo que no entendí, no una sensación difusa.
- ☐ Antes de ejecutar cualquier miniatura, escribí mi predicción.
- ☐ Sé distinguir, en mis propias notas, hecho documentado de inferencia mía.



Plantilla de ficha de paper

Contrato de 18 secciones. Es **obligatorio y verificado automáticamente**: cada ficha de papers/foundational/ debe tener estas 18 secciones, con estos títulos exactos y en este orden. Lo comprueba `ai_evolution.papers.validate_papers()`.

Copia el bloque de abajo para dar de alta un paper nuevo. Después registra su entrada en `catalog/papers.json` y ejecuta:

```
python scripts/generate_papers.py
```

Por qué 18 secciones

Cada una responde a una pregunta que un resumen normal se salta:

Sección	Pregunta que fuerza a responder
1–3	¿Qué había antes, qué propone y quién lo firma?
4–6	¿Lo entiendo sin fórmulas, con fórmulas y como flujo?
7–8	¿Dónde está la evidencia en el paper original ?
9–11	¿Qué cambió, qué no demuestra y en qué se equivoca la gente?
12–13	¿De qué depende y qué hizo posible?
14–17	¿Cómo lo ejecuto, lo practico y me autoevalúo?
18	¿De dónde salió cada afirmación?



Plantilla (copiar desde aquí)

PXX – Título en español

> Una frase que sitúe el hito en la evolución de la IA.

****Nivel:**** L? . ****Motor:**** `nombre` . ****Notebook:**** enlace relativo a `notebooks/papers/PXX\_slug.ipynb`

1. Identificación

Campo Valor
--- ---
Título original
Autoría
Año
Venue
Fuente primaria
Acceso abierto / restringido
Fecha de consulta AAAA-MM-DD

2. Problema anterior

Qué se hacía antes y por qué no bastaba. ****Sin mencionar la solución del paper.****

3. Propuesta

La contribución en 3-5 frases. Qué es nuevo y qué reutiliza de trabajos previos.

4. Intuición sin fórmulas

Una analogía honesta. Debe declarar dónde deja de funcionar.

5. Matemática mínima

Solo las ecuaciones imprescindibles, con todos los símbolos definidos.

6. Arquitectura o flujo

Diagrama (mermaid o bloque `text`) del mecanismo, entrada a salida.

7. Qué observar en el paper original

Secciones, figuras y tablas concretas que hay que mirar, con su número.

8. Evidencia y resultados

Qué demostró, con tarea, dataset, métrica y línea base. Si un número no se ha verificado en la fuente, se dice explícitamente en lugar de escribirlo.

9. Impacto

Qué cambió después. Distinguir impacto documentado de narrativa retrospectiva.

10. Limitaciones

Las que el paper admite ****y**** las que no admite. Mínimo tres.

11. Errores comunes

Malentendidos frecuentes, especialmente atribuciones anacrónicas.

12. Relación con trabajos anteriores

De qué depende. Con referencia y año.

13. Relación con trabajos posteriores

Qué hizo posible. Con referencia y año.

14. Notebook asociado

Qué implementa la miniatura, qué ****no**** implementa y cómo ejecutarla.

15. Actividades Bloom

Seis actividades: recordar, explicar, aplicar, analizar, evaluar y crear.

16. Autoevaluación

Entre 5 y 8 preguntas. Ninguna de definición pura.

17. Respuestas esperadas

Qué debe contener una buena respuesta. No una respuesta modelo para copiar.

18. Fuentes primarias

Lista con autores, año, venue, URL y fecha de consulta.

✓ Reglas de calidad no negociables

1. **No inventar** autores, fechas, datasets, métricas ni citas. Si no lo verificaste, escribe «verificar en la tabla N del paper» en lugar del número.
2. **No atribuir** al paper ideas posteriores (sección 11 existe para eso).
3. **Marcar** siempre qué es hecho documentado, simplificación didáctica, inferencia propia o práctica moderna.
4. **Fuente primaria obligatoria** con URL y fecha de consulta.
5. **No redistribuir** PDFs con restricciones de copyright: se enlaza.
6. **Sin rutas absolutas** en ningún ejemplo ni notebook.
7. **Limitaciones antes que entusiasmo**: la sección 10 no puede quedar en una línea.

🔍 Cómo se verifica

```
python -c "import sys; sys.path.insert(0,'src'); from ai_evolution.papers import validate_papers; print(validate_papers(strict=True))"
```

Comprueba: las 18 secciones y su orden, la existencia del notebook y sus 17 momentos, que el motor exista, que haya fuente primaria con URL, que las clases enlazadas existan y que los SHA-256 del manifiesto estén al día.

P01 — El perceptrón

El momento en que una máquina deja de ejecutar reglas escritas por una persona y empieza a ajustar sus propios parámetros a partir de ejemplos.

Nivel: L1 · Motor: perceptron · Notebook: P01_perceptron.ipynb

1. Identificación

Campo	Valor
Título original	<i>The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain</i>
Autoría	Frank Rosenblatt
Año	1958
Venue	<i>Psychological Review</i> , 65(6), 386–408
Fuente primaria	doi.org/10.1037/h0042519
Acceso	Restringido (revista de suscripción)
Fecha de consulta	2026-08-16

2. Problema anterior

En los años 50 un programa hacía exactamente lo que su autor había escrito. McCulloch y Pitts (1943) habían mostrado que una neurona de umbral podía computar funciones lógicas, pero sus pesos los fijaba el diseñador: la red no cambiaba con la experiencia. Hebb (1949) había propuesto que el aprendizaje biológico era plasticidad sináptica, pero como principio, no como algoritmo.

Faltaba lo del medio: **un procedimiento concreto y ejecutable para que un sistema modifique su comportamiento observando datos etiquetados**. Sin eso, «aprender» era una metáfora.

3. Propuesta

Rosenblatt propone una unidad que:

1. recibe entradas x , las pondera con w y suma un sesgo b ;
2. decide con una función escalón: positivo $\rightarrow$ clase 1, negativo $\rightarrow$ clase 0;
3. y —la parte nueva— **corrige w y b solo cuando se equivoca**, empujando el hiperplano hacia el ejemplo mal clasificado.

Lo enmarca como modelo probabilístico de almacenamiento de información en el cerebro, no como herramienta de ingeniería. El Mark I Perceptron lo implementó en hardware con conexiones ajustables físicamente.

4. Intuición sin fórmulas

Imagina puntos de dos colores sobre una hoja y una regla que intenta separarlos. Cada vez que un punto queda del lado equivocado, empujas la regla un poco hacia él. Si existe alguna posición de la regla que los separe, este procedimiento la encuentra.

Dónde deja de funcionar la analogía: si no existe ninguna recta que los separe, la regla no se «acerca poco a poco» a una solución aproximada — oscila indefinidamente. No hay degradación elegante.

5. Matemática mínima

$$z = w \cdot x + b = \sum_i w_i x_i + b$$

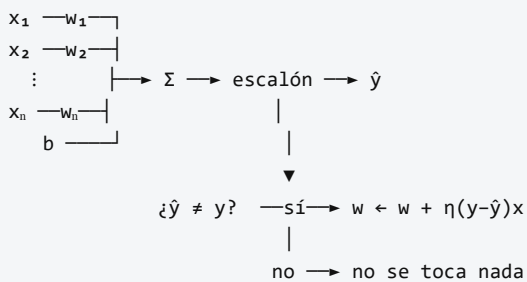
$$\begin{aligned}\hat{y} &= 1 & \text{si } z \geq 0 \\ \hat{y} &= 0 & \text{si } z < 0\end{aligned}$$

Regla de actualización (solo si $\hat{y} \neq y$):

$$\begin{aligned}w &\leftarrow w + \eta(y - \hat{y})x \\ b &\leftarrow b + \eta(y - \hat{y})\end{aligned}$$

- $x \in \mathbb{R}^n$ entrada · $w \in \mathbb{R}^n$ pesos · $b \in \mathbb{R}$ sesgo · $\eta > 0$ tasa de aprendizaje.
- $w \cdot x + b = 0$ define un **hiperplano**: la frontera de decisión.
- **Teorema de convergencia** (Novikoff, 1962 — posterior al paper): si los datos son linealmente separables con margen $\gamma > 0$ y radio R , el algoritmo converge en a lo sumo $(R/\gamma)^2$ correcciones.

6. Arquitectura o flujo



La última línea es la esencia del paper: **sin error no hay aprendizaje**.

7. Qué observar en el paper original

- La **motivación biológica**: Rosenblatt escribe para psicólogos, no para ingenieros. El vocabulario es de sistemas nerviosos, no de optimización.
- El planteamiento **probabilístico** de la conectividad, hoy poco recordado frente a la versión algorítmica simplificada que circula en los manuales.
- La ausencia de: función de pérdida diferenciable, descenso de gradiente y capas ocultas entrenables. Ninguna de esas ideas está en este trabajo.

8. Evidencia y resultados

El artículo es teórico y conceptual, acompañado del programa experimental del Mark I. La demostración formal de convergencia para datos separables es **posterior** (Novikoff, 1962; Block, 1962).

Verificar en la fuente primaria antes de citar cualquier cifra de desempeño: este eje no reproduce números de los experimentos originales de 1958 porque no fueron formulados con el protocolo de tarea/dataset/métrica/línea base que hoy se exige.

La miniatura de este eje sí produce evidencia verificable, con otro alcance: AND converge en 6 épocas con 11 correcciones; XOR no converge en 200 épocas.

9. Impacto

- Inaugura el **conexionismo** como programa de investigación.
- Es el ancestro directo de toda neurona artificial posterior: la forma $\sigma(w \cdot x + b)$ es la misma en un MLP, en una CNN y en la FFN de un Transformer.
- Su límite provocó el primer invierno de las redes neuronales tras el análisis de Minsky y Papert (1969) — un impacto negativo que resultó igual de determinante que el positivo.

Cuidado con la narrativa retrospectiva: la historia habitual («Minsky mató las redes neuronales») es una simplificación. El libro de 1969 es un análisis matemático riguroso de qué puede y qué no puede computar un perceptrón; la interpretación institucional que se hizo de él es otra cosa.

10. Limitaciones

1. **Solo fronteras lineales.** Si las clases no son linealmente separables, no hay solución dentro de la clase de hipótesis. XOR es el contraejemplo mínimo.
2. **No degrada con elegancia.** Con datos no separables no converge a una «mejor aproximación»: oscila. No devuelve nada útil.
3. **Sin noción de margen.** Cualquier hiperplano que separe le vale; no busca el mejor. Eso lo resolverán las SVM tres décadas después.
4. **Sin probabilidades.** La salida es 0 o 1: no expresa confianza.
5. **Sensible a la escala** de las características y al orden de presentación de los ejemplos.
6. **Sin capas ocultas entrenables.** El paper no ofrece forma de entrenar representaciones intermedias.

11. Errores comunes

Error	Corrección
«El perceptrón usa descenso de gradiente»	Usa una regla de corrección de error . La función escalón no es diferenciable.
«Minsky y Papert demostraron que las redes neuronales no sirven»	Demostraron límites del perceptrón de una capa . El propio libro discute redes multicapa.
«El teorema de convergencia está en el paper de 1958»	Es de Novikoff y Block, 1962.
«El perceptrón fracasó»	Funcionó exactamente como su teoría predecía. Lo que falló fueron las expectativas construidas sobre él.
«No converge porque le faltan épocas»	Con XOR no existe solución que alcanzar. Es un límite de capacidad, no de presupuesto.

12. Relación con trabajos anteriores

- **McCulloch y Pitts (1943)** — neurona de umbral con pesos fijos. Rosenblatt añade el aprendizaje.
- **Hebb (1949)** — la plasticidad sináptica como mecanismo de aprendizaje. Rosenblatt la convierte en un algoritmo con señal de error supervisada.

13. Relación con trabajos posteriores

- **Minsky y Papert (1969)** — *Perceptrons*: formalizan qué no puede computar.
- **Novikoff (1962)** — teorema de convergencia con margen.
- **Po2 Backpropagation (1986)** — resuelve el límite entrenando capas ocultas.
- **Vapnik y colaboradores (1992–1995)** — SVM: separadores lineales con margen máximo y kernels, otra respuesta al mismo límite.

14. Notebook asociado

P01_perceptron.ipynb

Qué implementa: la regla de aprendizaje original en 20 líneas, sobre AND y XOR, con historial de errores por época.

Qué NO implementa: el modelo probabilístico de conectividad del paper, el hardware Mark I, ni ningún experimento de percepción visual.

```
ai-evolution paper-lab P01 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la regla de actualización y define cada símbolo sin mirar.
Explicar	Explica por qué la regla no hace nada cuando la predicción es correcta, y qué consecuencia tiene.
Aplicar	Ejecuta el perceptrón sobre OR y NAND. Anota épocas hasta converger.
Analizar	Demuestra algebraicamente que XOR no es linealmente separable (4 desigualdades).
Evaluar	Un compañero afirma: «con más épocas, XOR converge». Refuta con evidencia experimental y con el argumento algebraico.
Crear	Diseña un conjunto de datos 2D donde el perceptrón converja muy lentamente sin dejar de ser separable, y explica qué propiedad geométrica lo provoca.

16. Autoevaluación

1. ¿Por qué la función escalón impide usar descenso de gradiente?
2. Con $w = (0, 0)$, $b = 0$ y $\eta = 1$, ejecuta a mano dos épocas de AND. ¿Qué valores obtienes?
3. ¿Qué garantiza exactamente el teorema de convergencia, y bajo qué condición?
4. ¿Qué le ocurre a la cota $(R/\gamma)^2$ cuando el margen γ tiende a cero? ¿Qué significa eso en la práctica?
5. Si añades la característica $x_3 = x_1 \cdot x_2$ a XOR, ¿se vuelve separable? ¿Qué te dice eso sobre la relación entre representación y separabilidad?
6. Nombra dos límites del perceptrón que **no** sean «no resuelve XOR».
7. ¿Qué idea que hoy se asocia al perceptrón apareció en realidad después de 1958?

17. Respuestas esperadas

1. La derivada del escalón es 0 en todas partes salvo en el salto, donde no existe. Sin gradiente informativo, no hay dirección de descenso.
2. Debe llegar a valores intermedios y converger hacia $w = (2, 1)$, $b = -3$ alrededor de la sexta época. Lo evaluable es el **procedimiento** paso a paso, no el número final.
3. Que si existe un hiperplano separador con margen $\gamma > 0$, el algoritmo hace un número **finito** de correcciones, acotado por $(R/\gamma)^2$. No garantiza nada si no hay separabilidad.
4. La cota diverge. Datos casi colineales o casi solapados pueden necesitar un número enorme de correcciones aun siendo técnicamente separables: la garantía existe pero es inútil.
5. Sí. Muestra que la separabilidad no es propiedad de los datos sino del **espacio de representación**. Es la idea que fundamenta tanto los kernels como las capas ocultas.
6. Se aceptan: ausencia de margen máximo, salida no probabilística, no degradación elegante, sensibilidad a la escala, dependencia del orden de presentación.
7. El teorema de convergencia (1962), el descenso de gradiente sobre pérdidas diferenciables, la idea de margen máximo y cualquier mención a capas ocultas entrenables.

18. Fuentes primarias

- Rosenblatt, F. (1958). *The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain*. **Psychological Review**, 65(6), 386–408. doi.org/10.1037/h0042519 · consultado 2026-08-16.

- Minsky, M. y Papert, S. (1969). *Perceptrons*. MIT Press. [ficha del editor](#) · consultado 2026-08-16.
- McCulloch, W. y Pitts, W. (1943). *A Logical Calculus of the Ideas Immanent in Nervous Activity*. **Bulletin of Mathematical Biophysics**, 5, 115–133. doi.org/10.1007/BF02478259 · consultado 2026-08-16.

Evaluación — P01

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain* (1958, Psychological Review, 65(6), 386–408) · **Nivel:** L1

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P01_perceptron.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P02 — Backpropagation

El procedimiento que hizo entrenables las capas ocultas: la red deja de recibir las representaciones y empieza a descubrirlas.

Nivel: L2 · Motor: `backprop` · Notebook: `P02_backpropagation.ipynb`

1. Identificación

Campo	Valor
Título original	<i>Learning representations by back-propagating errors</i>
Autoría	David E. Rumelhart, Geoffrey E. Hinton, Ronald J. Williams
Año	1986
Venue	<i>Nature</i> , 323, 533–536
Fuente primaria	doi.org/10.1038/323533a0
Acceso	Restringido (revista de suscripción)
Fecha de consulta	2026-08-16

2. Problema anterior

El perceptrón solo traza fronteras lineales. La solución evidente —apilar capas— chocaba con un obstáculo práctico: **¿cuánta culpa del error final tiene un peso de la capa intermedia?** Ese peso no está conectado directamente con la salida; su efecto pasa por todas las unidades que vienen después.

Se le llamó el *credit assignment problem*. Sin resolverlo, las capas ocultas eran una idea sin algoritmo: se podían dibujar, no entrenar.

3. Propuesta

Aplicar la **regla de la cadena** recorriendo la red hacia atrás:

- propagación hacia adelante: se calcula la salida y la pérdida;
- propagación hacia atrás: el error de cada capa se expresa en función del error de la siguiente, multiplicando por los pesos y por la derivada de la activación;
- actualización: cada peso se mueve en la dirección opuesta a su gradiente.

La aportación decisiva del artículo no es la fórmula —ya existía en otras formulaciones— sino **mostrar que funciona en la práctica y que las unidades ocultas aprenden representaciones internas útiles y no diseñadas por nadie**. El título lo dice: *learning representations*.

4. Intuición sin fórmulas

Una cadena de montaje produce una pieza defectuosa. El responsable de calidad no reparte la culpa al azar: retrocede puesto por puesto preguntando «¿cuánto habría cambiado el defecto si tú hubieras hecho tu parte un poco distinta?». Cada puesto recibe una responsabilidad proporcional a su influencia real.

Dónde deja de funcionar la analogía: la cadena de montaje tiene un número fijo de puestos con influencia comparable. En una red profunda, la influencia se multiplica en cada paso, y ese producto puede colapsar a cero o dispararse — el problema que Po3 tendrá que atacar.

5. Matemática mínima

Red $x \rightarrow h = \sigma(w_1x + b_1) \rightarrow o = \sigma(w_2h + b_2)$, pérdida $L = (o - y)^2$, con $\sigma(z) = 1/(1+e^{-z})$ y $\sigma'(z) = \sigma(z)(1 - \sigma(z))$:

$$\delta_o = \partial L / \partial o_{in} = 2(o - y) \cdot \sigma'(o_{in})$$

$$\partial L / \partial w_2 = \delta_o \cdot h^T$$

$$\partial L / \partial b_2 = \delta_o$$

$$\delta_h = (w_2^T \delta_o) \odot \sigma'(h_{in}) \quad \leftarrow \text{el error "viaja" hacia atrás}$$

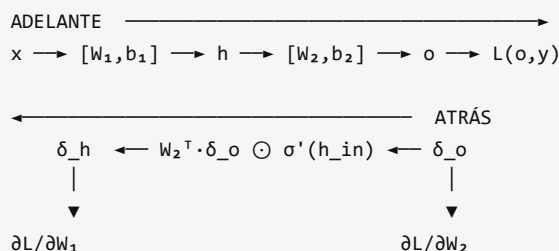
$$\partial L / \partial w_1 = \delta_h \cdot x^T$$

$$\partial L / \partial b_1 = \delta_h$$

$$\text{Actualización: } \theta \leftarrow \theta - \eta \cdot \partial L / \partial \theta$$

El único ingrediente nuevo respecto al cálculo elemental es el orden: computar δ de atrás hacia adelante evita recalcular derivadas repetidas. Eso es lo que lo hace viable.

6. Arquitectura o flujo



7. Qué observar en el paper original

- El artículo de *Nature* es **muy breve** (4 páginas). La formulación extendida está en el capítulo 8 de *Parallel Distributed Processing* (Rumelhart, Hinton y Williams, 1986).
- El experimento de las **representaciones internas**: la red descubre codificaciones en las unidades ocultas que corresponden a regularidades del dominio (por ejemplo, simetría o relaciones familiares) sin que se las especifique.
- Lo que **no** aparece: ReLU, dropout, normalización por lotes, Adam, inicialización cuidadosa, GPU. Todo eso es posterior y es lo que hizo escalar el método décadas después.

8. Evidencia y resultados

El paper demuestra, con problemas pequeños, que una red multicapa entrenada así:

- resuelve tareas no linealmente separables;
- **desarrolla representaciones internas interpretables** en las unidades ocultas.

Los problemas del artículo son de juguete para los estándares actuales; su valor es de **existencia** («esto se puede entrenar»), no de escala. Verificar las figuras concretas en la fuente primaria antes de citar cualquier resultado numérico.

La miniatura de este eje produce evidencia complementaria y verificable: XOR resuelto con 9 parámetros, y comprobación numérica del gradiente con error $\approx 1e-8$.

9. Impacto

- Desbloquea las redes multicapa y cierra el invierno abierto por el análisis de 1969.
- Es el algoritmo que **sigue entrenando todo hoy**: cada modelo del resto de esta ruta — LSTM, AlexNet, Transformer, GPT — se entrena con retropropagación.
- Establece la idea de *representación aprendida*, que es el concepto central del deep learning y da nombre a una conferencia entera (ICLR).

10. Limitaciones

1. **Gradiente desvaneciente o explosivo.** Al encadenar muchas capas, el producto de derivadas colapsa o diverge. El paper no lo aborda; será el tema de PO3.
2. **Óptimos locales y mesetas.** No hay garantía de convergencia al mínimo global.
3. **Coste de memoria.** Hay que guardar las activaciones de la pasada hacia adelante.
4. **Sensibilidad a la inicialización y a la tasa de aprendizaje.** Con sigmoides y pesos mal escalados, el entrenamiento se estanca en la zona de saturación.
5. **Poca plausibilidad biológica.** El cerebro no dispone de un canal simétrico que transporte el error hacia atrás con los mismos pesos (*weight transport problem*).
6. **Requiere activaciones diferenciables**, lo que excluye la función escalón de PO1.

11. Errores comunes

Error	Corrección
«Rumelhart, Hinton y Williams inventaron la retropropagación»	La popularizaron para redes neuronales . La diferenciación en modo reverso es anterior: Linnainmaa (1970), Werbos (1974). El propio artículo reconoce trabajo previo.
«Backpropagation es un algoritmo de optimización»	Es un algoritmo para calcular gradientes . Quien optimiza es SGD, Adam u otro.
«Backprop garantiza encontrar la mejor solución»	Encuentra un punto donde el gradiente se anula, no el óptimo global.
«El paper usa ReLU»	No. Usa sigmoides. ReLU se generaliza a partir de 2010–2012.
«Basta con más capas»	Sin las técnicas posteriores (inicialización, normalización, residuales), más capas empeoran el entrenamiento.

12. Relación con trabajos anteriores

- **P01 Perceptrón (1958)** — el límite lineal que motiva todo.
- **Linnainmaa (1970)** — diferenciación automática en modo reverso. doi.org/10.1007/BF01931367
- **Werbos (1974)** — tesis doctoral que aplica la idea a modelos de ciencias sociales.
- **Minsky y Papert (1969)** — el análisis que definió el problema a superar.

13. Relación con trabajos posteriores

- **P03 LSTM (1997)** — ataca el gradiente desvaneciente en secuencias.
- **P04 AlexNet (2012)** — demuestra que el método escala con GPU, ReLU, dropout y datos masivos.
- **Glorot y Bengio (2010), He et al. (2015)** — inicialización que evita la saturación.
- **Autograd moderno** (Theano, TensorFlow, PyTorch, JAX) — la generalización del método a grafos de cómputo arbitrarios.

14. Notebook asociado

P02_backpropagation.ipynb

Qué implementa: una red 2-2-1 con sigmoides entrenada sobre XOR, con gradientes derivados **a mano** (sin autograd) y verificación numérica del gradiente.

Qué NO implementa: el experimento de representaciones internas del paper, ni ninguna red de tamaño realista.

```
ai-evolution paper-lab P02 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la expresión de δ_h en función de δ_o y explica cada factor.
Explicar	Explica por qué se calcula δ de atrás hacia adelante y no al revés.
Aplicar	Cambia la sigmoide por <code>tanh</code> en el notebook y ajusta la derivada correspondiente.
Analizar	Calcula el producto de derivadas de sigmoide en 10 capas suponiendo $\sigma' \approx 0,25$. ¿Qué queda del gradiente?
Evaluar	Un modelo entrena y la pérdida baja, pero la verificación numérica del gradiente da <code>1e-2</code> de diferencia. ¿Confías en el entrenamiento? Justifica.
Crear	Implementa la comprobación de gradiente como una función reutilizable que reciba cualquier red y devuelva el error relativo máximo.

16. Autoevaluación

1. ¿Por qué la retropropagación es eficiente frente a calcular cada derivada parcial por separado?
2. ¿Qué papel juega $\sigma'(h_{in})$ en la propagación del error hacia atrás?
3. ¿Por qué $\sigma' \leq 0,25$ es una mala noticia en redes profundas?
4. ¿Qué diferencia hay entre backpropagation y descenso de gradiente estocástico?
5. ¿Por qué la verificación numérica del gradiente usa diferencia centrada y no hacia adelante?
6. ¿Qué ocurre si inicializas todos los pesos a cero? ¿Y todos al mismo valor no nulo?
7. ¿Qué idea del deep learning moderno **no** está en este paper y suele atribuírsele?

17. Respuestas esperadas

1. Porque reutiliza los δ ya calculados: el coste es proporcional al de una pasada hacia adelante, en lugar de una evaluación por parámetro.
2. Mide la sensibilidad local de la unidad. Si la neurona está saturada, $\sigma' \approx 0$ y el error deja de propagarse por esa ruta.
3. Porque los factores se multiplican: $0,25^{10} \approx 1e-6$. Las capas iniciales reciben una señal despreciable y dejan de aprender.
4. Backprop **calcula** el gradiente; SGD **usa** ese gradiente para actualizar. Son piezas distintas y se pueden sustituir por separado.
5. Porque su error es $O(\epsilon^2)$ frente a $O(\epsilon)$ de la diferencia hacia adelante: da varios dígitos más de precisión con el mismo número de evaluaciones.
6. Con todos a cero (o todos iguales), todas las unidades ocultas reciben el mismo gradiente y permanecen idénticas para siempre: la simetría no se rompe y la capa oculta es inútil.
7. ReLU, dropout, normalización por lotes, Adam, conexiones residuales, GPU. Todo posterior.

18. Fuentes primarias

- Rumelhart, D. E., Hinton, G. E. y Williams, R. J. (1986). *Learning representations by back-propagating errors*. **Nature**, 323, 533–536. doi.org/10.1038/323533a0 · consultado 2026-08-16.
- Linnainmaa, S. (1976). *Taylor expansion of the accumulated rounding error*. **BIT Numerical Mathematics**, 16, 146–160. doi.org/10.1007/BF01931367 · consultado 2026-08-16.

- Werbos, P. (1974). *Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences*. Tesis doctoral, Universidad de Harvard.

Evaluación — P02

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Learning representations by back-propagating errors* (1986, Nature, 323, 533–536) · **Nivel:** L2

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P02_backpropagation.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P03 — LSTM

La arquitectura que convirtió «recordar durante mucho tiempo» en un problema de ingeniería resoluble, sustituyendo una multiplicación encadenada por una suma con compuertas.

Nivel: L2 · Motor: lstm · Notebook: P03_lstm.ipynb

1. Identificación

Campo	Valor
Título original	Long Short-Term Memory
Autoría	Sepp Hochreiter, Jürgen Schmidhuber
Año	1997
Venue	Neural Computation, 9(8), 1735–1780
Fuente primaria	doi.org/10.1162/neco.1997.9.8.1735
Acceso	Restringido
Fecha de consulta	2026-08-16

2. Problema anterior

Una red recurrente procesa una secuencia reutilizando el mismo estado. Para entrenarla se despliega en el tiempo y se aplica retropropagación: el gradiente atraviesa un paso por cada instante.

Ahí está el problema. En cada paso el gradiente se multiplica por (aproximadamente) $w \cdot \sigma'$. Si ese factor es menor que 1, el producto colapsa exponencialmente; si es mayor, explota. Hochreiter lo había diagnosticado formalmente en su tesis de 1991. La consecuencia práctica: **una RNN clásica no aprende dependencias separadas por más de unas decenas de pasos**, y no por falta de datos ni de capacidad, sino por aritmética.

3. Propuesta

Introducir una **celda de memoria** cuyo estado se actualiza **sumando**, no aplicando una transformación no lineal encadenada. Esa ruta aditiva es el *constant error carousel* (CEC): por ella el gradiente viaja sin multiplicarse por derivadas de activación.

Alrededor del CEC se colocan **compuertas multiplicativas** que aprenden qué información entra (**i**) y qué información se expone al resto de la red (**o**). Las compuertas protegen la celda de escrituras y lecturas irrelevantes.

Precisión histórica obligatoria: la versión de 1997 tiene compuertas de **entrada** y **salida**. La compuerta de **olvido** —la que hoy aparece en todos los diagramas— la añadieron Gers, Schmidhuber y Cummins (1999/2000).

4. Intuición sin fórmulas

Una cinta transportadora que atraviesa toda la fábrica. Los operarios pueden depositar cosas encima (compuerta de entrada) y consultar lo que lleva (compuerta de salida), pero la cinta avanza sin que su contenido se transforme por el camino. Lo que subió al principio puede seguir intacto al final.

Dónde deja de funcionar la analogía: la cinta real no borra nada; la LSTM moderna sí, y lo hace con una compuerta aprendida. Y si esa compuerta aprende a cerrarse, el contenido se desvanece igual que en una RNN.

5. Matemática mínima

Formulación moderna (con compuerta de olvido, posterior al paper):

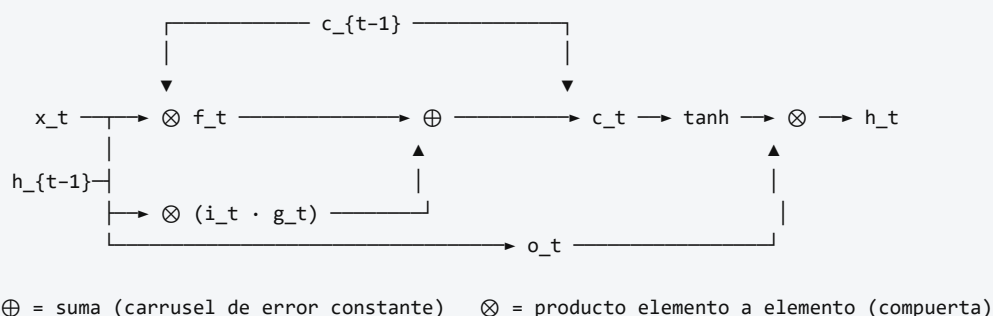
$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$	olvido
$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$	entrada
$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$	salida
$g_t = \tanh(W_g \cdot [h_{t-1}, x_t] + b_g)$	candidato
$c_t = f_t \odot c_{t-1} + i_t \odot g_t$ ← SUMA: aquí está el CEC	
$h_t = o_t \odot \tanh(c_t)$	

La clave está en la penúltima línea:

$$\partial c_t / \partial c_{t-1} = f_t$$

Con $f_t \approx 1$ durante T pasos: $\partial c_T / \partial c_0 \approx 1$
Con una RNN tanh y factor 0,4: $0,4^{40} \approx 1,2 \cdot 10^{-16}$

6. Arquitectura o flujo



7. Qué observar en el paper original

- El artículo es **largo** (46 páginas) y buena parte es el **análisis del flujo de error**: ahí está el argumento, no en la arquitectura.
- La justificación de por qué las compuertas deben ser multiplicativas y el estado aditivo.
- Los experimentos con **problemas sintéticos de dependencia larga** (retardos de cientos de pasos) diseñados para que ninguna RNN previa pudiera resolverlos.
- Comprueba tú mismo: busca la palabra «forget gate» en el artículo de 1997. No está.

8. Evidencia y resultados

El paper compara la LSTM con RNN clásicas, BPTT truncado y otras alternativas de la época sobre tareas construidas para exigir memoria a largo plazo. La LSTM resuelve retardos con los que las alternativas fracasan.

Los detalles de tareas, retardos y tasas de éxito están en la sección de experimentos del artículo. Este eje no reproduce esas cifras: verificarlas en la fuente antes de citarlas.

La miniatura de este eje aporta la evidencia del mecanismo: tras 40 pasos, un factor de decaimiento $0,378$ deja el gradiente en $\approx 10^{-17}$, mientras que una compuerta de olvido en $0,98$ lo deja en $\approx 0,45$.

9. Impacto

- Durante casi dos décadas fue la arquitectura por defecto para secuencias: reconocimiento de voz, escritura manuscrita, traducción, series temporales.
- Hizo posible Seq2Seq y, con él, la traducción automática neuronal — el problema que llevó a la atención y al Transformer.
- Instaló una idea que sobrevive a la arquitectura: **las rutas aditivas preservan el gradiente**. Es exactamente el mismo principio que las conexiones residuales de ResNet y de cada bloque Transformer.

10. Limitaciones

1. **Sigue siendo secuencial.** El paso t necesita el $t-1$: no se puede paralelizar sobre la longitud. Este es el límite que motiva Po8.
2. **No elimina el gradiente desvaneciente; lo pone bajo control de una compuerta aprendida.** Si f aprende valores bajos, el desvanecimiento vuelve.
3. **Cuatro veces más parámetros** que una RNN simple del mismo tamaño de estado.
4. **Sigue comprimiendo** toda la historia en un estado de tamaño fijo.
5. **Difícil de interpretar:** qué guarda cada dimensión de la celda no es legible.
6. **La versión de 1997 carece de compuerta de olvido**, y sin ella el estado de celda puede crecer sin límite en secuencias largas — motivo por el que se añadió después.

11. Errores comunes

Error	Corrección
«La LSTM de 1997 tiene compuerta de olvido»	No. Gers, Schmidhuber y Cummins (1999/2000). Es el anacronismo más repetido del campo.
«La LSTM resuelve el gradiente desvaneciente»	Lo mitiga cuando la compuerta de olvido se mantiene cerca de 1 en el intervalo relevante.
«La GRU es una LSTM simplificada del mismo paper»	La GRU es de Cho et al. (2014), 17 años después.
«Las LSTM quedaron obsoletas»	Siguen siendo competitivas en series temporales, dispositivos con poca memoria y latencia baja. «Obsoleto» es una afirmación sobre un contexto, no sobre una arquitectura.
«El estado de celda y el estado oculto son lo mismo»	<code>c_t</code> es la memoria protegida; <code>h_t</code> es lo que la celda expone , filtrado por <code>o_t</code> .

12. Relación con trabajos anteriores

- **Po2 Backpropagation (1986)** — el método de entrenamiento cuyo problema en el tiempo se ataca aquí.
- **Elman (1990)** — redes recurrentes simples.
- **Hochreiter (1991)** — tesis que diagnostica formalmente el gradiente desvaneciente.
- **Werbos (1990)** — retropropagación en el tiempo (BPTT).

13. Relación con trabajos posteriores

- **Gers, Schmidhuber y Cummins (2000)** — compuerta de olvido.
doi.org/10.1162/089976600300015015
- **Cho et al. (2014)** — GRU, versión con menos compuertas. [arXiv:1406.1078](https://arxiv.org/abs/1406.1078)
- **Po6 Seq2Seq (2014)** — dos LSTM encadenadas para traducir.
- **He et al. (2015), ResNet** — el mismo principio aditivo aplicado a la profundidad.
- **Po8 Transformer (2017)** — elimina la recurrencia entera.

14. Notebook asociado

P03_lstm.ipynb

Qué implementa: una pasada completa de la celda con valores explícitos de las cuatro compuertas, y la comparación cuantitativa del decaimiento del gradiente entre una RNN tanh y la ruta aditiva de la celda.

Qué NO implementa: entrenamiento real de una LSTM, BPTT, ni las tareas sintéticas del paper.

```
ai-evolution paper-lab P03 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Enumera las cuatro compuertas de la LSTM moderna y di cuáles estaban en 1997.
Explicar	Explica por qué una suma preserva el gradiente y un producto encadenado no.
Aplicar	Calcula el estado de celda tras 3 pasos con $f = 0,9$, $i = 0,5$ y $g = 1$ constantes.
Analizar	¿Cuál es el valor mínimo de f que conserva el 1 % del gradiente tras 100 pasos? Resuélvelo analíticamente y compruébalo.
Evaluar	Un artículo divulgativo dice: «la LSTM recuerda porque tiene memoria». Reescríbelo con precisión técnica en dos frases.
Crear	Diseña una tarea sintética mínima donde una RNN simple falle y una LSTM acierte, y justifica por qué tu tarea discrimina entre ambas.

16. Autoevaluación

1. ¿Qué significa exactamente «carrusel de error constante»?
2. ¿Por qué las compuertas usan sigmoide y el candidato usa tanh?
3. ¿Cuál es la diferencia funcional entre c_t y h_t ?
4. Si $f_t = 1$ e $i_t = 0$ para siempre, ¿qué hace la celda?
5. ¿Qué límite de la LSTM **no** resuelve el mecanismo de compuertas?
6. ¿Por qué la LSTM no se puede paralelizar sobre la longitud de la secuencia?
7. ¿Qué componente de la LSTM actual no aparece en el paper de 1997?

17. Respuestas esperadas

1. La ruta por la que el estado de celda se actualiza sumando, de modo que la derivada $\partial c_t / \partial c_{t-1}$ vale f_t en lugar de un producto de derivadas de activación. Con $f \approx 1$, el error se conserva a través de muchos pasos.
2. La sigmoide devuelve $[0, 1]$: es una compuerta, un porcentaje de paso. La tanh devuelve $[-1, 1]$: es contenido con signo, que puede sumar o restar de la memoria.
3. c_t es la memoria interna protegida. $h_t = o_t \odot \tanh(c_t)$ es la parte que la celda expone al resto de la red. Se puede recordar algo sin exponerlo.
4. Conserva su contenido indefinidamente sin admitir nada nuevo: memoria congelada.
5. La secuencialidad. El cómputo sigue siendo $O(n)$ pasos no paralelizables, y el camino entre dos posiciones distantes sigue siendo largo.
6. Porque h_t depende de h_{t-1} : hay una dependencia de datos estricta entre pasos.
7. La compuerta de olvido (y también la conexión *peephole*, de Gers y Schmidhuber, 2000).

18. Fuentes primarias

- Hochreiter, S. y Schmidhuber, J. (1997). *Long Short-Term Memory*. **Neural Computation**, 9(8), 1735–1780. doi.org/10.1162/neco.1997.9.8.1735 · consultado 2026-08-16.
- Gers, F. A., Schmidhuber, J. y Cummins, F. (2000). *Learning to Forget: Continual Prediction with LSTM*. **Neural Computation**, 12(10), 2451–2471. doi.org/10.1162/089976600300015015 · consultado 2026-08-16.

- Cho, K. et al. (2014). *Learning Phrase Representations using RNN Encoder–Decoder*. [arXiv:1406.1078](#)
· consultado 2026-08-16.

Evaluación — P03

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Long Short-Term Memory* (1997, Neural Computation, 9(8), 1735–1780) · **Nivel:** L2

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P03_lstm.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P04 — AlexNet

El resultado que sacó al deep learning del laboratorio: no una idea nueva, sino la combinación que demostró que las ideas viejas escalaban.

Nivel: L3 · Motor: convnet · Notebook: P04_alexnet.ipynb

1. Identificación

Campo	Valor
Título original	ImageNet Classification with Deep Convolutional Neural Networks
Autoría	Alex Krizhevsky, Ilya Sutskever, Geoffrey E. Hinton
Año	2012
Venue	NeurIPS (entonces NIPS) 2012
Fuente primaria	papers.nips.cc — actas de 2012 · versión CACM 2017
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Hasta 2012, la visión por computador se hacía con **descriptores diseñados a mano** (SIFT, HOG) seguidos de un clasificador. Un humano decidía qué características eran relevantes; el aprendizaje solo ocurría en la última etapa.

Las CNN existían desde LeNet (LeCun et al., 1998) y funcionaban en dígitos manuscritos, pero la comunidad dudaba de que escalaran a imágenes naturales de mil categorías. Faltaban tres cosas a la vez: **datos** (ImageNet llegó en 2009), **cómputo** (las GPU programables) y **técnicas de entrenamiento** que evitaran que una red profunda se estancara o memorizara.

3. Propuesta

Una CNN profunda —5 capas convolucionales y 3 completamente conectadas— entrenada directamente sobre ImageNet, combinando:

- **ReLU** en lugar de sigmoide o tanh: no satura para valores positivos y acelera mucho el entrenamiento;
- **entrenamiento en dos GPU** con la red repartida entre ambas, por límite de memoria;
- **dropout** en las capas densas, para reducir el sobreajuste;
- **aumento de datos** (recortes, reflejos, alteración de color);
- **pooling solapado** y normalización de respuesta local.

Ninguna pieza es enteramente nueva. La contribución es la **combinación funcionando a escala** y el resultado que la respaldó.

4. Intuición sin fórmulas

Un detector de bordes no debería reaprenderse en cada esquina de la imagen. La convolución desliza el mismo detector por toda la imagen: muchos menos parámetros y, sobre todo, la posición del objeto deja de importar. Apilando capas, los detectores simples (bordes) se componen en detectores complejos (texturas, partes, objetos).

Dónde deja de funcionar la analogía: un kernel no «reconoce» nada por sí solo. Lo que existe es un patrón de activación distribuido en muchos filtros; la interpretación limpia de «esta neurona detecta caras» es casi siempre una simplificación.

5. Matemática mínima

Convolución (correlación cruzada):

$$(I * K)[r, c] = \sum_i \sum_j I[r+i, c+j] \cdot K[i, j]$$

Activación:

$$\text{ReLU}(z) = \max(0, z) \quad \text{ReLU}'(z) = 1 \text{ si } z > 0, 0 \text{ si } z < 0$$

Max-pooling:

$$P[r, c] = \max \text{ de la ventana correspondiente}$$

Dropout (entrenamiento):

$$\tilde{h} = h \odot m, \quad m \sim \text{Bernoulli}(p)$$

Recuento de parámetros. Una capa densa que conecte una entrada de $H \times W$ con una salida de $H' \times W'$ necesita $H \cdot W \cdot H' \cdot W'$ pesos. Un kernel $k \times k$ necesita k^2 , reutilizados en toda la imagen. Esa reducción es lo que hace viable la profundidad.

6. Arquitectura o flujo

```
imagen 224x224x3
|
├─ conv1 → ReLU → norm → maxpool
├─ conv2 → ReLU → norm → maxpool
├─ conv3 → ReLU
├─ conv4 → ReLU
├─ conv5 → ReLU → maxpool
|
├─ fc6 → ReLU → dropout
├─ fc7 → ReLU → dropout
└─ fc8 → softmax (1000 categorías)
```

Repartida entre 2 GPU, con comunicación solo en capas concretas.

7. Qué observar en el paper original

- La **figura de la arquitectura** con la red partida en dos flujos: es una restricción de hardware convertida en decisión de diseño.
- La sección de **ReLU**: la comparación de velocidad de convergencia frente a tanh es uno de los argumentos más citados del artículo.
- La **visualización de los filtros de la primera capa**: bordes orientados y manchas de color, sin que nadie los programara.
- La **sección de reducción de sobreajuste**: aumento de datos y dropout.
- La tabla de resultados de ILSVRC-2012 con el error top-1 y top-5, y el margen respecto al segundo clasificado.

8. Evidencia y resultados

El sistema ganó el certamen ILSVRC-2012 de clasificación con un **error top-5 en torno al 15 %, frente a alrededor del 26 % del siguiente participante** — un margen inusual en una competición donde las mejoras anuales eran de uno o dos puntos.

Los valores exactos de top-1 y top-5, para el modelo individual y para el conjunto, están en la tabla de resultados del artículo. Verificarlos ahí antes de citarlos: circulan versiones distintas según se refieran al modelo único, al *ensemble* o al conjunto de validación.

El paper incluye **ablaciones** que muestran cuánto pierde el modelo al quitar capas convolucionales, y experimentos de recuperación por vecinos en el espacio de la penúltima capa.

9. Impacto

- Cambió el método por defecto de la visión por computador en menos de dos años.
- Consolidó la **GPU** como plataforma estándar de entrenamiento.
- Popularizó ReLU y dropout, que pasaron a ser configuración por defecto.
- Estableció la práctica de **preentrenar en un dominio grande y transferir** a tareas con pocos datos.
- Fuera del campo, es el resultado que convirtió el deep learning en tema de portada y desencadenó el ciclo de inversión posterior.

10. Limitaciones

1. **Coste de cómputo y datos.** El resultado depende de ImageNet y de GPU: no es reproducible con un dataset pequeño.
2. **Sensible a la traslación, no a la rotación ni a la escala.** La equivarianza es solo traslacional; el resto se aproxima con aumento de datos.
3. **Frágil ante perturbaciones adversarias**, como mostró trabajo posterior (Szegedy et al., 2013; Goodfellow et al., 2014).
4. **Poco interpretable** más allá de la primera capa.
5. **Sesgo del dataset.** Lo que la red «sabe» está determinado por qué contiene ImageNet y cómo se etiquetó; esa discusión llegó años después.

6. **La normalización de respuesta local** que propone acabó abandonada: trabajos posteriores mostraron que aporta poco frente a la normalización por lotes.

11. Errores comunes

Error	Corrección
«AlexNet inventó las CNN»	LeNet es de 1998; la convolución con retropropagación es anterior. AlexNet demostró que escalaban .
«AlexNet inventó ReLU y dropout»	Ambas son anteriores o contemporáneas (Nair y Hinton, 2010; Hinton et al., 2012). AlexNet las combinó y las popularizó.
«Ganó porque era más profunda»	Ganó por la combinación de profundidad, datos, GPU, ReLU, dropout y aumento de datos. La ablación del propio paper lo respalda.
«Las CNN son mejores que los métodos clásicos»	Afirmación vacía sin tarea, dataset, métrica y línea base.
«El 15 % de error significa que entiende las imágenes»	Significa que acierta una métrica en un conjunto concreto. Nada más.

12. Relación con trabajos anteriores

- **Po2 Backpropagation (1986)** — el método de entrenamiento.
- **LeCun et al. (1998)** — LeNet: convolución, pooling y retropropagación.
- **Deng et al. (2009)** — ImageNet, el dataset sin el cual no hay resultado.
- **Nair y Hinton (2010)** — unidades rectificadas.
- **Hinton et al. (2012)** — dropout. [arXiv:1207.0580](#)

13. Relación con trabajos posteriores

- **VGG (2014), GoogLeNet (2014), ResNet (2015)** — la carrera de profundidad que AlexNet abrió. ResNet resuelve el problema de entrenar cientos de capas con conexiones residuales, el mismo principio aditivo de Po3.
- **Szegedy et al. (2013)** — ejemplos adversarios: el reverso del éxito.
- **Po8 Transformer (2017) y ViT (2020)** — el fin del monopolio convolucional en visión.
- **Russakovsky et al. (2015)** — análisis del certamen ILSVRC. [arXiv:1409.0575](#)

14. Notebook asociado

P04_alexnet.ipynb

Qué implementa: convolución 2D con kernels de borde escritos a mano, ReLU, max-pooling, demostración de equivarianza traslacional y comparación de recuento de parámetros frente a una capa densa equivalente.

Qué NO implementa: ningún entrenamiento, ninguna GPU, ningún dato de ImageNet. Los kernels están escritos, no aprendidos — que es justamente lo contrario de lo que hizo AlexNet.

15. Actividades Bloom

Nivel	Actividad
Recordar	Enumera las cinco técnicas que AlexNet combinó y di cuál es anterior al paper.
Explicar	Explica por qué ReLU acelera el entrenamiento frente a tanh, en términos de derivada.
Aplicar	Aplica un kernel horizontal a la imagen del notebook y explica por qué no responde.
Analizar	Calcula el número de parámetros de una convolución $3 \times 3 \times 64 \rightarrow 128$ y compáralo con la densa equivalente sobre un mapa 56×56 .
Evaluar	Un informe afirma «nuestra CNN alcanza 94 % de exactitud». Escribe las cinco preguntas que debes hacer antes de aceptarlo.
Crear	Diseña un experimento que separe la contribución de ReLU de la del aumento de datos, y di qué necesitarías para ejecutarlo.

16. Autoevaluación

1. ¿Qué significa que la convolución sea *equivariante* a la traslación, y en qué se diferencia de ser *invariante*?
2. ¿Por qué compartir pesos reduce el sobreajuste además del cómputo?
3. ¿Qué hace exactamente el dropout durante el entrenamiento, y qué se hace en inferencia?
4. ¿Por qué la red se partió entre dos GPU?
5. ¿Qué componente del paper original acabó siendo abandonado por la comunidad?
6. ¿Por qué un resultado en ILSVRC-2012 no autoriza a afirmar nada sobre «visión» en general?
7. Nombra dos ideas anteriores a 2012 sin las cuales AlexNet no existiría.

17. Respuestas esperadas

1. Equivariante: si la entrada se desplaza, el mapa de activación se desplaza igual. Invariante: la salida no cambia. El pooling añade invarianza **local**; la convolución sola es equivariante.
2. Porque impone una restricción estructural: el mismo detector se aplica en todas las posiciones. Menos grados de libertad con la misma capacidad de detección.
3. Desactiva unidades al azar con probabilidad p , forzando representaciones redundantes. En inferencia se usan todas, con la escala correspondiente para mantener la magnitud esperada.
4. Por límite de memoria de las GPU de la época (3 GB). Es una restricción de hardware, no una decisión conceptual.
5. La normalización de respuesta local (LRN).
6. Porque la métrica está definida sobre un dataset concreto, con sus categorías, su distribución y sus sesgos. Generalizar a «visión» es un salto sin evidencia.
7. Se aceptan: retropropagación (1986), LeNet (1998), ImageNet (2009), ReLU (2010), dropout (2012), CUDA/GPU programables.

18. Fuentes primarias

- Krizhevsky, A., Sutskever, I. y Hinton, G. E. (2012). *ImageNet Classification with Deep Convolutional Neural Networks*. **NeurIPS 2012**. [actas](#) · consultado 2026-08-16.
- Krizhevsky, A., Sutskever, I. y Hinton, G. E. (2017). Versión revisada en **Communications of the ACM**, 60(6), 84–90. doi.org/10.1145/3065386 · consultado 2026-08-16.
- Russakovsky, O. et al. (2015). *ImageNet Large Scale Visual Recognition Challenge*. [arXiv:1409.0575](#) · consultado 2026-08-16.

Evaluación — P04

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *ImageNet Classification with Deep Convolutional Neural Networks* (2012, NeurIPS (NIPS) 2012) ·

Nivel: L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P04_alexnet.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P05 — Word2Vec

El momento en que el significado de una palabra se vuelve una dirección en el espacio, y esa dirección se puede calcular a escala de miles de millones de palabras.

Nivel: L2 · Motor: word2vec · Notebook: P05_word2vec.ipynb

1. Identificación

Campo	Valor
Título original	<i>Efficient Estimation of Word Representations in Vector Space</i>
Autoría	Tomas Mikolov, Kai Chen, Greg Corrado, Jeffrey Dean
Año	2013
Venue	arXiv:1301.3781 · presentado en el taller de ICLR 2013
Fuente primaria	arXiv:1301.3781
Complemento imprescindible	arXiv:1310.4546 (muestreo negativo, NIPS 2013)
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Representar palabras como identificadores dispersos (*one-hot*) tiene una consecuencia devastadora: **todas las palabras están a la misma distancia entre sí**. «Perro» no está más cerca de «gato» que de «hipoteca». Cualquier modelo que parta de ahí tiene que aprender la similitud desde cero para cada tarea.

Existían representaciones distribuidas: el modelo neuronal de lenguaje de Bengio et al. (2003) ya producía vectores de palabras como efecto colateral. Pero incluía una capa oculta no lineal y una softmax sobre todo el vocabulario: entrenarlo sobre corpus grandes era prohibitivo.

3. Propuesta

Dos arquitecturas **log-lineales, sin capa oculta**, diseñadas para ser baratas:

- **CBOW**: predice la palabra central a partir de su contexto.
- **Skip-gram**: predice el contexto a partir de la palabra central.

El artículo complementario (Mikolov et al., 2013b) añade el ingrediente que las hace prácticas: **muestreo negativo**, que sustituye la softmax sobre todo el vocabulario por unas pocas clasificaciones binarias «esta pareja es real / esta pareja es inventada».

El resultado inesperado: el espacio resultante exhibe **estructura lineal**. Ciertas relaciones semánticas y sintácticas aparecen como desplazamientos aproximadamente constantes.

4. Intuición sin fórmulas

«Dime con quién apareces y te diré qué significas.» Si dos palabras aparecen rodeadas de las mismas palabras, acabarán con vectores parecidos — sin que nadie escriba jamás una definición.

Dónde deja de funcionar la analogía: el método captura **distribución**, no significado. Antónimos como «caliente» y «frío» aparecen en contextos casi idénticos, y por tanto quedan cerca. El espacio no distingue «similar» de «intercambiable».

5. Matemática mínima

Skip-gram con muestreo negativo. Para un par observado (centro c , contexto o) y k negativos $n_1 \dots n_k$ muestreados de una distribución de ruido:

```
L = - log σ(v_c · u_o) - Σ_i log σ(-v_c · u_{n_i})  
  
σ(z) = 1/(1+e-z)  
v_c : vector de entrada (el que se usa como embedding final)  
u_o : vector de salida (contexto)
```

Similitud y analogía:

```
similitud(a, b) = cos(v_a, v_b) = (v_a · v_b) / (||v_a|| ||v_b||)  
  
analogía a : b :: c : ? → argmax_d cos(v_b - v_a + v_c, v_d), d ∉ {a, b, c}
```

La exclusión de $\{a, b, c\}$ del ranking **es parte del protocolo**, no un detalle de implementación: sin ella el resultado cambia por completo.

6. Arquitectura o flujo

```
CBOW          SKIP-GRAM  
contexto → suma → proyección   centro → proyección → predice  
      |               |               |               |  
      |               |               |               |  
      └─ predice el centro ─┘       └─ cada palabra del contexto ─┘
```

Entrenamiento (skip-gram + negativos):

```
par real      (rey, reino) → empujar v_rey · u_reino hacia arriba  
par inventado (rey, hornear) → empujar v_rey · u_hornear hacia abajo
```

7. Qué observar en el paper original

- La **tabla de coste computacional**: el argumento central del artículo es de eficiencia, no de calidad. Se trata de poder entrenar sobre miles de millones de palabras.

- El **conjunto de analogías** semánticas y sintácticas que los autores construyen, y la distinción entre ambos tipos.
- La comparación de calidad frente a dimensión del vector y tamaño del corpus.
- En el artículo complementario: **muestreo negativo**, submuestreo de palabras frecuentes y detección de frases.

8. Evidencia y resultados

Los autores evalúan sobre un conjunto de preguntas de analogía del tipo «*Atenas es a Grecia lo que Oslo es a ____*» (semánticas) y «*camina es a caminando lo que nada es a ____*» (sintácticas), midiendo exactitud de la respuesta exacta.

Las cifras de exactitud por tipo de analogía, dimensión y tamaño de corpus están en las tablas del artículo. Este eje no las reproduce: verificarlas allí antes de citarlas.

La miniatura de este eje produce evidencia a escala de juguete y honesta sobre su alcance: con un corpus de 8 frases, **rey - hombre + mujer** devuelve **reina** como vecino más cercano en las tres semillas probadas, con un coseno claramente separado del segundo candidato.

9. Impacto

- Los embeddings preentrenados se convierten en el punto de partida por defecto de cualquier sistema de PLN durante varios años.
- Fundan la **búsqueda vectorial** y, con ella, todo el ecosistema de bases de datos de vectores que sostiene hoy RAG.
- La estructura lineal del espacio abre una línea de investigación sobre **sesgo**: si el espacio codifica «rey - hombre + mujer = reina», también codifica asociaciones sociales problemáticas (Bolukbasi et al., 2016; Caliskan et al., 2017).
- Popularizan la idea de que **el preentrenamiento no supervisado produce representaciones reutilizables** — la premisa completa de BERT y GPT.

10. Limitaciones

1. **Un vector por palabra.** «Banco» tiene una única representación para el asiento, la entidad financiera y la orilla. Lo resolverán ELMo y BERT con embeddings contextuales.
2. **Sin composición.** No hay forma principada de obtener el vector de una frase.
3. **Vocabulario cerrado.** Una palabra no vista no tiene vector (lo abordará FastText con subpalabras).
4. **Captura distribución, no significado.** Antónimos quedan cerca.
5. **Hereda y amplifica sesgos** del corpus.
6. **La aritmética de analogías es más frágil de lo que sugiere su fama:** depende del protocolo de exclusión, de la normalización y del subconjunto evaluado.

11. Errores comunes

Error	Corrección
«Word2Vec inventó los embeddings»	Bengio et al. (2003) ya producía vectores de palabras. Word2Vec los hizo baratos a gran escala.
«rey - hombre + mujer = reina es una igualdad»	Es el vecino más cercano de un vector calculado, tras excluir las tres palabras de la consulta. No es una identidad algebraica.
«El muestreo negativo está en el paper de enero de 2013»	Está en el artículo complementario de octubre de 2013 (arXiv:1310.4546).
«Word2Vec entiende el significado»	Modela co-ocurrencia. Que la geometría resultante sea interpretable no implica comprensión.
«Los embeddings son neutrales porque los aprende una máquina»	Reflejan el corpus. La neutralidad no es un efecto secundario del automatismo.

12. Relación con trabajos anteriores

- **Harris (1954), Firth (1957)** — hipótesis distribucional.
- **Bengio et al. (2003)** — modelo neuronal de lenguaje con representaciones distribuidas.
- **LSA / análisis semántico latente (1990)** — factorización de matrices de co-ocurrencia.

13. Relación con trabajos posteriores

- **GloVe (Pennington et al., 2014)** — enfoque basado en factorizar co-ocurrencias globales. *ACL Anthology*
- **FastText (2016)** — subpalabras; resuelve el vocabulario cerrado.
- **ELMo (2018)** — embeddings **contextuales**: un vector distinto por aparición. *ACL Anthology*
- **P09 BERT (2018)** — representaciones contextuales profundas.
- **P11 RAG (2020)** — recuperación densa apoyada en embeddings.

14. Notebook asociado

P05_word2vec.ipynb

Qué implementa: skip-gram con muestreo negativo entrenado desde cero en Python puro sobre un corpus de 8 frases, con vecinos por coseno y evaluación de analogía con protocolo de exclusión explícito.

Qué NO implementa: CBOW, submuestreo de frecuentes, detección de frases, ni ninguna evaluación a escala. Con 21 palabras de vocabulario, todo resultado es ilustrativo.

```
ai-evolution paper-lab P05 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la pérdida de skip-gram con muestreo negativo y explica cada término.
Explicar	Explica por qué el muestreo negativo es más barato que la softmax completa.
Aplicar	Ejecuta el notebook con tres semillas y anota qué cambia y qué se mantiene.
Analizar	Quita la exclusión de {a, b, c} del ranking de analogías y explica el resultado.
Evaluar	Un artículo afirma que los embeddings «demuestran que el modelo entiende relaciones de género». Evalúa la afirmación.
Crear	Construye 10 analogías propias en español, ejecútalas y clasifica los fallos por tipo (frecuencia, polisemia, corpus).

16. Autoevaluación

1. ¿Qué diferencia hay entre CBOW y skip-gram, y cuándo conviene cada uno?
2. ¿Por qué la softmax completa es costosa y cómo lo evita el muestreo negativo?
3. ¿Por qué «caliente» y «frío» acaban cerca en el espacio?
4. ¿Qué papel juega la ventana de contexto en el tipo de similitud que se aprende?
5. ¿Por qué hay dos matrices de vectores (entrada y salida) y cuál se usa como embedding?
6. ¿Qué problema de la polisemia no puede resolver este método, y qué lo resolvió?
7. ¿Qué parte del método que hoy se llama «Word2Vec» no está en el paper de enero de 2013?

17. Respuestas esperadas

1. CBOW predice el centro desde el contexto (más rápido, mejor con palabras frecuentes); skip-gram predice el contexto desde el centro (mejor con palabras raras y corpus pequeños).
2. Porque normalizar exige sumar sobre todo el vocabulario en cada paso. El muestreo negativo sustituye eso por $k+1$ clasificaciones binarias, con k típicamente entre 5 y 20.
3. Porque aparecen en contextos casi idénticos. El método mide sustituibilidad distribucional, no relación semántica.
4. Ventanas pequeñas favorecen similitud sintáctica y funcional; ventanas grandes favorecen similitud temática.
5. Cada palabra juega dos papeles (centro y contexto) y necesita un vector para cada uno. Habitualmente se usa la matriz de entrada como embedding final.
6. Que una palabra tenga un único vector para todos sus sentidos. Lo resolvieron los embeddings contextuales: ELMo y después BERT.
7. El muestreo negativo, el submuestreo de palabras frecuentes y la detección de frases: todo ello está en arXiv:1310.4546.

18. Fuentes primarias

- Mikolov, T., Chen, K., Corrado, G. y Dean, J. (2013). *Efficient Estimation of Word Representations in Vector Space*. arXiv:1301.3781 · consultado 2026-08-16.
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G. y Dean, J. (2013). *Distributed Representations of Words and Phrases and their Compositionality*. **NIPS 2013**. arXiv:1310.4546 · consultado 2026-08-

16.

- Pennington, J., Socher, R. y Manning, C. (2014). *GloVe: Global Vectors for Word Representation*. **EMNLP 2014**. [ACL Anthology](#) · consultado 2026-08-16.

Evaluación — P05

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Efficient Estimation of Word Representations in Vector Space* (2013, arXiv:1301.3781 · ICLR 2013 (workshop)) · **Nivel:** L2

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P05_word2vec.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P06 — Seq2Seq

Una sola red aprende a convertir una secuencia de longitud variable en otra de longitud distinta, de extremo a extremo. Y, al hacerlo, revela el cuello de botella que definirá los tres papers siguientes.

Nivel: L3 · Motor: seq2seq · Notebook: P06_seq2seq.ipynb

1. Identificación

Campo	Valor
Título original	Sequence to Sequence Learning with Neural Networks
Autoría	Ilya Sutskever, Oriol Vinyals, Quoc V. Le
Año	2014
Venue	arXiv:1409.3215 · NeurIPS (NIPS) 2014
Fuente primaria	arXiv:1409.3215
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Las redes profundas funcionaban con entradas y salidas de **dimensión fija**: una imagen de 224×224, un vector de características. Pero traducir «el gato negro duerme» a inglés produce una secuencia de longitud distinta a la de entrada, y esa longitud no se conoce de antemano.

La traducción automática del momento se hacía con **sistemas estadísticos por frases**: muchos componentes ajustados por separado (modelo de traducción, modelo de lenguaje, reordenamiento, ajuste de pesos). Funcionaban bien, pero cada pieza se optimizaba con un criterio propio y no con el objetivo final.

3. Propuesta

Dos LSTM encadenadas:

- el **codificador** lee la secuencia de entrada token a token y termina con un estado interno c , un vector de tamaño fijo que pretende resumirla entera;
- el **decodificador** parte de c y genera la salida token a token, alimentándose de lo que ya generó.

Todo se entrena con un único objetivo: maximizar la probabilidad de la traducción correcta.

El artículo aporta además un truco empírico decisivo: **invertir el orden de la secuencia fuente**. Con la entrada invertida, las primeras palabras de origen quedan temporalmente cerca de las primeras de destino, lo que acorta las dependencias que la LSTM debe mantener.

4. Intuición sin fórmulas

Leer una frase entera, cerrar los ojos y escribir la traducción solo de memoria. Con frases cortas funciona. Con un párrafo, cuando llegas al final ya no recuerdas con precisión cómo empezaba.

Dónde deja de funcionar la analogía: una persona puede releer. El decodificador de este modelo no puede: solo dispone de c .

5. Matemática mínima

Codificador: $c = h_n$, con $h_t = \text{LSTM}(h_{t-1}, x_t)$

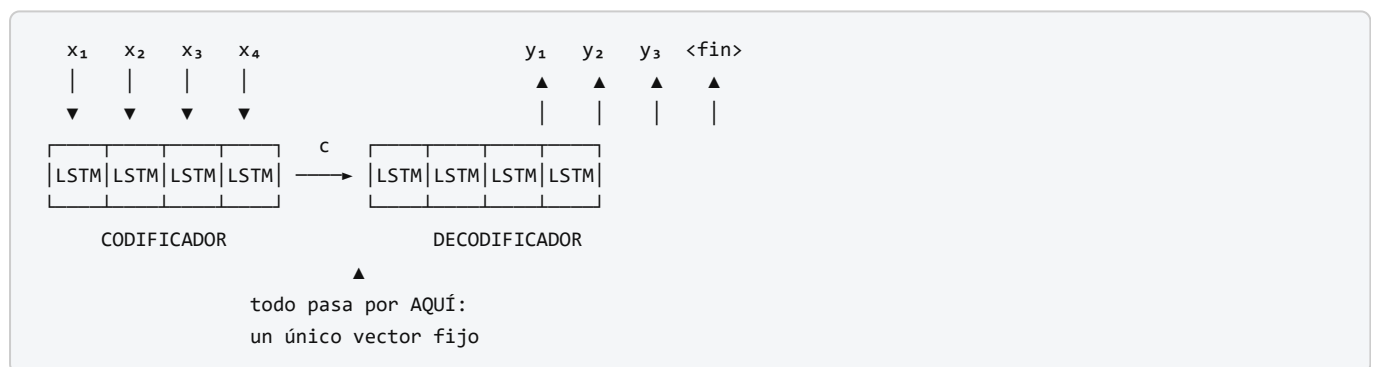
Decodificador: $p(y_1 \dots y_m \mid x_1 \dots x_n) = \prod_i p(y_i \mid y_{<i}, c)$

Entrenamiento: maximizar $\sum \log p(y \mid x)$ sobre el corpus paralelo

Inferencia: búsqueda en haz (beam search) sobre la secuencia de salida

El punto crítico: $c \in \mathbb{R}^d$ tiene dimensión **constante**, mientras que la información de $x_1 \dots x_n$ crece con n . Comprimir algo de tamaño creciente en algo de tamaño fijo tiene un coste, y ese coste crece con la longitud.

6. Arquitectura o flujo



7. Qué observar en el paper original

- La **discusión sobre invertir la secuencia fuente**: es una de las observaciones empíricas más comentadas del artículo, y los autores admiten no tener una explicación completa.
- El uso de **LSTM profundas** (varias capas apiladas) y su efecto en el resultado.
- La **búsqueda en haz** en inferencia y el efecto del tamaño del haz.
- La **visualización PCA** de los estados c : frases con significado parecido quedan cerca, y el espacio muestra sensibilidad al orden de las palabras y a la voz activa/pasiva.
- El experimento de **rescorear** las n -mejores hipótesis del sistema estadístico, en lugar de traducir directamente.

8. Evidencia y resultados

Evaluación sobre traducción **inglés** $\rightarrow$ **francés** de WMT'14, medida en BLEU, comparada contra un sistema estadístico por frases de referencia.

El artículo reporta dos regímenes: traducción directa con un conjunto de LSTM, y rescoring de las hipótesis del sistema estadístico —este último mejor que el primero—. También documenta que la calidad **cae con la longitud de la frase**, y que invertir la fuente mejora el resultado de forma consistente.

Los valores exactos de BLEU para cada configuración están en las tablas del artículo. Verificarlos allí: circulan cifras mezcladas entre el modelo único, el conjunto y el rescoring.

La miniatura de este eje mide el fenómeno estructural, no el BLEU: al pasar de longitud 2 a 32, la similitud del vector de contexto con el **primer** token se desploma mientras la del **último** se mantiene alta.

9. Impacto

- Convierte la traducción automática en un problema de aprendizaje de extremo a extremo, y en pocos años el paradigma estadístico por frases queda desplazado.
- Establece el patrón **codificador–decodificador** que sigue siendo la arquitectura de referencia para transformar una secuencia en otra.
- Populariza la **búsqueda en haz** como procedimiento estándar de decodificación.
- Y, sobre todo, **hace visible el cuello de botella**: al medir la caída de calidad con la longitud, define el problema que Bahdanau resolverá el mismo año.

10. Limitaciones

1. **Cuello de botella del vector fijo**. Es el límite estructural: capacidad constante frente a información creciente.
2. **La calidad cae con la longitud de la frase**. Documentado en el propio artículo.
3. **Sigue siendo secuencial**: no paraleliza sobre la longitud.
4. **Invertir la fuente es un parche**, no una solución. Reordena qué información se pierde.
5. **Coste de entrenamiento alto** para la época, y dependencia de grandes corpus paralelos.
6. **Sin alineación explícita**: no hay forma de saber qué parte de la entrada generó qué parte de la salida.

11. Errores comunes

Error	Corrección
«Seq2Seq usa atención»	No. La atención llega con P07, unas semanas después.
«El problema se arregla agrandando el estado»	Un estado mayor retrasa el problema; no cambia que la capacidad sea constante y la longitud variable.
«Invertir la entrada resuelve el cuello de botella»	Lo mitiga reordenando las dependencias. El cuello sigue ahí.
«Seq2Seq inventó el codificador–decodificador»	Cho et al. (2014) publicó una formulación muy próxima meses antes (arXiv:1406.1078). Ambos trabajos son contemporáneos e independientes.
«BLEU mide calidad de traducción»	Mide solapamiento de n-gramas con referencias. Correlaciona, imperfectamente, y no es comparable entre tokenizaciones distintas.

12. Relación con trabajos anteriores

- **P03 LSTM (1997)** — la celda que hace viable el codificador.
- **Cho et al. (2014)** — RNN encoder–decoder y GRU, contemporáneo. [arXiv:1406.1078](#)
- **Modelos estadísticos de traducción** (Brown et al., 1993; Koehn et al., 2003) — la línea base contra la que se compara.

13. Relación con trabajos posteriores

- **P07 Attention (2014)** — elimina el vector fijo.
- **P08 Transformer (2017)** — conserva el patrón codificador–decodificador y elimina la recurrencia.
- **BART, T5 (2019–2020)** — codificador–decodificador preentrenado.
- **Modelos de imagen a texto y voz a texto** — el mismo patrón aplicado a otras modalidades.

14. Notebook asociado

P06_seq2seq.ipynb

Qué implementa: una medición directa del cuello de botella. Un codificador de juguete comprime secuencias de longitud creciente en un vector fijo y se mide cuánta información del principio sobrevive.

Qué NO implementa: LSTM entrenadas, traducción real, búsqueda en haz ni BLEU. El codificador es una mezcla con decaimiento fijo: aísla el fenómeno, no lo mide en un modelo real.

```
ai-evolution paper-lab P06 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la factorización $p(y \setminus x)$ del decodificador y explica de qué depende cada término.
Explicar	Explica por qué invertir la secuencia fuente ayuda, en términos de distancia entre dependencias.
Aplicar	Ejecuta el notebook y anota a partir de qué longitud el primer token deja de ser recuperable.
Analizar	Argumenta por qué el problema es estructural y no de capacidad, con un contraejemplo numérico.
Evaluar	Alguien propone «duplicar la dimensión del estado» como solución. Evalúa la propuesta con evidencia.
Crear	Diseña una métrica que capture la pérdida de información del principio de la secuencia y justificala.

16. Autoevaluación

1. ¿Por qué las redes profundas previas no servían para traducir?
2. ¿Qué es exactamente el vector c y qué se le pide?
3. ¿Por qué la calidad cae con la longitud de la frase?
4. ¿Qué hace la búsqueda en haz y por qué no se usa simplemente el token más probable en cada paso?
5. ¿Invertir la fuente resuelve o mitiga el problema? Justifica.

6. ¿Qué información se pierde por completo en este modelo respecto a la alineación?
7. ¿Qué idea que hoy asociamos a Seq2Seq **no** está en este paper?

17. Respuestas esperadas

1. Porque exigían entrada y salida de dimensión fija, y la traducción tiene longitudes variables y desconocidas de antemano.
2. El estado final del codificador: un vector de dimensión fija que debe contener toda la información necesaria para reconstruir la salida.
3. Porque la información de entrada crece con n mientras la capacidad de c es constante. El estado acaba dominado por los últimos tokens leídos.
4. Mantiene las k hipótesis parciales más probables. La decodificación voraz puede quedar atrapada tras una primera elección localmente buena y globalmente mala.
5. Mitiga. Acorta la distancia entre las primeras palabras de origen y de destino, pero la compresión a tamaño fijo permanece intacta.
6. Qué parte de la entrada dio lugar a qué parte de la salida: no hay ningún mecanismo que lo exponga ni lo aprenda.
7. La atención. Es de Bahdanau, Cho y Bengio, y la publicación de arXiv es de septiembre de 2014, prácticamente simultánea.

18. Fuentes primarias

- Sutskever, I., Vinyals, O. y Le, Q. V. (2014). *Sequence to Sequence Learning with Neural Networks*. **NIPS 2014**. arXiv:1409.3215 · consultado 2026-08-16.
- Cho, K. et al. (2014). *Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation*. **EMNLP 2014**. arXiv:1406.1078 · consultado 2026-08-16.

Evaluación — P06

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Sequence to Sequence Learning with Neural Networks* (2014, arXiv:1409.3215 · NeurIPS (NIPS) 2014) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P06_seq2seq.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P07 — Atención (Bahdanau)

El decodificador deja de depender de un único vector y aprende a mirar la parte de la entrada que necesita en cada paso. Nace el mecanismo que tres años después dará nombre al paper más influyente del campo.

Nivel: L3 · Motor: bahdanau · Notebook: P07_attention_bahdanau.ipynb

1. Identificación

Campo	Valor
Título original	Neural Machine Translation by Jointly Learning to Align and Translate
Autoría	Dzmitry Bahdanau, Kyunghyun Cho, Yoshua Bengio
Año	2014 (arXiv) · 2015 (ICLR)
Venue	arXiv:1409.0473 · ICLR 2015
Fuente primaria	arXiv:1409.0473
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Seq2Seq comprime la frase de origen en un vector de tamaño fijo. Los propios autores de aquel trabajo documentaron que la calidad **cae con la longitud**: un vector de dimensión constante no puede sostener frases cada vez más largas.

El diagnóstico era claro y el parche disponible (invertir la entrada) no atacaba la causa. La pregunta abierta era: **¿y si el decodificador no tuviera que depender de un único vector?**

3. Propuesta

Que el decodificador construya **un vector de contexto distinto en cada paso de salida**, como suma ponderada de **todos** los estados del codificador. Los pesos los calcula una pequeña red que puntúa la compatibilidad entre el estado actual del decodificador y cada estado de entrada, y se normalizan con softmax.

Dos consecuencias, ambas en el título:

- **align** — los pesos son una alineación suave entre origen y destino, y se pueden inspeccionar;
- **translate** — todo se entrena junto, con la pérdida de traducción. **Nadie etiqueta la alineación**: emerge sola.

El codificador además pasa a ser **bidireccional**, para que cada estado h_j resuma el contexto a ambos lados de la posición j .

4. Intuición sin fórmulas

En lugar de memorizar la frase entera y cerrar los ojos, el traductor deja el texto original sobre la mesa y, para cada palabra que escribe, vuelve a mirar la parte que le hace falta.

Dónde deja de funcionar la analogía: un traductor humano sabe **por qué** mira donde mira. Los pesos α indican dónde se concentró la mezcla, no una razón. Confundir ambas cosas es el error que la sección 11 documenta.

5. Matemática mínima

Puntuación de compatibilidad (aditiva, con una capa oculta):

$$e_{ij} = v^T \cdot \tanh(W \cdot s_{i-1} + U \cdot h_j)$$

Normalización:

$$\alpha_{ij} = \exp(e_{ij}) / \sum_k \exp(e_{ik}) \quad \rightarrow \quad \sum_j \alpha_{ij} = 1$$

Vector de contexto del paso i :

$$c_i = \sum_j \alpha_{ij} \cdot h_j$$

Salida:

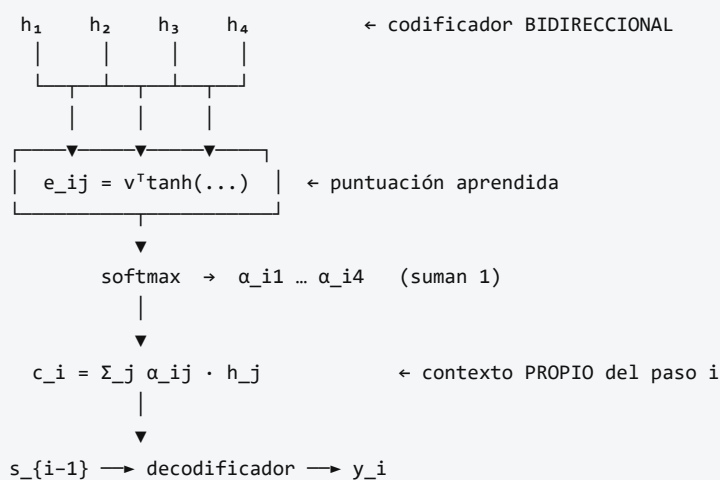
$$s_i = f(s_{i-1}, y_{i-1}, c_i)$$

$$p(y_i | y_{<i}, x) = g(y_{i-1}, s_i, c_i)$$

- h_j : estado del codificador bidireccional en la posición j de origen.
- s_{i-1} : estado del decodificador antes de emitir el token i .
- v, W, U : parámetros **aprendidos** junto con el resto del modelo.

Obsérvese que c_i depende de i : **hay un contexto por paso de salida**, no uno por frase.

6. Arquitectura o flujo



7. Qué observar en el paper original

- Las **matrices de alineación** visualizadas como mapas de calor entre frase de origen y traducción. Es la figura más reproducida del artículo y muestra que la alineación aprendida se corresponde con la intuición lingüística, incluida la reordenación entre idiomas.
- La **curva de BLEU frente a longitud de frase**: el modelo con atención no se degrada como el de vector fijo. Ese gráfico **es** el argumento del paper.
- La justificación del **codificador bidireccional**.
- El detalle de que la red de puntuación es pequeña y se entrena de forma conjunta.

8. Evidencia y resultados

Traducción inglés → francés sobre WMT'14, comparando el modelo con atención (llamado *RNNsearch*) contra el codificador–decodificador de vector fijo (*RNNenc*), con dos límites de longitud de entrenamiento.

El resultado central no es una cifra puntual sino una **forma de curva**: el modelo de vector fijo pierde calidad conforme crece la longitud de la frase, y el modelo con atención mantiene su rendimiento.

Los valores de BLEU por configuración están en las tablas y figuras del artículo. Verificarlos allí antes de citarlos.

La miniatura de este eje aporta evidencia del mecanismo: una atención aditiva con 18 parámetros aprende una alineación correcta 4/4 y produce distribuciones con entropía cercana a cero, con $\sum \alpha = 1$ en cada fila.

9. Impacto

- Fija la atención como componente estándar de la traducción automática neuronal.
- Introduce la idea de **acceso a contenido por compatibilidad aprendida**, que es exactamente lo que generaliza el Transformer.
- Abre la línea de **interpretabilidad por atención**, con su promesa y su posterior crítica.
- Reformula el problema: de «comprimir bien» a «recuperar lo relevante en cada paso» — un cambio de marco que reaparece en RAG.

10. Limitaciones

1. **Coste cuadrático** en el producto (longitud de entrada × longitud de salida): se calcula una puntuación por cada par de posiciones.
2. **Sigue siendo recurrente**. No paraleliza sobre la longitud; ese límite persiste hasta Po8.
3. **La atención no es una explicación**. Trabajo posterior mostró que existen distribuciones de atención muy distintas que producen la misma salida.
4. **Una sola «cabeza»**: un único tipo de relación por paso. El multi-head de Po8 lo generaliza.
5. **La alineación aprendida no siempre coincide** con la alineación lingüística, y cuando coincide no hay garantía de que sea la causa de la traducción.
6. **Depende de corpus paralelos grandes**.

11. Errores comunes

Error	Corrección
«La atención muestra en qué se fijó el modelo, luego explica su decisión»	Jain y Wallace (2019) mostraron que se pueden construir distribuciones de atención muy distintas con la misma salida. Es una pista, no una explicación causal.
«Este paper inventó el Transformer»	Inventó el mecanismo de atención para traducción. El Transformer (2017) lo generaliza y elimina la recurrencia.
«La atención de Bahdanau es producto escalar»	Es aditiva : una capa con tanh. La multiplicativa es de Luong et al. (2015).
«Los pesos α son parámetros del modelo»	Son calculados en cada paso a partir de la entrada. Los parámetros son v , W , U .
«La alineación se supervisa»	No. Emerge de entrenar únicamente la traducción. Esa es la parte notable.

12. Relación con trabajos anteriores

- **P06 Seq2Seq (2014)** — el cuello de botella que motiva el trabajo.
- **Cho et al. (2014)** — codificador–decodificador con GRU, del mismo grupo. [arXiv:1406.1078](#)
- **Graves (2013)** — mecanismos de atención sobre secuencias en generación de escritura.
- **Alineación en traducción estadística** (modelos IBM, 1993) — la noción clásica de alineación que aquí se vuelve suave y diferenciable.

13. Relación con trabajos posteriores

- **Luong et al. (2015)** — atención multiplicativa, global y local. [arXiv:1508.04025](#)
- **P08 Transformer (2017)** — self-attention multi-cabeza sin recurrencia.
- **Jain y Wallace (2019)** — *Attention is not Explanation*. [arXiv:1902.10186](#)
- **Wiegrefe y Pinter (2019)** — *Attention is not not Explanation*: la réplica. Leer ambas es un ejercicio de nivel L3.

14. Notebook asociado

P07_attention_bahdanau.ipynb

Qué implementa: atención aditiva con W y U diagonales, entrenada por descenso de gradiente hasta producir una matriz de alineación correcta; medición de entropía por fila y comprobación de que los pesos suman 1.

Qué NO implementa: traducción real, codificador bidireccional entrenado, ni alineación latente. **Aquí la alineación se supervisa**; en el paper emerge sola. La diferencia es importante y se declara en el propio notebook.

```
ai-evolution paper-lab P07 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe las tres ecuaciones de la atención aditiva y nombra cada símbolo.
Explicar	Explica por qué c_i lleva subíndice i y qué habría cambiado si no lo llevara.
Aplicar	Ejecuta el notebook con tres semillas y compara entropías.
Analizar	Sustituye el softmax por una normalización dividiendo por la suma y explica qué se rompe.
Evaluar	Lee el resumen de Jain y Wallace (2019) y decide si invalida el uso de la atención para depurar modelos. Argumenta.
Crear	Diseña un experimento que distinga «la atención señala lo relevante» de «la atención causa la salida». Di qué necesitarías para ejecutarlo.

16. Autoevaluación

1. ¿Qué problema concreto de Seq2Seq resuelve este mecanismo, y cómo se mide que lo resuelve?
2. ¿Por qué los pesos α deben sumar 1?
3. ¿Qué se aprende exactamente: los pesos α o los parámetros v, W, U ?
4. ¿Por qué el codificador es bidireccional?
5. ¿Qué diferencia hay entre atención aditiva y multiplicativa?
6. ¿Por qué una matriz de atención interpretable no equivale a una explicación?
7. ¿Qué límite de este trabajo persiste y motiva el paper siguiente?

17. Respuestas esperadas

1. El cuello de botella del vector fijo. Se mide con la curva de calidad frente a longitud de frase: con atención deja de degradarse.
2. Porque c_i es una **combinación convexa** de los estados del codificador. Sumar 1 garantiza que el contexto viva en el casco convexo de h_j y que la mezcla sea comparable entre pasos.
3. Se aprenden v, W, U . Los α se **calculan** en cada paso a partir de la entrada concreta.
4. Para que h_j resuma el contexto a ambos lados de la posición j , y no solo lo anterior.
5. La aditiva usa una capa con tanh y un vector v ; la multiplicativa usa directamente un producto escalar (opcionalmente con una matriz). La segunda es más barata y es la que adopta el Transformer con la escala $\sqrt{d_k}$.
6. Porque correlación entre peso y salida no implica causalidad; existen distribuciones alternativas que producen la misma predicción.
7. La recurrencia: el cómputo sigue siendo secuencial en la longitud, y eso limita la escala del entrenamiento.

18. Fuentes primarias

- Bahdanau, D., Cho, K. y Bengio, Y. (2014). *Neural Machine Translation by Jointly Learning to Align and Translate*. **ICLR 2015**. arXiv:1409.0473 · consultado 2026-08-16.

- Luong, M.-T., Pham, H. y Manning, C. D. (2015). *Effective Approaches to Attention-based Neural Machine Translation*. **EMNLP 2015**. arXiv:1508.04025 · consultado 2026-08-16.
- Jain, S. y Wallace, B. C. (2019). *Attention is not Explanation*. **NAACL 2019**. arXiv:1902.10186 · consultado 2026-08-16.

Evaluación — P07

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Neural Machine Translation by Jointly Learning to Align and Translate* (2014, arXiv:1409.0473 · ICLR 2015) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P07_attention_bahdanau.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P08 — Transformer · *Attention Is All You Need*

El paper que quitó la recurrencia del modelado de secuencias y, con ella, el último obstáculo para entrenar a gran escala. Casi todo lo que vino después es una rama de este bloque.

Nivel: L4 · Motor: `transformer` · Notebook: `P08_transformer.ipynb` · Miniaturas: T01–T08

1. Identificación

Campo	Valor
Título original	<i>Attention Is All You Need</i>
Autoría	Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin
Año	2017
Venue	arXiv:1706.03762 (junio) · NeurIPS 2017 (diciembre)
Fuente primaria	arXiv:1706.03762 · actas NeurIPS 2017
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Tras P07, la atención había resuelto el cuello de botella. Pero el modelo seguía siendo **recurrente**, y eso arrastraba dos costes:

- Cómputo secuencial.** El estado `ht` necesita `h{t-1}`. No se puede paralelizar sobre la longitud de la secuencia, por muchas GPU que haya. El entrenamiento sobre corpus grandes se vuelve el cuello de botella real.
- Camino largo entre posiciones.** La señal entre la posición 1 y la 100 atraviesa 99 pasos. Cada paso es una oportunidad para que el gradiente se degrade.

Las alternativas convolucionales (ByteNet, ConvS2S) reducían la secuencialidad pero el camino entre posiciones distantes seguía creciendo con la distancia: logarítmica o linealmente.

La pregunta del paper: **si la atención ya conecta cualquier par de posiciones directamente, ¿para qué sigue haciendo falta la recurrencia?**

3. Propuesta

Una arquitectura codificador–decodificador construida **solo** con:

- **self-attention multi-cabeza** (la única operación que mezcla información entre posiciones);
- **redes feed-forward por posición** (aplicadas de forma independiente a cada token);
- **conexiones residuales** y **layer normalization** alrededor de cada subcapa;
- **codificación posicional** sumada a los embeddings, porque la atención por sí sola es indiferente al orden.

Sin recurrencia y sin convolución. Todas las posiciones de una capa se computan a la vez, y el camino entre dos posiciones cualesquiera es de longitud constante.

4. Intuición sin fórmulas

Cada palabra pregunta al resto de la frase «¿quién de vosotros me importa?», recibe una respuesta ponderada y se actualiza con ella. Y todas las palabras lo hacen **a la vez**, no en fila. Un RNN es una fila de personas pasándose un mensaje al oído; el Transformer es una sala donde todos leen el mismo tablón simultáneamente.

Dónde deja de funcionar la analogía: en la sala, cada persona tiene que leer a todas las demás. Con n personas eso son n^2 lecturas. Por eso el contexto largo es caro, y por eso el Transformer no es «gratis»: cambió coste secuencial por coste cuadrático.

5. Matemática mínima

Atención por producto escalar escalado (ecuación 1)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

- $Q \in \mathbb{R}^{n \times d_k}$ consultas · $K \in \mathbb{R}^{m \times d_k}$ claves · $V \in \mathbb{R}^{m \times d_v}$ valores.
- $Q \cdot K^T$ mide compatibilidad; **softmax** la convierte en pesos que suman 1; el producto por V mezcla la información.

Por qué $\sqrt{d_k}$: si las componentes de q y k son independientes con media 0 y varianza 1, entonces $q \cdot k$ tiene media 0 y **varianza d_k** . Sin normalizar, la magnitud de los scores crece con la dimensión, el softmax se satura y los gradientes se apagan.

Multi-cabeza

$$\begin{aligned} \text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W^O \\ \text{head}_i &= \text{Attention}(Q \cdot W_i^Q, K \cdot W_i^K, V \cdot W_i^V) \\ d_k &= d_v = d_{\text{model}} / h \end{aligned}$$

Partir d_{model} en h subespacios permite atender a varios tipos de relación a la vez **sin aumentar el número de parámetros**, porque cada cabeza trabaja en una dimensión h veces menor.

Máscara causal

$$\text{score}_{ij} \leftarrow -\infty \quad \text{para } j > i \quad (\text{antes del softmax})$$

Se aplica **antes** del softmax, no después: poner a cero los pesos tras normalizar rompería la distribución.

Codificación posicional

```
PE(pos, 2i)   = sin( pos / 10000^{2i/d_model} )
PE(pos, 2i+1) = cos( pos / 10000^{2i/d_model} )
```

Se **suma** al embedding (no se concatena: eso cambiaría `d_model` y rompería la compatibilidad con las residuales).

Subcapa completa

```
salida = LayerNorm( x + Sublayer(x) )

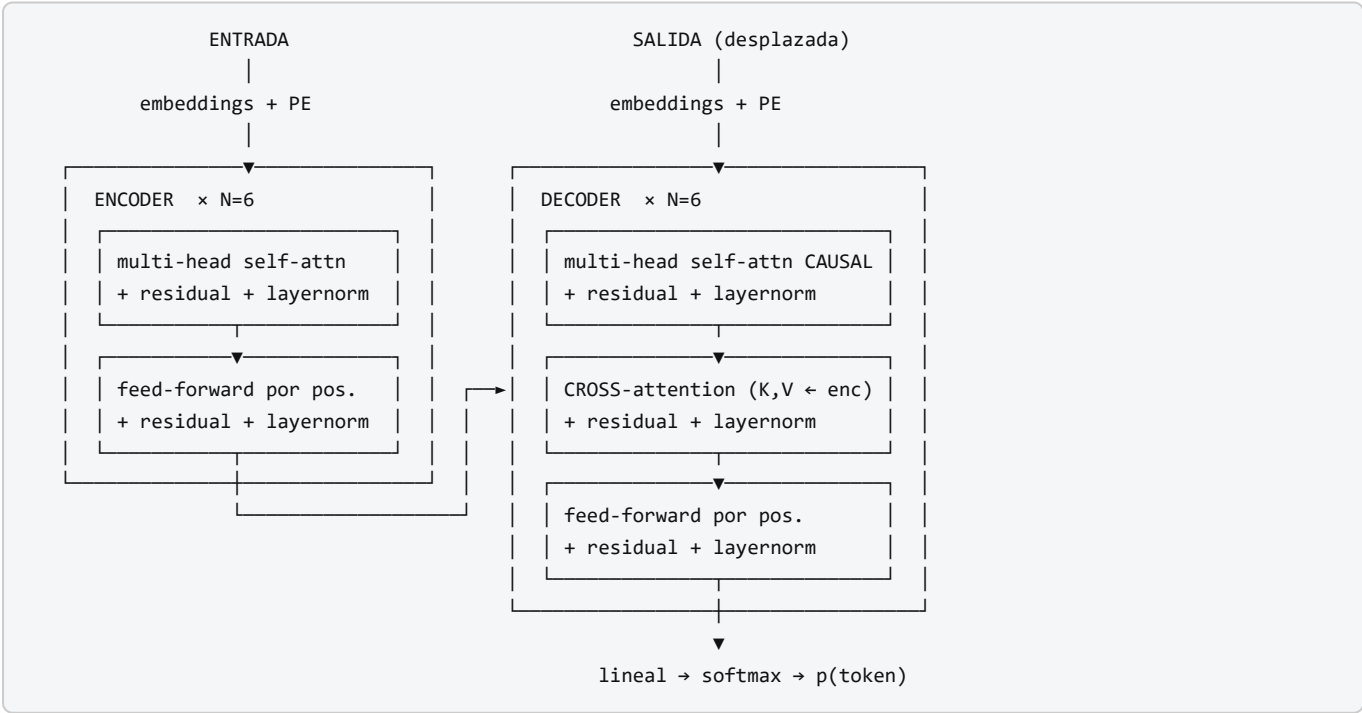
FFN(x) = max(0, x·W1 + b1) · W2 + b2      (misma FFN en todas las posiciones)
```

Complejidad (tabla 1 del paper)

Tipo de capa	Operaciones	Pasos secuenciales	Camino máximo
Self-attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrente	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutacional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k n)$

La atención sale más barata en operaciones cuando `n < d`, y siempre gana en paralelismo y en longitud de camino.

6. Arquitectura o flujo



Configuración base del paper: `N=6`, `d_model=512`, `h=8`, `d_k=d_v=64`, `d_ff=2048`, dropout `0,1`, *label smoothing* `0,1`, optimizador Adam con calentamiento de la tasa de aprendizaje.

7. Qué observar en el paper original

Dónde	Qué buscar
Figura 1	El diagrama de la arquitectura. Cuenta cuántas subcapas hay en el decoder frente al encoder: son tres contra dos.
Sección 3.2.1	La ecuación 1 y la justificación explícita de $\sqrt{d_k}$. Está escrita, no es folclore.
Sección 3.2.2	Por qué multi-cabeza y por qué <code>d_k = d_model/h</code> .
Sección 3.3	La FFN por posición. Es donde vive la mayoría de los parámetros del bloque.
Sección 3.5	Codificación posicional, y la comparación con una versión aprendida (resultados casi idénticos; eligen la sinusoidal por extrapolación a longitudes mayores).
Tabla 1	Complejidad por tipo de capa. Es la tabla que justifica todo el diseño.
Tabla 2	Resultados de traducción y coste de entrenamiento en FLOPs . Compara calidad y cómputo.
Tabla 3	Las ablaciones . La sección más honesta: qué pasa al variar <code>h</code> , <code>d_k</code> , el dropout o la codificación posicional.
Sección 5.4	Regularización: dropout y label smoothing. Detalles sin los cuales no se reproduce.

8. Evidencia y resultados

Traducción sobre **WMT 2014**, en dos pares: inglés→alemán e inglés→francés, medida en BLEU.

- El modelo grande reporta **28,4 BLEU en EN→DE** y **41,8 BLEU en EN→FR**, superando a los mejores modelos y conjuntos publicados hasta entonces.
- El modelo base alcanza resultados competitivos con **un coste de entrenamiento sustancialmente menor** que las alternativas; el modelo grande se entrenó en el orden de días sobre 8 GPU P100.
- Generaliza a otra tarea (análisis sintáctico de constituyentes en inglés) con cambios mínimos.

Los valores exactos por configuración, junto con los FLOPs de entrenamiento, están en la tabla 2 del artículo, y las ablaciones en la tabla 3. **Verificarlos allí antes de citarlos:** es habitual encontrar cifras mezcladas entre modelo base, modelo grande y conjuntos.

La miniatura de este eje aporta evidencia del mecanismo, no del resultado: sin la escala $\sqrt{d_k}$ la entropía media de la atención cae de `1,09` a `0,49` con `d_model=8`, y la matriz causal es exactamente triangular inferior con filas que suman 1.

9. Impacto

- **Toda la familia de modelos de lenguaje actual** descende de este bloque: BERT usa el encoder, GPT usa el decoder, T5 y BART usan ambos.
- La paralelización hizo económicamente viable el entrenamiento a escala, lo que a su vez hizo observables las **leyes de escalado** (Kaplan et al., 2020; Hoffmann et al., 2022).
- Desplazó a las CNN en visión (ViT, 2020) y llegó a audio, código, proteínas y series temporales.
- Convirtió el bloque en una **pieza de infraestructura**: se optimiza a nivel de kernel de GPU, de compilador y de hardware.

🚫 Qué no significa el título

Attention Is All You Need es una consigna, no un teorema. El propio modelo del paper necesita, además de la atención:

Pieza	Sin ella...
Feed-forward por posición	El bloque pierde la mayor parte de su capacidad; la atención sola es una mezcla lineal ponderada
Conexiones residuales	El gradiente no atraviesa 6 bloques apilados
Layer normalization	El entrenamiento se desestabiliza
Codificación posicional	El modelo es ciego al orden : «el perro muerde» y «muerde el perro» son idénticos
Escala $\sqrt{d_k}$	El softmax se satura y los gradientes se apagan
Label smoothing y warmup	No se reproducen los resultados reportados

Lo que el título afirma con precisión es que **no hacen falta recurrencia ni convolución**. Eso es mucho, y es distinto de «basta la atención».

10. Limitaciones

1. **Coste y memoria** $O(n^2)$ en la longitud de secuencia. Con $n = 128\ 000$, la matriz de atención de una sola cabeza y capa ocupa decenas de gigabytes en `float32`.
2. **Extrapolación limitada** a longitudes mucho mayores que las de entrenamiento, pese a la codificación sinusoidal.
3. **Hambriento de datos**. Sin sesgo inductivo de localidad (a diferencia de una CNN), necesita más ejemplos para aprender estructura.
4. **La atención no es explicación**, igual que en P07.
5. **Sin memoria persistente** entre secuencias: todo lo que hay es la ventana de contexto.
6. **Coste ecológico y económico** del entrenamiento, no discutido en el artículo.
7. **El resultado se demuestra en traducción**, no en «lenguaje» en general. La generalización posterior es un hecho histórico, no una afirmación del paper.

11. Errores comunes

Error	Corrección
«El Transformer solo usa atención»	Ver la tabla de la sección 9. Usa FFN, residuales, layer norm y codificación posicional.
«El Transformer inventó la atención»	La inventó Bahdanau et al. (2014). Este paper la generaliza y elimina el resto.
«Es más eficiente que un RNN»	Es más paralelizable siempre; más eficiente en operaciones solo si $n < d$.
«GPT usa el Transformer completo»	GPT usa solo el decoder , sin cross-attention. BERT usa solo el encoder.
«La codificación posicional se concatena»	Se suma . Concatenar cambiaría <code>d_model</code> y rompería las residuales.
«La máscara causal se aplica al softmax»	Se aplica antes , poniendo los scores a $-\infty$.
« $\sqrt{d_k}$ es una constante de ajuste empírica»	El paper la justifica por la varianza del producto escalar (sección 3.2.1).
«El Transformer entiende el lenguaje»	Ganó en BLEU sobre WMT 2014. Todo lo demás es interpretación.

12. Relación con trabajos anteriores

- **P07 Attention (2014)** — el mecanismo que se generaliza.
- **P06 Seq2Seq (2014)** — el patrón codificador–decodificador que se conserva.
- **P04 AlexNet (2012)** y **ResNet (2015)** — profundidad y conexiones residuales.
- **Ba, Kiros y Hinton (2016)** — layer normalization. [arXiv:1607.06450](#)
- **Lin et al. (2017)**, **Cheng et al. (2016)** — self-attention previa en otras formulaciones.
- **ByteNet y ConvS2S (2016–2017)** — alternativas convolucionales a la recurrencia.

13. Relación con trabajos posteriores

- **P09 BERT (2018)** — rama encoder.
- **P10 GPT-3 (2020)** — rama decoder llevada a escala.
- **T5, BART (2019–2020)** — rama codificador–decodificador preentrenada.
- **ViT (2020)** — el mismo bloque aplicado a parches de imagen.
- **Kaplan et al. (2020)** — leyes de escalado. [arXiv:2001.08361](#)
- **Hoffmann et al. (2022)** — Chinchilla: cómputo óptimo entre datos y parámetros. [arXiv:2203.15556](#)
- **Variantes eficientes** (atención dispersa, lineal, con E/S optimizada) — todas atacan el $O(n^2)$ que este paper introdujo.

14. Notebook asociado

Este hito tiene **tratamiento especial**: una miniatura general más ocho que desmontan el bloque pieza por pieza.

Notebook	Qué aísla
P08_transformer.ipynb	Vista integrada: escala, máscara, multi-cabeza, PE, residual, complejidad
T01	Por qué había que quitar la recurrencia
T02	Q, K, V y la ecuación 1
T03	Softmax, escala $\sqrt{d_k}$ y saturación
T04	Self-attention y máscara causal
T05	Multi-head attention
T06	Codificación posicional
T07	Residual, layer norm y feed-forward
T08	Encoder–decoder, complejidad y qué no dice el título

Qué NO implementan: proyecciones W^Q , W^K , W^V aprendidas, entrenamiento, tokenización BPE, traducción ni evaluación BLEU. Son miniaturas del mecanismo, en Python estándar.

```
ai-evolution paper-lab P08 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la ecuación 1 y define Q , K , V , d_k sin mirar.
Explicar	Explica por qué la codificación posicional es imprescindible, usando el experimento de permutación de T06.
Aplicar	Ejecuta T04 y verifica que la masa sobre el futuro es exactamente 0 y que cada fila suma 1.
Analizar	Con $d_{model}=512$ y $h=8$, calcula parámetros de la atención multi-cabeza y de la FFN de un bloque. ¿Cuál domina?
Evaluar	Un artículo divulgativo titula «la atención es todo lo que necesitas: el resto sobra». Escribe una refutación de 5 líneas apoyada en la tabla 3 del paper.
Crear	Diseña una ablación propia: elimina la codificación posicional de un modelo de juguete y define qué tarea revelaría el fallo.

16. Autoevaluación

- ¿Por qué se divide por $\sqrt{d_k}$ y qué ocurre exactamente si no se hace?
- ¿Cuál es la diferencia entre self-attention y cross-attention, y dónde aparece cada una?
- ¿Por qué la máscara causal se aplica antes del softmax?
- ¿Por qué h cabezas de dimensión d/h no cuestan más parámetros que una de dimensión d ?
- ¿Por qué el modelo sería ciego al orden sin codificación posicional? Da un ejemplo concreto.
- ¿En qué régimen de n y d la atención hace más operaciones que la recurrencia?
- ¿Qué gana y qué paga el Transformer respecto a un RNN? Una frase para cada cosa.
- ¿Qué afirma exactamente el título y qué no afirma?

17. Respuestas esperadas

1. Porque $q \cdot k$ tiene varianza d_k cuando las componentes son independientes con varianza 1. Sin escalar, los scores crecen con la dimensión, el softmax se satura hacia un vector casi one-hot y el gradiente de las posiciones no seleccionadas se anula.
2. En self-attention, Q , K y V proceden de la misma secuencia (encoder, y decoder con máscara). En cross-attention, Q viene del decoder y K , V del encoder: es el puente entre ambos.
3. Porque poner pesos a cero después de normalizar deja una distribución que ya no suma 1. Con $-\infty$ antes del softmax, el peso resultante es exactamente 0 y la fila sigue sumando 1.
4. Porque cada cabeza opera en d_{model}/h dimensiones. El total de las proyecciones concatenadas es el mismo que el de una única proyección de dimensión d_{model} .
5. Porque la atención es equivariante a permutaciones: reordenar la entrada solo reordena la salida. «El perro muerde al cartero» y «el cartero muerde al perro» producirían el mismo conjunto de representaciones.
6. Cuando $n > d$: $n^2 \cdot d > n \cdot d^2$ equivale a $n > d$. Con $d = 512$, a partir de secuencias de más de 512 tokens.
7. **Gana:** paralelismo total dentro de la capa y camino de longitud 1 entre cualquier par de posiciones.
Paga: coste y memoria cuadráticos en la longitud.
8. Afirma que **no hacen falta recurrencia ni convolución**. No afirma que baste la atención: el propio modelo lleva FFN, residuales, layer norm y codificación posicional.

18. Fuentes primarias

- Vaswani, A. et al. (2017). *Attention Is All You Need*. **NeurIPS 2017**. arXiv:1706.03762 · actas · consultado 2026-08-16.
- Ba, J. L., Kiros, J. R. y Hinton, G. E. (2016). *Layer Normalization*. arXiv:1607.06450 · consultado 2026-08-16.
- Kaplan, J. et al. (2020). *Scaling Laws for Neural Language Models*. arXiv:2001.08361 · consultado 2026-08-16.
- Hoffmann, J. et al. (2022). *Training Compute-Optimal Large Language Models*. arXiv:2203.15556 · consultado 2026-08-16.

Evaluación — P08

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Attention Is All You Need* (2017, arXiv:1706.03762 · NeurIPS (NIPS) 2017) · **Nivel:** L4

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P08_transformer.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P09 — BERT

El preentrenamiento deja de ser un truco y se convierte en la norma: un modelo base, muchas tareas, un ajuste pequeño para cada una.

Nivel: L3 · Motor: bert_mlm · Notebook: P09_bert.ipynb

1. Identificación

Campo	Valor
Título original	BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding
Autoría	Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova
Año	2018 (arXiv) · 2019 (NAACL)
Venue	arXiv:1810.04805 · NAACL-HLT 2019
Fuente primaria	arXiv:1810.04805 · ACL Anthology
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Con el Transformer disponible, la pregunta pasó a ser **cómo preentrenarlo**. Los modelos de lenguaje predicen el token siguiente y son, por construcción, **unidireccionales**: solo ven el pasado.

Para tareas de comprensión —responder preguntas, clasificar, extraer entidades— eso es un desperdicio: para interpretar una palabra hace falta lo que hay antes **y** después. ELMo (2018) lo había abordado concatenando dos LSTM entrenadas en direcciones opuestas, pero cada una seguía viendo un solo lado; la fusión era superficial.

El obstáculo técnico era real: **no se puede entrenar un modelo profundo y bidireccional con el objetivo de predecir el token siguiente**, porque a través de las capas cada token acabaría viéndose a sí mismo. La predicción sería trivial.

3. Propuesta

Cambiar el objetivo de preentrenamiento:

- 1. Masked Language Modeling (MLM).** Se enmascara un porcentaje de los tokens ($\approx 15\%$) y el modelo predice cada uno usando **todo** el contexto, a ambos lados. Enmascarar es lo que hace legítima la bidireccionalidad.
- 2. Next Sentence Prediction (NSP).** Dado un par de segmentos, predecir si el segundo sigue realmente al primero. Pensado para tareas que relacionan dos frases.

Y un procedimiento uniforme: **preentrenar una vez, ajustar todo el modelo** para cada tarea añadiendo una capa de salida mínima. La misma arquitectura sirve para clasificación, etiquetado y respuesta a preguntas.

Detalle importante del MLM: los tokens seleccionados no siempre se sustituyen por [MASK]. Se reparten entre [MASK], una palabra aleatoria y la palabra original, para que el modelo no aprenda a ignorar las posiciones sin máscara durante el ajuste fino, donde [MASK] no aparece.

4. Intuición sin fórmulas

Un examen de rellenar huecos. Para adivinar la palabra tapada lees toda la frase, no solo la mitad izquierda. Un modelo que renuncia al lado derecho está tirando la mitad de la evidencia.

Dónde deja de funcionar la analogía: rellenar huecos no es lo mismo que escribir un texto. BERT es excelente **comprendiendo** y no es un generador: su objetivo nunca fue producir texto token a token.

5. Matemática mínima

MLM:

$$L_{MLM} = - \sum_{t \in M} \log p(x_t \mid x_{\setminus M})$$

M = conjunto de posiciones enmascaradas

$x_{\setminus M}$ = la secuencia con esas posiciones ocultas (contexto completo a ambos lados)

NSP:

$$L_{NSP} = - \log p(\text{esSiguiente} \mid \text{segmentoA}, \text{segmentoB})$$

Objetivo total:

$$L = L_{MLM} + L_{NSP}$$

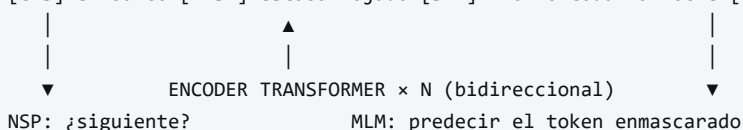
Entrada: [CLS] tokens_A [SEP] tokens_B [SEP], con tres embeddings sumados por posición (token + segmento + posición). El vector de [CLS] se usa como representación agregada de la secuencia para clasificación.

Escala del modelo: BERT-base con 12 capas, $d_{\text{model}}=768$, 12 cabezas, ≈ 110 M de parámetros; BERT-large con 24 capas, $d_{\text{model}}=1024$, 16 cabezas, ≈ 340 M.

6. Arquitectura o flujo

PREENTRENAMIENTO (no supervisado, corpus masivo)

[CLS] el banco [MASK] estaba mojado [SEP] llovió toda la noche [SEP]



AJUSTE FINO (supervisado, pocos datos por tarea)

mismo modelo + una capa de salida → clasificación / QA / etiquetado
se actualizan TODOS los parámetros

7. Qué observar en el paper original

- La **figura comparativa** entre BERT, GPT y ELMo: muestra visualmente qué significa «bidireccional profundo» frente a «unidireccional» y «bidireccional superficial».
- La **justificación de por qué no se puede usar el objetivo autorregresivo** para entrenar bidireccionalmente. Es el argumento central y suele resumirse mal.
- El detalle del **80/10/10** en el enmascarado y su motivación (la discrepancia entre preentrenamiento y ajuste fino).
- Los **estudios de ablación**: qué aporta NSP, qué aporta el número de pasos de preentrenamiento y qué pasa al usar el modelo como extractor de características congelado.
- La tabla de **GLUE** y las de **SQuAD**.

8. Evidencia y resultados

Evaluación sobre **GLUE** (once tareas de comprensión), **SQuAD v1.1 y v2.0** (respuesta a preguntas extractiva) y **SWAG** (inferencia de sentido común).

BERT establece nuevos máximos en las tres familias de tareas con el mismo procedimiento de ajuste, y el artículo reporta una mejora agregada en GLUE que llevó la puntuación por encima del 80 %. Las ablaciones muestran que **la bidireccionalidad es el factor dominante** y que la contribución de NSP es menor.

Las cifras exactas por tarea, para base y large, están en las tablas del artículo. Verificarlas allí antes de citarlas.

La miniatura de este eje aporta el argumento estructural, no las cifras: contando candidatos compatibles en un corpus de juguete, añadir el contexto derecho **nunca amplía** el conjunto de candidatos y en general lo reduce.

9. Impacto

- Convierte **preentrenar y ajustar** en el procedimiento estándar del PLN, desplazando a los modelos entrenados desde cero por tarea.
- Genera una familia enorme de descendientes: RoBERTa, ALBERT, DistilBERT, ELECTRA, DeBERTa y variantes por idioma y dominio.
- Sus embeddings contextuales son la base de los **recuperadores densos** que sostienen RAG.
- Instala la idea de **modelo fundacional**: un artefacto costoso de entrenar y barato de adaptar.

10. Limitaciones

1. **No genera texto.** El objetivo MLM no define una distribución autorregresiva utilizable para generación libre.
2. **Discrepancia preentrenamiento/ajuste.** [MASK] aparece en el preentrenamiento y nunca en el uso real; el reparto 80/10/10 mitiga el problema sin eliminarlo.

3. **Ineficiencia del MLM.** Solo se aprende de $\approx 15\%$ de los tokens por paso (ELECTRA atacó exactamente esto).
4. **NSP resultó de valor dudoso:** RoBERTa (2019) mostró que quitarlo no perjudica.
5. **Longitud de contexto limitada** (512 tokens en las configuraciones del paper).
6. **Coste de ajuste fino por tarea:** hay que guardar una copia completa del modelo por cada tarea, lo que motivó después los métodos de ajuste eficiente en parámetros.
7. **Sesgos del corpus** heredados y amplificados.

11. Errores comunes

Error	Corrección
«BERT es un LLM generativo»	Es un encoder para comprensión. La familia generativa viene de la rama decoder (P10).
«BERT fue el primer modelo bidireccional»	ELMo (2018) ya combinaba dos direcciones. BERT fue el primero profundamente bidireccional en todas las capas.
«MLM es lo mismo que predecir el token siguiente»	Son objetivos distintos y producen modelos con capacidades distintas.
«NSP es esencial en BERT»	Las ablaciones posteriores (RoBERTa) mostraron que se puede eliminar sin pérdida.
«BERT entiende el lenguaje»	Obtiene buenas puntuaciones en GLUE y SQuAD. Trabajos posteriores mostraron que parte de ese rendimiento se apoya en atajos estadísticos del dataset.
«Se enmascara siempre con [MASK]»	80 % [MASK] , 10 % palabra aleatoria, 10 % palabra original.

12. Relación con trabajos anteriores

- **Po8 Transformer (2017)** — la arquitectura (solo el encoder).
- **ELMo (Peters et al., 2018)** — embeddings contextuales con LSTM bidireccionales. [ACL Anthology](#)
- **GPT-1 (Radford et al., 2018)** — preentrenar y ajustar con un decoder unidireccional; el trabajo con el que BERT se compara directamente.
- **ULMFiT (Howard y Ruder, 2018)** — transferencia en PLN con LSTM. [arXiv:1801.06146](#)
- **Po5 Word2Vec (2013)** — el antecesor estático de la idea de representación reutilizable.

13. Relación con trabajos posteriores

- **RoBERTa (2019)** — mismo modelo, mejor entrenado, sin NSP. [arXiv:1907.11692](#)
- **ELECTRA (2020)** — objetivo más eficiente que MLM. [arXiv:2003.10555](#)
- **Sentence-BERT (2019)** y los recuperadores densos → P11 RAG.
- **P10 GPT-3 (2020)** — la rama alternativa, que acabó dominando la conversación pública.

14. Notebook asociado

P09_bert.ipynb

Qué implementa: el argumento de la bidireccionalidad, mediante conteo de candidatos compatibles con contexto izquierdo frente a contexto completo sobre un corpus de juguete con polisemia («banco»).

Qué NO implementa: ningún Transformer, ningún preentrenamiento, ni NSP, ni el reparto 80/10/10, ni evaluación GLUE. Es un modelo de conteo que ilustra **por qué** el contexto derecho aporta información, no **cómo** lo aprovecha BERT.

```
ai-evolution paper-lab P09 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe los dos objetivos de preentrenamiento y qué predice cada uno.
Explicar	Explica por qué predecir el token siguiente impide entrenar bidireccionalmente en profundidad.
Aplicar	Añade frases al corpus del notebook y observa cómo cambia la ambigüedad del hueco.
Analizar	Compara las tres familias (encoder, decoder, encoder-decoder) por objetivo y tareas idóneas.
Evaluar	RoBERTa quitó NSP sin pérdida. ¿Qué dice eso sobre las ablaciones del paper original?
Crear	Diseña un objetivo de preentrenamiento alternativo y argumenta qué capacidad induciría.

16. Autoevaluación

1. ¿Por qué enmascarar hace legítima la bidireccionalidad?
2. ¿Qué problema resuelve el reparto 80/10/10?
3. ¿Por qué el MLM es menos eficiente por token que el objetivo autorregresivo?
4. ¿Para qué sirve el token [CLS] ?
5. ¿Por qué no conviene usar BERT para generar texto libre?
6. ¿Qué mostró RoBERTa sobre NSP, y qué implicación metodológica tiene?
7. ¿Qué significa exactamente una puntuación alta en GLUE, y qué no significa?

17. Respuestas esperadas

1. Porque el token objetivo se oculta de la entrada. Sin ocultarlo, la información fluiría por las capas y el modelo se vería a sí mismo: la predicción sería trivial.
2. La discrepancia entre preentrenamiento y uso: [MASK] nunca aparece en el ajuste fino ni en inferencia. Manteniendo a veces la palabra original o una aleatoria, el modelo debe construir una representación útil de **todas** las posiciones, no solo de las marcadas.
3. Porque solo aporta señal de aprendizaje el $\approx 15\%$ de posiciones enmascaradas, mientras que el objetivo autorregresivo produce una predicción por cada token de la secuencia.
4. Es una posición fija cuya representación final se usa como resumen de toda la secuencia para tareas de clasificación.
5. Porque su objetivo no define $p(x_t \mid x_{<t})$. Puede rellenar huecos, no continuar un texto de forma coherente token a token.

6. Que su aportación era prescindible. Implicación: una ablación del propio paper puede ser insuficiente si el resto de la configuración no está bien ajustada; la replicación independiente importa.
7. Que el modelo acierta un conjunto de tareas de comprensión con sus datasets y métricas. No significa comprensión general: parte del rendimiento puede provenir de correlaciones superficiales de esos datasets.

18. Fuentes primarias

- Devlin, J., Chang, M.-W., Lee, K. y Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. **NAACL-HLT 2019**. arXiv:1810.04805 · ACL Anthology · consultado 2026-08-16.
- Peters, M. et al. (2018). *Deep Contextualized Word Representations (ELMo)*. **NAACL 2018**. ACL Anthology · consultado 2026-08-16.
- Liu, Y. et al. (2019). *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. arXiv:1907.11692 · consultado 2026-08-16.

Evaluación — P09

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding* (2018, arXiv:1810.04805 · NAACL-HLT 2019) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P09_bert.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P10 — GPT-3 y el aprendizaje en contexto

La tarea deja de especificarse con un dataset etiquetado y pasa a especificarse **en el prompt**. Ningún peso cambia: cambia lo que hay escrito antes de la pregunta.

Nivel: L3 · Motor: gpt3_icl · Notebook: P10_gpt3.ipynb

1. Identificación

Campo	Valor
Título original	<i>Language Models are Few-Shot Learners</i>
Autoría	Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah y otros (OpenAI)
Año	2020
Venue	arXiv:2005.14165 · NeurIPS 2020
Fuente primaria	arXiv:2005.14165
Acceso	Abierto (el modelo, no)
Fecha de consulta	2026-08-16

2. Problema anterior

El patrón que BERT consolidó exigía, para cada tarea nueva, un conjunto etiquetado y un ajuste fino que producía una copia entera del modelo. Eso tiene tres costes: **datos** anotados, **cómputo** de ajuste y **almacenamiento** por tarea.

Además, ese patrón no se parece a cómo una persona recibe una tarea: a un humano se le explica con una instrucción y dos ejemplos, no con diez mil casos etiquetados.

GPT-2 (2019) ya había mostrado indicios de resolver tareas sin ajuste fino. La pregunta abierta era si eso era una curiosidad o una **tendencia con el tamaño**.

3. Propuesta

Escalar la rama **decoder** del Transformer hasta 175 000 millones de parámetros, entrenarla con el objetivo autorregresivo de siempre sobre un corpus masivo, y evaluar sin ninguna actualización de gradiente en tres regímenes:

- **zero-shot** — solo la instrucción;
- **one-shot** — la instrucción y un ejemplo;
- **few-shot** — la instrucción y `k` ejemplos, con `k` limitado por la ventana de contexto.

El artículo mide sistemáticamente el rendimiento frente al **tamaño del modelo** en decenas de tareas, y documenta que la ventaja del régimen few-shot sobre el zero-shot **crece con la escala**.

4. Intuición sin fórmulas

En lugar de reentrenar a un empleado para cada tarea nueva, le dejas una nota con dos ejemplos resueltos y la tarea pendiente. Lo que hace es **seguir el patrón que ve en la nota**.

Dónde deja de funcionar la analogía: el empleado recuerda mañana lo que aprendió hoy. El modelo no. Cierras la sesión y no queda rastro: no hubo aprendizaje, hubo condicionamiento.

5. Matemática mínima

El objetivo de entrenamiento es exactamente el de siempre:

$$L = - \sum_t \log p_{\theta}(x_t \mid x_{<t})$$

Lo nuevo es el **protocolo de evaluación**, no la matemática:

```
zero-shot : p(y | instrucción, x)
one-shot  : p(y | instrucción, (x1,y1), x)
few-shot  : p(y | instrucción, (x1,y1)...(xk,yk), x)
```

θ NO cambia en ninguno de los tres casos.

Escala del modelo mayor: 175 000 millones de parámetros, 96 capas, `d_model = 12 288`, 96 cabezas de atención, ventana de contexto de 2 048 tokens.

6. Arquitectura o flujo



Pesos actualizados: 0. Memoria entre llamadas: ninguna.

7. Qué observar en el paper original

- La **figura de las curvas de escala**: rendimiento frente a tamaño del modelo, con una curva por régimen (zero, one, few-shot). La separación entre curvas creciendo con el tamaño es el resultado

principal.

- La **tabla de composición del corpus** de entrenamiento (Common Crawl filtrado, WebText2, libros, Wikipedia) y sus pesos de muestreo.
- La **sección sobre contaminación de datos**: los autores analizan explícitamente el solapamiento entre los benchmarks y el corpus de entrenamiento, y reconocen limitaciones en ese análisis. Leerla es obligatorio antes de citar cualquier cifra.
- La **sección de impactos más amplios**: desinformación, sesgo y consumo energético. Es inusualmente extensa para la época.
- Las tareas donde el modelo **falla**: razonamiento aritmético de varios dígitos, inferencia en algunos conjuntos, comparaciones que requieren varios pasos.
- El artículo tiene decenas de páginas: casi todo es apéndice de evaluación.

8. Evidencia y resultados

Evaluación en más de veinte conjuntos de datos: modelado de lenguaje, respuesta a preguntas de libro cerrado, traducción, razonamiento de sentido común, comprensión lectora, SuperGLUE y tareas sintéticas (aritmética, uso de palabras nuevas, corrección gramatical).

Resultado central: **en varias tareas el régimen few-shot se aproxima a métodos ajustados específicamente**, y la brecha se estrecha conforme crece el modelo. En otras, sigue muy por detrás.

Las cifras por tarea y régimen están en las tablas del artículo. Verificarlas allí, y leer antes la sección de contaminación: parte del rendimiento en algunos conjuntos podría estar afectado por solapamiento con el corpus de entrenamiento.

La miniatura de este eje **no reproduce nada de esto**. Simula el fenómeno con inducción explícita de hipótesis para mostrar cómo cada ejemplo del prompt restringe el espacio de tareas compatibles.

9. Impacto

- Desplaza el centro de gravedad del PLN del **ajuste fino** al **diseño de prompts**, y crea de facto la práctica que hoy se llama ingeniería de prompts y de contexto.
- Hace del **tamaño** una variable de primer orden y motiva el estudio sistemático de las leyes de escalado.
- Populariza el acceso a modelos por **API** en lugar de por pesos, con las consecuencias de reproducibilidad que eso implica.
- Es el punto de partida de InstructGPT: un modelo potente que **no** hace lo que se le pide de forma fiable necesita alineación.

10. Limitaciones

1. **El contexto se paga en cada llamada**. Los ejemplos ocupan tokens, cuestan latencia y dinero, y compiten con el resto del contexto.
2. **Sensibilidad al prompt**. El orden de los ejemplos, el formato y hasta los separadores alteran el resultado. Eso convierte muchas comparaciones en poco fiables.

3. **Sin memoria entre llamadas.** No hay aprendizaje persistente de ningún tipo.
4. **Contaminación de benchmarks,** reconocida por los propios autores.
5. **Aritmética y razonamiento de varios pasos** siguen siendo débiles.
6. **Alucinación:** genera afirmaciones plausibles y falsas, sin forma de citar fuente. Es el problema que ataca RAG.
7. **Modelo cerrado:** sin pesos públicos, la replicación independiente es imposible.
8. **Coste energético y económico** del entrenamiento, no repetible por la mayoría de equipos.
9. **No sigue instrucciones de forma fiable.** Completa texto; que eso coincida con lo que quería el usuario es otra cosa.

11. Errores comunes

Error	Corrección
«Con los ejemplos del prompt, el modelo aprende»	Se condiciona . No hay gradiente, ni actualización, ni persistencia. Llamar «aprendizaje» a esto es la fuente de la mayoría de los malentendidos.
«GPT-3 inventó el prompting»	El artículo lo sistematiza y lo mide a escala. La idea de condicionar con instrucciones es anterior (GPT-2, 2019).
«GPT-3 es ChatGPT»	ChatGPT deriva de modelos alineados con instrucciones (P12). GPT-3 base no está ajustado para dialogar ni para obedecer.
«Few-shot supera al ajuste fino»	En algunas tareas se acerca. En muchas otras el ajuste fino sigue ganando claramente.
«El modelo razona»	Produce continuaciones estadísticamente plausibles. Que a veces coincidan con un razonamiento correcto no autoriza la afirmación.
«Los benchmarks del paper son limpios»	Los propios autores documentan solapamiento con el corpus y las limitaciones de su análisis.

12. Relación con trabajos anteriores

- **P08 Transformer (2017)** — la rama decoder.
- **GPT-1 (Radford et al., 2018)** — preentrenar y ajustar con decoder. Informe técnico de OpenAI.
- **GPT-2 (Radford et al., 2019)** — primeros indicios de tareas resueltas sin ajuste fino.
- **Kaplan et al. (2020)** — leyes de escalado, del mismo equipo y meses antes. [arXiv:2001.08361](#)
- **P09 BERT (2018)** — el paradigma que este trabajo cuestiona.

13. Relación con trabajos posteriores

- **P11 RAG (2020)** — ataca la alucinación y el conocimiento congelado.
- **P12 InstructGPT (2022)** — alineación con instrucciones.
- **Wei et al. (2022), Chain-of-Thought** — razonamiento paso a paso inducido en el prompt. [arXiv:2201.11903](#)
- **Hoffmann et al. (2022), Chinchilla** — corrige el reparto entre parámetros y datos. [arXiv:2203.15556](#)
- **P13 ReAct (2022)** — el modelo pasa a controlar un bucle.

14. Notebook asociado

P10_gpt3.ipynb

Qué implementa: una maqueta del aprendizaje en contexto mediante eliminación explícita de hipótesis: con 0 ejemplos hay cuatro tareas compatibles; con 2, queda una sola. La precisión en casos no vistos sube en consecuencia.

Qué NO implementa —y esto es lo importante—: nada de GPT-3. No hay modelo de lenguaje, ni 175 000 millones de parámetros, ni corpus. GPT-3 **no enumera hipótesis**: condiciona una distribución aprendida. La maqueta sirve para razonar sobre el mecanismo, no para afirmar nada sobre el modelo real.

```
ai-evolution paper-lab P10 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Define zero-shot, one-shot y few-shot con precisión.
Explicar	Explica por qué el aprendizaje en contexto no es aprendizaje, usando tres propiedades ausentes.
Aplicar	Cambia la tarea latente del notebook y mide cuántos ejemplos hacen falta para desambiguar.
Analizar	Diseña dos prompts con el mismo contenido y distinto orden de ejemplos. ¿Qué implicación tiene para comparar modelos?
Evaluar	Lee la sección de contaminación del paper y decide qué cifras aceptarías sin reservas.
Crear	Diseña un benchmark resistente a la contaminación y justifica cada decisión de diseño.

16. Autoevaluación

1. ¿Qué cambia exactamente en el modelo entre zero-shot y few-shot?
2. ¿Por qué el aprendizaje en contexto no es aprendizaje?
3. ¿Qué limita el número de ejemplos que puedes poner en el prompt?
4. ¿Qué es la contaminación de benchmark y por qué es crítica en este paper concreto?
5. ¿Por qué la sensibilidad al orden de los ejemplos es un problema metodológico serio?
6. ¿Qué diferencia hay entre GPT-3 y un asistente conversacional actual?
7. ¿Qué tres problemas de GPT-3 atacan directamente los tres papers siguientes de la ruta?

17. Respuestas esperadas

1. Nada en el modelo. Cambia únicamente el texto de entrada. Los pesos son idénticos.
2. Porque no hay actualización de parámetros, no hay persistencia entre llamadas y no hay generalización acumulativa: cada llamada parte de cero.
3. La ventana de contexto (2 048 tokens en el modelo del paper) y el coste por token.
4. Que ejemplos de test aparezcan en el corpus de preentrenamiento. Es crítico aquí porque el corpus es Common Crawl a gran escala, donde es probable que estén muchos benchmarks públicos; los autores lo

analizan y reconocen que su análisis es imperfecto.

5. Porque si reordenar los mismos ejemplos cambia el resultado, la métrica no mide solo la capacidad del modelo: mide también una elección arbitraria del evaluador. Sin reportar la varianza sobre órdenes, las comparaciones son frágiles.
6. El asistente está **alineado** con instrucciones mediante ajuste supervisado y aprendizaje con preferencias humanas. GPT-3 base solo continúa texto.
7. Conocimiento congelado y no citable → RAG. No seguir instrucciones → InstructGPT. No poder consultar el mundo ni actuar → ReAct.

18. Fuentes primarias

- Brown, T. B. et al. (2020). *Language Models are Few-Shot Learners*. **NeurIPS 2020**. arXiv:2005.14165 · consultado 2026-08-16.
- Kaplan, J. et al. (2020). *Scaling Laws for Neural Language Models*. arXiv:2001.08361 · consultado 2026-08-16.
- Bender, E. M., Gebru, T., McMillan-Major, A. y Shmitchell, S. (2021). *On the Dangers of Stochastic Parrots*. **FAccT 2021**. doi.org/10.1145/3442188.3445922 · consultado 2026-08-16.

Evaluación — P10

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Language Models are Few-Shot Learners* (2020, arXiv:2005.14165 · NeurIPS 2020) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P10_gpt3.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P11 — RAG

Separar lo que el sistema **sabe** de cómo **razona**: el conocimiento pasa a un índice consultable, actualizable y citable, en vez de quedar congelado en los pesos.

Nivel: L3 · Motor: rag · Notebook: P11_rag.ipynb

1. Identificación

Campo	Valor
Título original	Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks
Autoría	Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni y otros
Año	2020
Venue	arXiv:2005.11401 · NeurIPS 2020
Fuente primaria	arXiv:2005.11401
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Todo lo que un modelo preentrenado sabe está en sus pesos. Eso tiene cuatro consecuencias incómodas:

1. **No se puede actualizar** sin reentrenar o ajustar.
2. **No se puede auditar**: no hay forma de saber de dónde salió una afirmación.
3. **No se puede corregir** un dato erróneo de forma quirúrgica.
4. **El modelo alucina**: cuando no sabe, genera algo plausible con la misma confianza.

Trabajos previos habían mostrado que los modelos de lenguaje almacenan cantidades sorprendentes de conocimiento factual (Petroni et al., 2019), y a la vez que ese conocimiento es opaco y difícil de mantener.

3. Propuesta

Combinar **memoria paramétrica** (los pesos del modelo generativo) con **memoria no paramétrica** (un índice denso sobre un corpus de documentos):

1. Un **recuperador denso** (DPR) codifica la consulta y los documentos en el mismo espacio vectorial y devuelve los *k* pasajes más similares.
2. Un **generador seq2seq** (BART) produce la respuesta condicionada a la consulta y a los pasajes recuperados.
3. Ambos se entrenan de forma **conjunta**: la señal de la generación llega al codificador de la consulta (el índice de documentos se mantiene fijo, por coste).

El artículo propone dos variantes: **RAG-Sequence**, que usa el mismo documento para toda la respuesta, y **RAG-Token**, que puede cambiar de documento en cada token generado.

4. Intuición sin fórmulas

Un examen a libro cerrado frente a uno a libro abierto. A libro cerrado, si no lo recuerdas, lo inventas. A libro abierto, buscas la página, la citas y quien corrige puede verificarla.

Dónde deja de funcionar la analogía: en el examen a libro abierto, encontrar la página garantiza casi la respuesta. Aquí no: el generador puede recuperar el documento correcto y aun así contradecirlo. Recuperar y responder son dos fallos independientes.

5. Matemática mínima

Recuperación densa:

$$\text{sim}(x, z) = q(x)^T \cdot d(z) \rightarrow \text{top-}k(x) \text{ por producto escalar}$$

RAG-Sequence (un documento para toda la salida):

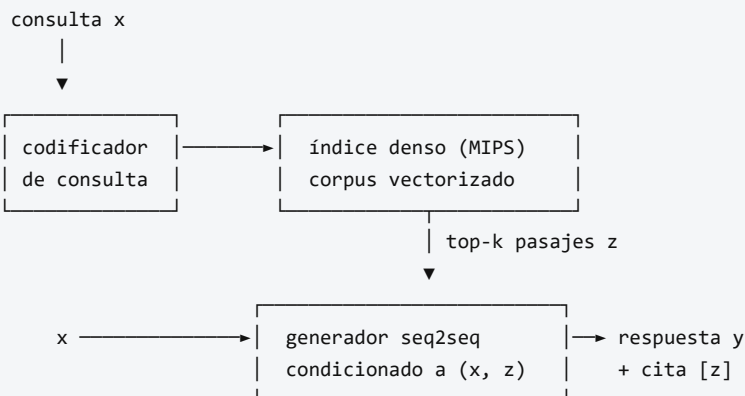
$$p(y \mid x) \approx \sum_{z \in \text{top-}k(x)} p_{\eta}(z \mid x) \cdot \prod_t p_{\theta}(y_t \mid x, z, y_{<t})$$

RAG-Token (documento distinto por token):

$$p(y \mid x) \approx \prod_t \sum_{z \in \text{top-}k(x)} p_{\eta}(z \mid x) \cdot p_{\theta}(y_t \mid x, z, y_{<t})$$

- p_{η} — recuperador: memoria **no paramétrica**, sustituible sin reentrenar.
- p_{θ} — generador: memoria **paramétrica**.
- El documento latente z se **marginaliza**: no se elige uno y se descarta el resto.

6. Arquitectura o flujo



Actualizar conocimiento = reindexar documentos. Sin tocar los pesos.

7. Qué observar en el paper original

- La **distinción entre RAG-Sequence y RAG-Token** y en qué tipo de pregunta gana cada una.

- La **marginalización sobre documentos**: es lo que diferencia RAG de «meter los documentos en el prompt». Los **k** documentos contribuyen a la vez, ponderados.
- El **experimento de actualización de conocimiento**: cambiar el índice cambia lo que el sistema responde, **sin reentrenar**. Es la demostración práctica de la tesis.
- La comparación contra modelos de **libro cerrado** (solo paramétricos) y contra sistemas extractivos.
- La discusión sobre **diversidad y especificidad** de las respuestas generadas frente a las extractivas.

8. Evidencia y resultados

Evaluación en tareas intensivas en conocimiento: **respuesta a preguntas de dominio abierto** (Natural Questions, TriviaQA, WebQuestions, CuratedTrec), **generación de preguntas de Jeopardy** y **verificación de hechos** (FEVER).

RAG establece nuevos máximos en varias de las tareas de respuesta a preguntas de libro abierto del momento, y genera respuestas más específicas y factuales que los modelos puramente paramétricos comparables. En verificación de hechos alcanza resultados cercanos a sistemas con supervisión de evidencia mucho más rica.

Las cifras por tarea (Exact Match, precisión) están en las tablas del artículo. Verificarlas allí antes de citarlas.

La miniatura de este eje aporta el mecanismo y un fallo real: con recuperación léxica, la consulta correcta sitúa el documento pertinente en primer lugar (score 0,78); una consulta sin respuesta en el corpus **también** devuelve un documento, con score bajo. De ahí la necesidad de un umbral de abstención.

9. Impacto

- Da nombre a un patrón que hoy es la arquitectura por defecto de la mayoría de aplicaciones empresariales de IA generativa.
- Convierte la **base de datos vectorial** en pieza de infraestructura estándar.
- Introduce la exigencia de **atribución**: una respuesta sin fuente pasa a considerarse incompleta en dominios regulados.
- Reformula el mantenimiento: actualizar conocimiento se convierte en un problema de **ingeniería de datos**, no de entrenamiento.

10. Limitaciones

1. **Tres fallos independientes.** (a) el documento no está en el índice; (b) está y no se recupera; (c) se recupera y el generador lo contradice. Evaluar RAG exige medir los tres por separado.
2. **Alucinación con cita.** El caso más peligroso: la cita es real y relevante, pero la afirmación no está en ella. Difícil de detectar a simple vista.
3. **Calidad limitada por la del corpus.** Un índice con documentos obsoletos o contradictorios produce respuestas obsoletas o contradictorias, con toda la apariencia de rigor.
4. **Coste de latencia** añadido por la recuperación.

5. **El índice de documentos permanece fijo** durante el entrenamiento conjunto del paper: no se reentrena el codificador de documentos.
6. **Fragmentación (chunking) y granularidad** son decisiones de ingeniería con enorme impacto y ninguna respuesta universal.
7. **No sabe abstenerse.** El paper no incorpora un mecanismo de «no lo sé»; hay que añadirlo.

11. Errores comunes

Error	Corrección
«RAG elimina las alucinaciones»	Las reduce y las hace auditables . Un modelo puede citar bien y afirmar mal.
«RAG es meter documentos en el prompt»	En el paper, el documento es una variable latente marginalizada y el sistema se entrena conjuntamente. Meter texto en el prompt es un caso particular, mucho más simple.
«Si la respuesta lleva cita, es correcta»	Hay que verificar que la afirmación esté en el documento citado.
«Recuperar bien basta»	Recuperación y generación fallan de forma independiente; hay que medir ambas.
«RAG usa búsqueda por palabras clave»	Usa recuperación densa (DPR). La léxica es una alternativa más simple, útil y a menudo complementaria.
«Con RAG el modelo ya no necesita saber nada»	La memoria paramétrica sigue haciendo el trabajo de comprender la consulta y redactar la respuesta.

12. Relación con trabajos anteriores

- **P05 Word2Vec (2013)** y **P09 BERT (2018)** — la representación densa que hace posible la recuperación semántica.
- **Karpukhin et al. (2020)**, **DPR** — el recuperador denso que RAG usa. [arXiv:2004.04906](#)
- **Lewis et al. (2019)**, **BART** — el generador seq2seq. [arXiv:1910.13461](#)
- **Petroni et al. (2019)** — cuánto conocimiento factual almacenan los modelos de lenguaje. [arXiv:1909.01066](#)
- **REALM (Gua et al., 2020)** — trabajo contemporáneo con preentrenamiento aumentado por recuperación. [arXiv:2002.08909](#)

13. Relación con trabajos posteriores

- **FiD (2021)** — fusión en el decodificador de muchos pasajes.
- **Self-RAG, RAG con reordenamiento y RAG con verificación (2023+)** — añaden crítica y abstención.
- **GraphRAG y RAG sobre grafos de conocimiento** — estructura explícita sobre el corpus.
- **P13 ReAct (2022)** — la recuperación deja de ser un paso fijo y se convierte en una **acción** que el modelo decide ejecutar.

14. Notebook asociado

Qué implementa: recuperación por similitud de coseno sobre frecuencias de términos, generación con citas explícitas, el contraste con la respuesta sin recuperación, un caso de **alucinación con cita** y un mecanismo de abstención por umbral.

Qué NO implementa: DPR, entrenamiento conjunto, marginalización sobre documentos ni BART. La recuperación es léxica, no densa: ilustra el patrón, no el método del paper.

```
ai-evolution paper-lab P11 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la fórmula de RAG-Sequence e identifica qué representa z .
Explicar	Explica la diferencia entre memoria paramétrica y no paramétrica con un ejemplo de mantenimiento.
Aplicar	Ejecuta el notebook y ajusta el umbral de abstención hasta que la consulta sin respuesta se rechace.
Analizar	Diseña tres consultas que provoquen cada uno de los tres modos de fallo.
Evaluar	Te presentan un sistema con «95 % de respuestas con cita». ¿Qué métricas pides antes de aceptarlo?
Crear	Diseña un protocolo de evaluación con tres métricas separadas (recuperación, fidelidad, abstención) y define cómo medir cada una.

16. Autoevaluación

1. ¿Qué significa marginalizar sobre el documento latente y por qué no es lo mismo que elegir el mejor?
2. ¿Cuál es la diferencia práctica entre RAG-Sequence y RAG-Token?
3. Nombra los tres modos de fallo de un sistema RAG y una métrica para cada uno.
4. ¿Por qué una alucinación con cita es más peligrosa que una sin cita?
5. ¿Qué se necesita para actualizar el conocimiento de un sistema RAG?
6. ¿Por qué hace falta un umbral de abstención y qué se rompe sin él?
7. ¿Qué parte del sistema sigue dependiendo de la memoria paramétrica del modelo?

17. Respuestas esperadas

1. Sumar sobre los k documentos ponderando por su probabilidad de recuperación, en vez de condicionar solo al mejor. Así la respuesta agrega evidencia de varios pasajes y es más robusta a un error del recuperador en la primera posición.
2. RAG-Sequence usa el mismo documento para toda la respuesta (mejor cuando la respuesta viene de una fuente única); RAG-Token puede cambiar de documento en cada token (mejor cuando la respuesta combina hechos de varias fuentes).
3. (a) cobertura del índice $\rightarrow$ recall del corpus; (b) recuperación $\rightarrow$ recall@k / MRR; (c) fidelidad $\rightarrow$ proporción de afirmaciones verificables en el contexto recuperado.

4. Porque la cita aporta una señal de credibilidad que el lector usa como atajo. Verificarla exige leer la fuente, y la mayoría no lo hace.
5. Reindexar los documentos. No hace falta tocar los pesos del modelo.
6. Porque el recuperador **siempre** devuelve algo: sin umbral, una consulta fuera del dominio recibe el documento «menos malo» y el generador construirá una respuesta sobre él.
7. Comprender la consulta, integrar los pasajes y redactar una respuesta coherente. RAG cambia de dónde vienen los hechos, no quién los redacta.

18. Fuentes primarias

- Lewis, P. et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. **NeurIPS 2020**. arXiv:2005.11401 · consultado 2026-08-16.
- Karpukhin, V. et al. (2020). *Dense Passage Retrieval for Open-Domain Question Answering*. **EMNLP 2020**. arXiv:2004.04906 · consultado 2026-08-16.
- Guu, K. et al. (2020). *REALM: Retrieval-Augmented Language Model Pre-Training*. arXiv:2002.08909 · consultado 2026-08-16.

Evaluación — P11

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks* (2020, arXiv:2005.11401 · NeurIPS 2020) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P11_rag.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P12 — InstructGPT y RLHF

El salto de «modelo que completa texto» a «asistente que hace lo que se le pide». Y la constatación incómoda de que «mejor» es una decisión de quién etiqueta, no una propiedad del modelo.

Nivel: L3 · Motor: rlhf · Notebook: P12_instructgpt_rlhf.ipynb

1. Identificación

Campo	Valor
Título original	Training language models to follow instructions with human feedback
Autoría	Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida y otros (OpenAI)
Año	2022
Venue	arXiv:2203.02155 · NeurIPS 2022
Fuente primaria	arXiv:2203.02155
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

GPT-3 se entrena maximizando la verosimilitud del texto de internet. Ese objetivo **no es** «sé útil, honesto e inocuo». Es «predice qué viene después».

La consecuencia práctica: el modelo continúa el texto de la forma más plausible, que a menudo no es lo que el usuario quería. Ante «explícame X», puede generar más preguntas sobre X en lugar de explicarlo, porque en internet eso también es una continuación plausible.

A esto se le llama **desalineación de objetivo**: el objetivo de entrenamiento y la intención del usuario no coinciden. Y no se arregla con más escala, porque escalar mejora el objetivo equivocado.

3. Propuesta

Tres etapas encadenadas:

- SFT (ajuste supervisado).** Etiquetadores escriben respuestas de demostración a prompts reales; el modelo se ajusta sobre ellas.
- Modelo de recompensa (RM).** Para cada prompt se muestrean varias respuestas y los etiquetadores las **ordenan por preferencia**. Con esas comparaciones se entrena un modelo que asigna un escalar $r(x, y)$.
- Optimización por refuerzo.** La política se optimiza con PPO para maximizar r , con una **penalización KL** que la mantiene cerca del modelo SFT.

La decisión metodológica clave: **pedir comparaciones, no puntuaciones**. Ordenar dos respuestas es más fácil, más rápido y mucho más consistente entre anotadores que puntuar del 1 al 10.

4. Intuición sin fórmulas

Enseñar a alguien a cocinar sin darle recetas: le pones dos platos delante y le dices cuál está mejor. Con suficientes comparaciones, aprende qué buscas — aunque tú nunca hayas sabido formularlo con palabras.

Dónde deja de funcionar la analogía: el aprendiz humano entiende *por qué* un plato es mejor. El modelo de recompensa solo aprende **qué correlaciona** con la preferencia. Si tus platos preferidos resultan ser siempre los más grandes, aprenderá a servir raciones enormes.

5. Matemática mínima

Modelo de recompensa (Bradley-Terry)

$$p(y_w > y_l \mid x) = \sigma(r(x, y_w) - r(x, y_l))$$

$$L_{RM} = - E[\log \sigma(r(x, y_w) - r(x, y_l))]$$

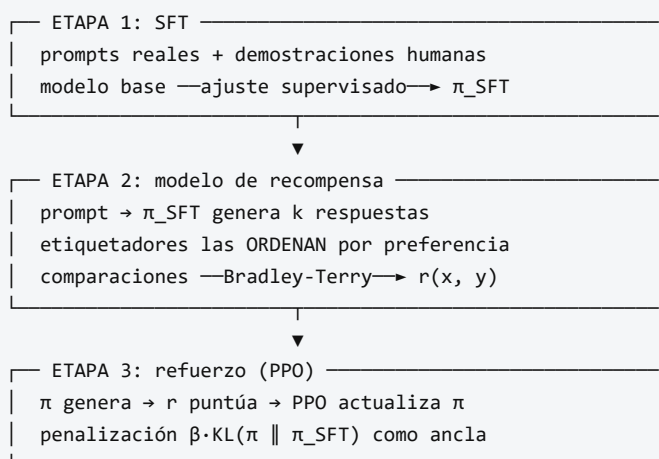
y_w = respuesta preferida, y_l = rechazada. Solo importa la **diferencia**: la escala absoluta de r no está determinada.

Optimización con restricción

$$\max_{\pi} E_{\{y \sim \pi(\cdot|x)\}} [r(x, y)] - \beta \cdot KL(\pi(\cdot|x) \parallel \pi_{SFT}(\cdot|x))$$

El término KL es imprescindible: sin él la política deriva hacia texto degenerado que engaña al modelo de recompensa. β fija el precio de alejarse del modelo base.

6. Arquitectura o flujo



7. Qué observar en el paper original

- El **resultado más citado**: los etiquetadores prefieren las salidas del modelo InstructGPT de **1 300 millones** de parámetros sobre las de GPT-3 de **175 000 millones**. Alinear resultó más rentable que escalar dos órdenes de magnitud.
- La sección sobre **quién etiquetó**: perfil demográfico de los anotadores, criterios de selección y guía de anotación. Es una de las secciones más honestas y menos leídas del artículo, y determina qué significa «mejor» en todo el trabajo.
- El **impuesto de alineación** (*alignment tax*): la caída de rendimiento en algunos benchmarks académicos, y la mezcla de gradientes de preentrenamiento que usan para mitigarla.
- Las secciones sobre **límites**: el modelo sigue alucinando, sigue siendo sensible al prompt y puede obedecer instrucciones dañinas si están bien formuladas.

8. Evidencia y resultados

Evaluación principal por **preferencia humana**: se comparan salidas de distintos modelos sobre la distribución de prompts real de la API y sobre conjuntos públicos.

- InstructGPT es preferido a GPT-3 por un margen amplio, **incluso con modelos mucho más pequeños**.
- Mejora en veracidad (TruthfulQA) y reducción de generación tóxica frente al modelo base.
- Generaliza a instrucciones y a idiomas poco representados en los datos de ajuste.
- Se documenta un retroceso en algunos benchmarks académicos, mitigado parcialmente.

Las proporciones exactas de preferencia y los resultados por benchmark están en las figuras y tablas del artículo. Verificarlos allí antes de citarlos.

La miniatura de este eje muestra el mecanismo del modelo de recompensa: cinco comparaciones bastan para que un `r` lineal ordene cuatro respuestas candidatas, situando arriba la útil y honesta y abajo la peligrosa.

9. Impacto

- Es la técnica que hizo posibles los asistentes conversacionales de uso masivo. **ChatGPT descende directamente de este trabajo**, no de GPT-3 base.
- Convierte la **alineación** en una disciplina de ingeniería con etapas, datos y métricas, no solo en una discusión conceptual.
- Instala la **preferencia humana como métrica**, con todo lo que eso implica: la evaluación deja de ser puramente automática.
- Abre la línea de trabajo sobre reward hacking, RLAIIF y alineación con principios explícitos (Constitutional AI, 2022).

10. Limitaciones

1. **Reward hacking.** Si una característica superficial correlaciona con la preferencia (por ejemplo, la longitud), la política aprende a explotarla.
2. **Los valores son los de los anotadores.** El modelo de recompensa no descubre qué es mejor: reproduce el criterio de un grupo concreto de personas con una guía concreta.
3. **Pipeline complejo y caro:** tres modelos, datos humanos y un bucle de RL delicado de estabilizar. Este es el problema que ataca DPO.
4. **Impuesto de alineación:** caída en algunos benchmarks académicos.
5. **Sigue alucinando.** La alineación cambia el estilo y la obediencia, no añade conocimiento ni verificación.
6. **Sicofancia:** si a los anotadores les gustan las respuestas que les dan la razón, el modelo aprende a darla.
7. **PPO es sensible** a hiperparámetros; reproducirlo requiere mucho oficio no documentado.

11. Errores comunes

Error	Corrección
«RLHF hace que el modelo diga la verdad»	Hace que produzca respuestas preferidas por los anotadores . La correlación con la verdad es parcial y no está garantizada.
«RLHF añade conocimiento al modelo»	No. Reorganiza el comportamiento sobre el conocimiento que ya tenía.
«El modelo de recompensa es objetivo por ser un modelo»	Es un resumen estadístico de juicios humanos concretos, con sus sesgos.
«InstructGPT es GPT-3 con más datos»	Son tres etapas y dos modelos adicionales. La diferencia es de procedimiento, no de escala.
«Este paper inventó RLHF»	Christiano et al. (2017) ya lo aplicaba a control con preferencias humanas. Este trabajo lo lleva a modelos de lenguaje a escala.
«El término KL es un detalle de regularización»	Sin él la política colapsa hacia texto degenerado con recompensa alta. Es estructural.

12. Relación con trabajos anteriores

- **P10 GPT-3 (2020)** — el modelo base y el problema.
- **Christiano et al. (2017)** — RL a partir de preferencias humanas. [arXiv:1706.03741](#)
- **Stiennon et al. (2020)** — RLHF aplicado a resumen; el precedente directo. [arXiv:2009.01325](#)
- **Schulman et al. (2017)**, **PPO** — el algoritmo de optimización. [arXiv:1707.06347](#)
- **Bradley y Terry (1952)** — el modelo de comparaciones por pares.

13. Relación con trabajos posteriores

- **Bai et al. (2022)**, **Constitutional AI** — reemplazar parte del juicio humano por principios explícitos y crítica del propio modelo. [arXiv:2212.08073](#)
- **P15 DPO (2023)** — el mismo objetivo sin modelo de recompensa ni RL.
- **RLAIF (2023)** — retroalimentación generada por IA.
- **Trabajo sobre sicofancia y reward hacking (2023+)** — el estudio sistemático de los fallos que este pipeline induce.

14. Notebook asociado

P12_instructgpt_rlhf.ipynb

Qué implementa: el modelo de recompensa Bradley-Terry entrenado por descenso de gradiente sobre comparaciones por pares, el ranking resultante, una selección best-of-n, y una demostración numérica de cómo el término KL penaliza alejarse del modelo base.

Qué NO implementa: SFT, PPO, generación de texto, ni etiquetadores humanos. Las «respuestas» son cuatro vectores de características escritos a mano.

```
ai-evolution paper-lab P12 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Enumera las tres etapas y qué produce cada una.
Explicar	Explica por qué se piden comparaciones en vez de puntuaciones absolutas.
Aplicar	Cambia una comparación en el notebook y observa cómo se reordena el ranking.
Analizar	Construye un conjunto de preferencias donde el modelo aprenda a premiar la longitud.
Evaluar	¿Un modelo alineado es más seguro o solo lo parece? Argumenta con dos límites del propio paper.
Crear	Diseña una guía de anotación de una página para un dominio que conozcas, y explica qué sesgo introduce cada regla.

16. Autoevaluación

1. ¿Qué significa exactamente «desalineación de objetivo» en este contexto?
2. ¿Por qué las comparaciones son mejores datos que las puntuaciones?
3. ¿Qué ocurre si se elimina el término KL del objetivo?
4. ¿Qué es el reward hacking y cómo lo detectarías en un sistema propio?
5. ¿Por qué un modelo de 1 300 millones alineado puede preferirse a uno de 175 000 millones sin alinear?
6. ¿Qué es el impuesto de alineación?
7. ¿Qué **no** arregla RLHF?

17. Respuestas esperadas

1. Que el objetivo optimizado (verosimilitud del texto) difiere de lo que el usuario quiere (una respuesta útil). Escalar mejora el primero sin acercar el segundo.
2. Porque son más consistentes entre anotadores: una escala numérica se interpreta de forma distinta por cada persona y deriva con el tiempo, mientras que «cuál prefieres» es estable.
3. La política deriva libremente hacia regiones donde el modelo de recompensa da valores altos pero el texto es degenerado. El KL ancla la política al modelo SFT.

4. Explotar una característica que correlaciona con la recompensa sin mejorar la calidad real. Se detecta evaluando con humanos independientes del modelo de recompensa y controlando por variables superficiales como la longitud.
5. Porque el modelo grande optimiza un objetivo distinto del que el usuario quiere. La alineación reorganiza el comportamiento, y eso vale más que capacidad bruta mal dirigida.
6. La pérdida de rendimiento en algunos benchmarks académicos que aparece al alinear, y que el paper mitiga mezclando gradientes de preentrenamiento.
7. La alucinación, la falta de conocimiento actualizado, la sensibilidad al prompt y el sesgo heredado del corpus. Cambia el comportamiento, no lo que el modelo sabe.

18. Fuentes primarias

- Ouyang, L. et al. (2022). *Training language models to follow instructions with human feedback*. **NeurIPS 2022**. [arXiv:2203.02155](https://arxiv.org/abs/2203.02155) · consultado 2026-08-16.
- Christiano, P. et al. (2017). *Deep Reinforcement Learning from Human Preferences*. [arXiv:1706.03741](https://arxiv.org/abs/1706.03741) · consultado 2026-08-16.
- Stiennon, N. et al. (2020). *Learning to summarize with human feedback*. [arXiv:2009.01325](https://arxiv.org/abs/2009.01325) · consultado 2026-08-16.
- Bai, Y. et al. (2022). *Constitutional AI: Harmlessness from AI Feedback*. [arXiv:2212.08073](https://arxiv.org/abs/2212.08073) · consultado 2026-08-16.

Evaluación — P12

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Training language models to follow instructions with human feedback* (2022, arXiv:2203.02155 · NeurIPS 2022) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P12_instructgpt_rlhf.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P13 — ReAct

El modelo deja de ser un generador de texto y pasa a ser el **controlador** de un bucle que observa el mundo y actúa sobre él. Es el paper donde empieza, en sentido técnico, el agente.

Nivel: L2 · Motor: `react` · Notebook: `P13_react.ipynb`

1. Identificación

Campo	Valor
Título original	<i>ReAct: Synergizing Reasoning and Acting in Language Models</i>
Autoría	Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, Yuan Cao
Año	2022 (arXiv) · 2023 (ICLR)
Venue	arXiv:2210.03629 · ICLR 2023
Fuente primaria	arXiv:2210.03629
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Existían dos líneas de trabajo que no se hablaban:

- **Razonar sin actuar.** El razonamiento en cadena (Wei et al., 2022) mejora tareas de varios pasos haciendo que el modelo escriba su razonamiento. Pero ese razonamiento ocurre **dentro** del modelo, sin contacto con el mundo: si un hecho intermedio es falso, nada lo corrige y el error se propaga con toda la apariencia de rigor.
- **Actuar sin razonar.** Los modelos que emiten acciones sobre un entorno (navegar, buscar, manipular) no descomponen la tarea ni replantean el plan cuando una acción no da lo esperado.

Faltaba el acoplamiento: que **lo que se observa condicione lo que se piensa a continuación**.

3. Propuesta

Intercalar dos tipos de salida en la misma secuencia generada:

Thought → Action → Observation → Thought → Action → Observation → ... → Finish

- **Thought** — texto libre: descomponer, planificar, replantear, decidir cuándo parar.
- **Action** — llamada a una herramienta del entorno (buscar, consultar, moverse).
- **Observation** — resultado real devuelto por el entorno; **no lo genera el modelo**.

Lo notable es la sinergia bidireccional: el razonamiento decide qué acción tomar, y la observación real ancla el siguiente razonamiento. El método se aplica mediante prompting con pocos ejemplos, sin entrenamiento adicional.

4. Intuición sin fórmulas

Un investigador que piensa en voz alta mientras consulta archivos. No decide toda la investigación de antemano ni abre carpetas al azar: piensa qué necesita, lo busca, lee lo que encontró y ese hallazgo cambia lo que busca después.

Dónde deja de funcionar la analogía: el investigador sabe cuándo su fuente es poco fiable. El bucle no: si la herramienta devuelve un dato erróneo, lo propaga sin dudar. Lo comprueba el notebook.

5. Matemática mínima

No hay ecuación nueva. Lo que cambia es el **espacio de acciones** del modelo:

Generación estándar:

$a_t \in \text{vocabulario de tokens}$

ReAct:

$a_t \in \text{vocabulario} \cup \text{espacio de acciones del entorno}$

$\text{contexto}_t = (\text{objetivo}, o_1, a_1, \dots, o_{\{t-1\}}, a_{\{t-1\}})$

$a_t \sim p_\theta(\cdot \mid \text{contexto}_t)$

Las observaciones **o** entran en el contexto pero **no** las genera el modelo: vienen del entorno. Ese es el anclaje.

6. Arquitectura o flujo

pregunta: «¿cuántos habitantes tiene la capital de Francia?»

Thought: no sé qué ciudad es; primero busco la capital	
Action: buscar("capital de Francia")	
Observation: "París"	← DEL ENTORNO
Thought: es París; ahora busco su población	
Action: buscar("población de París")	
Observation: "2 100 000"	← DEL ENTORNO
Thought: tengo el dato, puedo responder	
Action: finish("2 100 000")	

Sin el primer paso, la pregunta es irresoluble con una sola búsqueda.

7. Qué observar en el paper original

- La **comparación de cuatro modos**: solo razonar (CoT), solo actuar (Act), ReAct, y las combinaciones ReAct+CoT-SC. La tabla que las compara es el resultado central.
- Las **trazas de ejemplo** en los apéndices: contienen tanto casos de éxito como fallos, y los fallos son más instructivos.
- El análisis de **modos de error**: alucinación de razonamiento, bucles repetitivos, búsquedas poco informativas.
- Los **cuatro entornos**: HotpotQA y FEVER (razonamiento sobre conocimiento), ALFWorld y WebShop (toma de decisiones interactiva). La variedad importa: demuestra que el patrón no es específico de la búsqueda de texto.

8. Evidencia y resultados

- En **HotpotQA** y **FEVER**, ReAct reduce la alucinación frente a CoT puro, porque las observaciones de la Wikipedia anclan los hechos; en exactitud pura, CoT puede ganar en algunos casos, y el artículo lo reconoce en lugar de ocultarlo.
- En **ALFWorld** y **WebShop** —tareas interactivas— ReAct supera de forma clara a los métodos de solo actuar (imitación y refuerzo), con muy pocos ejemplos en el prompt.
- La combinación de ReAct con autoconsistencia obtiene los mejores resultados globales.

Las cifras por entorno y método están en las tablas del artículo. Verificarlas allí: la comparación es matizada y resumirla como «ReAct gana siempre» es incorrecto.

La miniatura de este eje aporta la evidencia estructural: una pregunta de dos saltos es irresoluble con una sola búsqueda, y la traza muestra cómo la primera observación **construye** la segunda consulta.

9. Impacto

- Es el patrón de referencia de casi todos los frameworks de agentes que vinieron después.
- Introduce la **traza auditable** como artefacto de primera clase: se puede revisar qué hizo el sistema y por qué, sin abrir los pesos.
- Convierte «llamar a una herramienta» en parte del bucle de razonamiento, no en un postprocesado.
- Prepara el terreno para Toolformer (aprender **cuándo** llamar) y para los sistemas agentic (memoria, reflexión, presupuesto).

10. Limitaciones

1. **Sin criterio de parada, el bucle no termina.** Es el fallo operativo número uno: la herramienta falla, el agente reintenta, el coste crece sin límite.
2. **La calidad está acotada por la de las herramientas.** Si la fuente miente, el agente miente con seguridad y con traza.
3. **La traza no es fiel.** El texto del «pensamiento» es una generación más y puede no describir el proceso real del modelo. Sirve para depurar, no como prueba.
4. **Coste:** cada paso es una llamada al modelo más una a la herramienta. La latencia se multiplica.

5. **Superficie de ataque:** una observación puede contener texto que intente redirigir al agente (inyección de prompt indirecta).
6. **Depende de un modelo grande:** con modelos pequeños, la calidad de los pensamientos se degrada rápidamente.
7. **Sin memoria entre episodios:** cada tarea empieza de cero.

11. Errores comunes

Error	Corrección
«La traza explica cómo razonó el modelo»	Es texto generado. Útil para auditar decisiones, no una prueba del proceso interno.
«ReAct siempre supera a CoT»	Depende de la tarea. El propio paper reporta casos donde CoT es mejor.
«ReAct es un framework»	Es un patrón de prompting . Los frameworks lo implementan; no son el paper.
«Un agente sin límite de pasos es más capaz»	Es más caro y más frágil. El criterio de parada es una función de seguridad, no una limitación.
«Si el agente cita una observación, el dato es correcto»	El dato es tan fiable como la herramienta que lo produjo.
«ReAct requiere entrenamiento especial»	Se aplica con pocos ejemplos en el prompt, sin entrenar.

12. Relación con trabajos anteriores

- **P10 GPT-3 (2020)** — el aprendizaje en contexto que hace viable aplicar el patrón con pocos ejemplos.
- **Wei et al. (2022), Chain-of-Thought** — razonar explícitamente en el prompt. [arXiv:2201.11903](#)
- **Wang et al. (2022), autoconsistencia** — muestrear varios razonamientos y votar. [arXiv:2203.11171](#)
- **P11 RAG (2020)** — recuperar como paso fijo; aquí pasa a ser una acción decidida por el modelo.

13. Relación con trabajos posteriores

- **P14 Toolformer (2023)** — aprender cuándo llamar a la herramienta.
- **Reflexion (Shinn et al., 2023)** — añadir autocritica y memoria entre intentos. [arXiv:2303.11366](#)
- **Generative Agents (Park et al., 2023)** — memoria episódica y recuperación por relevancia. [arXiv:2304.03442](#)
- **P16 Sistemas agentic** — el bucle convertido en sistema.
- **Model Context Protocol** — estandarización posterior del acceso a herramientas. [modelcontextprotocol.io](#)

14. Notebook asociado

`P13_react.ipynb`

Qué implementa: la comparación entre una estrategia de solo actuar y el bucle completo sobre una pregunta de dos saltos; un bucle sin criterio de parada como anti-patrón; una versión con límite de pasos,

detección de repetición y escalamiento; y un experimento donde la base de conocimiento devuelve un dato corrupto.

Qué NO implementa: ningún modelo de lenguaje. Los «pensamientos» están escritos a mano. En el paper los genera el modelo, y esa es precisamente la parte que puede fallar.

```
ai-evolution paper-lab P13 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe los tres elementos del bucle y di cuál no genera el modelo.
Explicar	Explica por qué una pregunta de dos saltos falla con una sola búsqueda.
Aplicar	Añade una tercera consulta encadenada al notebook y comprueba que la traza sigue siendo legible.
Analizar	Corrompe la base de conocimiento y analiza cómo se propaga el error por la traza.
Evaluar	Un agente resuelve una tarea en 20 pasos y otro en 4. ¿Cuál es mejor? Di qué necesitas saber antes de responder.
Crear	Diseña un verificador que contraste cada observación con una segunda fuente, y estima su coste en llamadas.

16. Autoevaluación

1. ¿Qué aporta la observación que no puede aportar el razonamiento interno?
2. ¿Por qué el razonamiento en cadena puro alucina hechos?
3. ¿Qué tres mecanismos evitan que un bucle se vuelva infinito?
4. ¿Por qué una traza legible no es una explicación del modelo?
5. ¿Qué riesgo de seguridad introduce leer observaciones de fuentes externas?
6. ¿En qué se diferencia ReAct de RAG?
7. ¿Qué límite de ReAct ataca directamente el paper siguiente?

17. Respuestas esperadas

1. Información del mundo que el modelo no tiene o tiene desactualizada, y una señal de corrección: si la acción no da lo esperado, el siguiente pensamiento puede replantear.
2. Porque genera los hechos intermedios desde sus parámetros, sin contrastarlos. Un eslabón falso se integra en la cadena y arrastra al resto con apariencia de solidez.
3. Límite de pasos, detección de estados o consultas repetidas, y escalamiento a un humano ante fallo persistente de la herramienta. Se aceptan también: presupuesto de tokens y de coste.
4. Porque es texto generado por el mismo proceso que produce la respuesta. Puede ser una racionalización posterior y no la causa de la acción.
5. Inyección de prompt indirecta: la observación puede contener instrucciones dirigidas al agente. Todo contenido observado debe tratarse como dato, nunca como instrucción.
6. En RAG la recuperación es un paso fijo del pipeline. En ReAct es una **acción** que el modelo decide ejecutar, cuántas veces y con qué consulta.

7. Que las herramientas y cuándo usarlas se especifican a mano en el prompt. Toolformer aprende ese «cuándo» de forma autosupervisada.

18. Fuentes primarias

- Yao, S. et al. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. **ICLR 2023**. arXiv:2210.03629 · consultado 2026-08-16.
- Wei, J. et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. **NeurIPS 2022**. arXiv:2201.11903 · consultado 2026-08-16.
- Shinn, N. et al. (2023). *Reflexion: Language Agents with Verbal Reinforcement Learning*. arXiv:2303.11366 · consultado 2026-08-16.

Evaluación — P13

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *ReAct: Synergizing Reasoning and Acting in Language Models* (2022, arXiv:2210.03629 · ICLR 2023) · **Nivel:** L2

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P13_react.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P14 — Toolformer

El modelo decide por sí solo cuándo le conviene llamar a una herramienta, usando como criterio su propia pérdida. Nadie etiqueta nada.

Nivel: L3 · Motor: `toolformer` · Notebook: `P14_toolformer.ipynb`

1. Identificación

Campo	Valor
Título original	<i>Toolformer: Language Models Can Teach Themselves to Use Tools</i>
Autoría	Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu y otros
Año	2023
Venue	arXiv:2302.04761 · NeurIPS 2023
Fuente primaria	arXiv:2302.04761
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

ReAct demostró que un modelo puede usar herramientas dentro de un bucle. Pero **qué herramientas existen y cuándo conviene llamarlas** venía escrito a mano en el prompt, o requería datos anotados por humanos.

Eso tiene dos problemas: anotar es caro, y lo anotado limita el modelo a las herramientas y situaciones que alguien previó. Además, los modelos de lenguaje fallan de forma sistemática y predecible en cosas que un programa de tres líneas resuelve: aritmética exacta, fechas, datos actualizados, traducción de términos raros.

La pregunta: **¿puede el modelo descubrir solo dónde le conviene pedir ayuda?**

3. Propuesta

Un procedimiento autosupervisado en cuatro pasos:

1. **Muestrear** posiciones candidatas del texto donde podría insertarse una llamada, y generar llamadas plausibles usando unos pocos ejemplos en el prompt.
2. **Ejecutar** cada llamada y obtener su resultado real.
3. **Filtrar**: conservar la llamada solo si el resultado **reduce la pérdida** de predecir el texto que viene después, por encima de un umbral τ .
4. **Reentrenar** el modelo sobre el corpus enriquecido con las llamadas que sobrevivieron.

La idea central es el criterio del paso 3: **la utilidad de una herramienta se mide por cuánto ayuda a predecir el texto siguiente**. Es una señal automática, disponible en cualquier corpus, sin ningún anotador.

Herramientas del artículo: calculadora, sistema de preguntas y respuestas, buscador, traductor y calendario.

4. Intuición sin fórmulas

Un estudiante con una calculadora sobre la mesa. Prueba a usarla en muchos sitios del examen y se queda con la costumbre solo donde le ayudó de verdad. Donde no, deja de usarla. Nadie le dice cuándo: lo deduce del resultado.

Dónde deja de funcionar la analogía: el estudiante sabe si la calculadora dio un número correcto. El modelo solo sabe si el resultado le **ayudó a predecir el texto siguiente**. Son cosas distintas: una herramienta que devuelve siempre lo mismo puede resultar «útil» por razones espurias.

5. Matemática mínima

Para una posición i , con llamada c y respuesta r :

$$\begin{aligned} L^+ &= L(x_{\{i:\}} \mid \text{prefijo} + [c \rightarrow r]) && \text{con el resultado de la herramienta} \\ L^- &= \min(L(x_{\{i:\}} \mid \text{prefijo}), && \text{sin llamada} \\ &\quad L(x_{\{i:\}} \mid \text{prefijo} + [c \rightarrow \emptyset])) && \text{con la llamada pero sin resultado} \end{aligned}$$

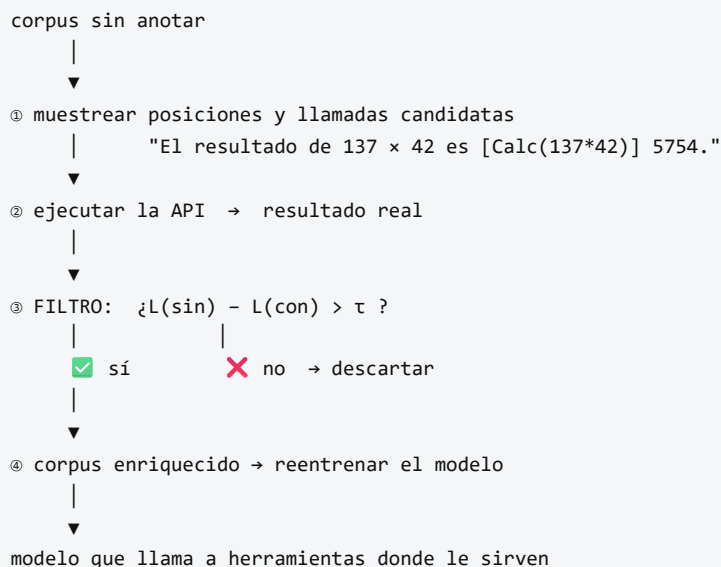
Se conserva la llamada si:

$$L^- - L^+ > \tau$$

L es la pérdida ponderada de predecir los tokens siguientes. τ es el umbral de filtrado.

El segundo término de L^- importa: compara contra **hacer la llamada sin recibir respuesta**, lo que aísla el valor del **resultado** frente al valor de la mera presencia del texto de la llamada.

6. Arquitectura o flujo



7. Qué observar en el paper original

- El **criterio de filtrado** y su justificación: es el corazón del método y ocupa poco espacio en el artículo.
- La **comparación de L^-** contra dos alternativas (sin llamada y con llamada sin resultado). Ese detalle evita conservar llamadas que solo ayudan por el texto que introducen.
- La **tabla de rendimiento por herramienta**: cuánto aporta cada una en qué benchmark. No todas aportan igual.
- El **análisis de escala**: el método necesita un modelo suficientemente capaz para generar llamadas plausibles; por debajo de cierto tamaño no funciona.
- La comprobación de que **no se degrada** la capacidad de modelado de lenguaje del modelo base.

8. Evidencia y resultados

El modelo base es un GPT-J de 6 700 millones de parámetros, enriquecido con el procedimiento y reentrenado.

Resultados evaluados en modo zero-shot sobre tareas donde cada herramienta debería ayudar: aritmética (calculadora), preguntas de conocimiento (buscador y sistema de QA), tareas multilingües (traductor) y preguntas sensibles a la fecha (calendario).

Toolformer mejora sustancialmente sobre el modelo base del mismo tamaño y, en varias de estas tareas, alcanza o supera a modelos mucho mayores **sin** herramientas, manteniendo su capacidad de modelado de lenguaje.

Las cifras por benchmark y herramienta están en las tablas del artículo. Verificarlas allí antes de citarlas.

La miniatura de este eje aísla el criterio de filtrado: de tres candidatas, sobrevive la que reduce la pérdida por encima de τ ; la llamada absurda, que la **aumenta**, se descarta sola.

9. Impacto

- Establece que el uso de herramientas puede **aprenderse**, no solo prompt-earse. Es un cambio conceptual respecto a ReAct.
- Introduce un criterio automático y barato de utilidad, reutilizable en otros contextos.
- Anticipa la normalización del *tool calling* como capacidad nativa de los modelos, en lugar de como capa externa.
- Su lógica sobrevive a su implementación: hoy el acceso a herramientas se estandariza con protocolos, pero la pregunta «¿esta llamada aporta valor medible?» sigue siendo la correcta.

10. Limitaciones

1. **Enseña cuándo llamar, no garantiza que la herramienta acierte.** Un Toolformer conectado a una API que devuelve basura aprenderá a llamarla con confianza.

2. **τ no tiene valor universal.** Bajo, conserva llamadas inútiles (coste y latencia); alto, descarta llamadas útiles.
3. **Coste de construcción del corpus:** hay que muestrear muchas candidatas y ejecutarlas todas.
4. **Cada llamada es independiente.** No hay composición: no aprende a encadenar dos herramientas para un resultado intermedio.
5. **Depende de la escala del modelo base:** por debajo de cierto tamaño, las llamadas candidatas no son suficientemente buenas.
6. **La reducción de pérdida es una aproximación de la utilidad**, no la utilidad misma. Puede premiar correlaciones espurias.
7. **Sin gestión de errores:** no aborda qué hacer cuando la API falla o tarda.

11. Errores comunes

Error	Corrección
«Toolformer garantiza respuestas correctas»	Garantiza que la llamada ayudó a predecir el texto . La corrección del resultado depende de la herramienta.
«Más llamadas a herramientas = mejor agente»	Cada llamada cuesta latencia y dinero. La métrica útil es la utilidad marginal por llamada, no el conteo.
«Es lo mismo que ReAct»	ReAct usa herramientas en el prompt; Toolformer aprende dónde insertarlas y reentrena con ello.
«Necesita anotadores humanos»	No. El criterio es la propia pérdida del modelo: por eso el título dice <i>teach themselves</i> .
«El filtro compara solo con no llamar»	Compara también contra llamar sin recibir respuesta, para aislar el valor del resultado.

12. Relación con trabajos anteriores

- **P13 ReAct (2022)** — usar herramientas dentro del bucle.
- **P10 GPT-3 (2020)** — aprendizaje en contexto, que permite generar las llamadas candidatas con pocos ejemplos.
- **WebGPT (Nakano et al., 2021)** — navegación con supervisión humana; el contraste directo con la autosupervisión de Toolformer. [arXiv:2112.09332](https://arxiv.org/abs/2112.09332)
- **PAL / Program-Aided Language Models (2022)** — delegar el cálculo a un intérprete. [arXiv:2211.10435](https://arxiv.org/abs/2211.10435)

13. Relación con trabajos posteriores

- **Tool calling nativo** en las APIs de modelos comerciales: la funcionalidad se vuelve producto.
- **Model Context Protocol** — estandarización del descubrimiento y la invocación de herramientas. modelcontextprotocol.io
- **P16 Sistemas agentic** — herramientas tipadas, permisos, presupuesto y seguridad de la cadena de suministro.
- **Trabajo sobre composición de herramientas (2023+)** — encadenar llamadas, que este paper no aborda.

14. Notebook asociado

P14_toolformer.ipynb

Qué implementa: el criterio de filtrado sobre tres candidatas (útil, marginal y absurda), un barrido de τ que muestra el compromiso entre conservar de más y de menos, y métricas que sustituyen al conteo bruto de llamadas.

Qué NO implementa: modelo de lenguaje, muestreo de candidatas, ejecución real de APIs ni reentrenamiento. Las pérdidas están fijadas para exhibir el criterio, y así se declara en la salida.

```
ai-evolution paper-lab P14 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe el criterio de filtrado y explica qué compara.
Explicar	Explica por qué comparar contra «llamada sin resultado» es necesario.
Aplicar	Barre τ en el notebook y anota cuántas llamadas sobreviven en cada caso.
Analizar	Diseña un caso donde la reducción de pérdida premie una llamada inútil.
Evaluar	Un equipo mide el éxito de su agente por «llamadas a herramientas por respuesta». Evalúa la métrica y propón tres alternativas.
Crear	Define una herramienta nueva para un dominio que conozcas y describe qué texto del corpus revelaría su utilidad.

16. Autoevaluación

1. ¿Qué señal sustituye a la anotación humana y por qué está disponible gratis?
2. ¿Qué pasa con τ muy bajo? ¿Y muy alto?
3. ¿Por qué el método necesita un modelo base suficientemente grande?
4. ¿Qué **no** garantiza este método sobre las respuestas de las herramientas?
5. ¿Por qué el conteo de llamadas es una mala métrica de calidad?
6. ¿En qué se diferencia de ReAct, en una frase?
7. ¿Qué capacidad relacionada con herramientas queda fuera del alcance de este paper?

17. Respuestas esperadas

1. La reducción de la pérdida de predecir el texto siguiente. Está disponible en cualquier corpus sin etiquetar, porque el «objetivo» es el propio texto que ya venía después.
2. Con τ bajo se conservan llamadas inútiles: el modelo aprende a invocar herramientas constantemente, con su coste y su latencia. Con τ alto se descartan llamadas útiles y el modelo vuelve a inventar los datos.

3. Porque las llamadas candidatas se generan con pocos ejemplos en el prompt. Si el modelo no produce llamadas plausibles, el filtro no tiene nada bueno que conservar.
4. Que sean correctas. El criterio mide utilidad predictiva, no veracidad.
5. Porque no distingue competencia de bucle. Siete llamadas pueden ser una descomposición excelente o un reintento caro que no converge.
6. ReAct **usa** herramientas mediante prompting; Toolformer **aprende** dónde llamarlas y reentrena el modelo con ese conocimiento.
7. La composición: encadenar la salida de una herramienta como entrada de otra. Cada llamada se evalúa de forma independiente.

18. Fuentes primarias

- Schick, T. et al. (2023). *Toolformer: Language Models Can Teach Themselves to Use Tools*. **NeurIPS 2023**. [arXiv:2302.04761](https://arxiv.org/abs/2302.04761) · consultado 2026-08-16.
- Nakano, R. et al. (2021). *WebGPT: Browser-assisted question-answering with human feedback*. [arXiv:2112.09332](https://arxiv.org/abs/2112.09332) · consultado 2026-08-16.
- Gao, L. et al. (2022). *PAL: Program-aided Language Models*. [arXiv:2211.10435](https://arxiv.org/abs/2211.10435) · consultado 2026-08-16.

Evaluación — P14

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Toolformer: Language Models Can Teach Themselves to Use Tools* (2023, arXiv:2302.04761 · NeurIPS 2023) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P14_toolformer.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P15 — DPO

El resultado que eliminó el andamiaje: si la política óptima del objetivo RLHF tiene forma cerrada, la recompensa se puede despejar y el modelo se ajusta directamente con las preferencias. Sin modelo de recompensa. Sin aprendizaje por refuerzo.

Nivel: L4 · Motor: dpo · Notebook: P15_dpo.ipynb

1. Identificación

Campo	Valor
Título original	Direct Preference Optimization: Your Language Model is Secretly a Reward Model
Autoría	Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, Chelsea Finn
Año	2023
Venue	arXiv:2305.18290 · NeurIPS 2023
Fuente primaria	arXiv:2305.18290
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

El pipeline de InstructGPT funciona, pero es caro y frágil:

- entrena **un modelo adicional** (el de recompensa) que hay que mantener y validar;
- requiere **muestreo on-policy** durante el entrenamiento por refuerzo;
- **PPO es sensible** a hiperparámetros y difícil de estabilizar;
- hay que mantener **varios modelos en memoria** a la vez (política, referencia, recompensa, crítico).

La pregunta: ¿todo ese aparato es necesario, o es un rodeo?

3. Propuesta

Un resultado teórico con una consecuencia práctica inmediata.

El resultado. La solución óptima del objetivo RLHF con restricción KL tiene forma cerrada:

$$\pi^*(y \mid x) = (1 / Z(x)) \cdot \pi_{\text{ref}}(y \mid x) \cdot \exp(r(x, y) / \beta)$$

La consecuencia. De ahí se despeja la recompensa:

$$r(x, y) = \beta \cdot \log[\pi^*(y|x) / \pi_{\text{ref}}(y|x)] + \beta \cdot \log Z(x)$$

Al sustituir esto en el objetivo Bradley-Terry, **el término $Z(x)$ se cancela** —porque aparece en ambas ramas de la comparación— y queda una pérdida de clasificación binaria sobre pares de preferencias, expresada únicamente en términos de la política.

En una frase: **la política ya es el modelo de recompensa**. No hace falta entrenar uno aparte para luego optimizar contra él; se optimiza la política directamente.

4. Intuición sin fórmulas

RLHF entrena un juez y luego entrena al alumno a gustarle al juez. DPO demuestra que el alumno **ya contiene** al juez: su preferencia se lee comparando lo que dice ahora con lo que decía antes de entrenarse. Basta con ajustarlo para que suba lo preferido y baje lo rechazado, sin alejarse demasiado de donde estaba.

Dónde deja de funcionar la analogía: el juez de RLHF puede evaluar respuestas nuevas, generadas durante el entrenamiento. DPO solo aprende de los pares que ya tiene: no explora.

5. Matemática mínima

Pérdida DPO:

$$L = - \mathbb{E}_{\{(x, y_w, y_l)\}} [\log \sigma(\beta \cdot [\log(\pi(y_w|x)/\pi_{\text{ref}}(y_w|x)) - \log(\pi(y_l|x)/\pi_{\text{ref}}(y_l|x))])]$$

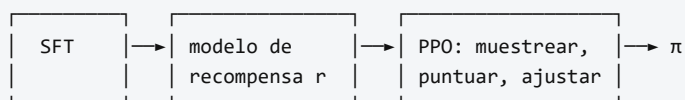
Recompensa implícita:

$$\hat{r}(x, y) = \beta \cdot \log[\pi(y|x) / \pi_{\text{ref}}(y|x)]$$

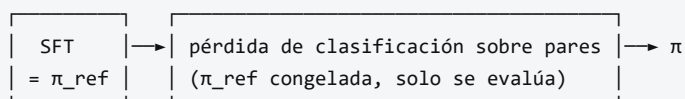
- y_w preferida, y_l rechazada, π_{ref} la política de referencia (habitualmente el modelo SFT), β el coeficiente que controla la desviación permitida.
- **π_{ref} es imprescindible.** Sin ella el objetivo premia subir $p(y_w)$ sin límite y la política colapsa a una distribución degenerada. El log-ratio es lo que convierte «subir» en «subir *relativamente a la referencia*», y β pone el precio.
- **La restricción KL no desapareció:** quedó absorbida en la forma del log-ratio.

6. Arquitectura o flujo

RLHF (tres etapas, dos modelos extra)



DPO (una etapa, ningún modelo extra)



7. Qué observar en el paper original

- La **derivación completa** de la forma cerrada y la cancelación de $Z(x)$. Es corta y vale la pena seguirla símbolo a símbolo: es el núcleo de todo el trabajo.
- El **análisis del gradiente**: el peso de cada par es mayor cuanto más se equivoca el modelo de recompensa implícito, lo que da una lectura intuitiva del comportamiento.
- La **comparación de la frontera recompensa–KL** frente a PPO. No basta con «gana en la métrica»: hay que ver a qué distancia de la referencia.
- Los experimentos de **generación controlada de sentimiento, resumen y diálogo**.
- La discusión sobre **generalización fuera de distribución**, donde los autores son cautos.

8. Evidencia y resultados

Tres escenarios:

- **generación controlada de sentimiento**, donde se puede computar la frontera óptima recompensa–KL y comparar objetivamente;
- **resumen** (Reddit TL;DR), evaluado con preferencia de un modelo juez;
- **diálogo de un turno** (Anthropic HH).

DPO iguala o supera a PPO en calidad de preferencia, con una implementación mucho más simple, más estable y sin ajustar un modelo de recompensa separado. En generación de sentimiento, DPO alcanza una frontera recompensa–KL mejor que PPO.

Las tasas de victoria y las curvas por escenario están en las figuras del artículo. Verificarlas allí antes de citarlas.

La miniatura de este eje muestra el mecanismo en su forma más desnuda: sobre una política de tres opciones, optimizar la pérdida DPO desplaza la masa hacia la respuesta preferida y produce recompensas implícitas positivas para ella y negativas para las rechazadas.

9. Impacto

- **Simplificó radicalmente la alineación** y la puso al alcance de equipos sin infraestructura de RL. Es hoy la vía por defecto para ajustar modelos abiertos con preferencias.
- Abrió una familia de métodos derivados: IPO, KTO, ORPO, SimPO y otros, cada uno con una variante de la pérdida.
- Reforzó una lección metodológica: **antes de construir maquinaria, comprobar si el problema tiene solución analítica**. El resultado estaba implícito en la formulación de RLHF desde el principio.

10. Limitaciones

1. **No explora**. Solo aprende de los pares disponibles; RLHF puede muestrear respuestas nuevas y evaluarlas. Si los pares no cubren una región, DPO no aprende nada de ella.

- 2. **Depende críticamente de la calidad y cobertura de las preferencias.** Basura entra, basura sale — igual que RLHF, pero sin un modelo de recompensa que se pueda inspeccionar por separado.
- 3. **Sin recompensa explícita que auditar.** En RLHF se puede estudiar r de forma aislada: qué premia, dónde falla. En DPO esa información está distribuida en la política.
- 4. **Sensible a β :** demasiado bajo, colapso; demasiado alto, apenas se mueve.
- 5. **Tendencia a reducir la probabilidad de ambas respuestas** del par en algunos regímenes, fenómeno documentado en trabajo posterior.
- 6. **Requiere π_{ref} en memoria** durante todo el entrenamiento.
- 7. **La equivalencia teórica con RLHF supone** un modelo de preferencias Bradley-Terry y un óptimo alcanzable; en la práctica ninguna de las dos cosas se cumple exactamente.

11. Errores comunes

Error	Corrección
«DPO es siempre mejor que RLHF»	Es más simple y a menudo igual de bueno. RLHF conserva la ventaja de explorar respuestas nuevas on-policy.
«DPO no tiene restricción KL»	La tiene, absorbida en el log-ratio contra π_{ref} y escalada por β .
«Se puede quitar π_{ref} y simplificar más»	Sin π_{ref} no hay ancla y la política colapsa. Es el anti-patrón del notebook.
«DPO no necesita datos de preferencia»	Los necesita exactamente igual. Lo que elimina es el modelo de recompensa, no los datos .
«La recompensa implícita es una metáfora»	Es una identidad: $\hat{r} = \beta \cdot \log(\pi/\pi_{\text{ref}})$ se deriva del óptimo del objetivo RLHF.
«DPO evita el reward hacking»	Reduce una superficie (el modelo de recompensa explotable), no el problema de fondo: sigue optimizando un proxy de la preferencia.

12. Relación con trabajos anteriores

- **P12 InstructGPT (2022)** — el pipeline que se simplifica.
- **Bradley y Terry (1952)** — el modelo de preferencias por pares. doi.org/10.2307/2334029
- **Schulman et al. (2017), PPO** — el algoritmo que DPO evita. [arXiv:1707.06347](https://arxiv.org/abs/1707.06347)
- **RL con regularización KL** — la formulación de la que se deriva la forma cerrada.

13. Relación con trabajos posteriores

- **IPO (2023)** — corrige supuestos del modelo de preferencias. [arXiv:2310.12036](https://arxiv.org/abs/2310.12036)
- **KTO (2024)** — aprende de señales binarias sin necesidad de pares. [arXiv:2402.01306](https://arxiv.org/abs/2402.01306)
- **ORPO, SimPO (2024)** — variantes que eliminan π_{ref} o fusionan etapas.
- **P16 Sistemas agentic** — la alineación como componente de un sistema, no como paso final.

14. Notebook asociado

P15_dpo.ipynb

Qué implementa: la pérdida DPO optimizada sobre una política categórica de tres opciones, la comparación entre π_{ref} y π_{DPO} , el cálculo de la recompensa implícita para varios β , y una demostración de por qué eliminar π_{ref} provoca colapso.

Qué NO implementa: modelo de lenguaje, generación, tokenización ni longitud variable. Tres opciones discretas no son una política sobre secuencias; el mecanismo es el mismo, la escala no.

```
ai-evolution paper-lab P15 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la pérdida DPO y la expresión de la recompensa implícita.
Explicar	Explica por qué $Z(x)$ se cancela y por qué eso es lo que hace viable el método.
Aplicar	Ejecuta el notebook con tres valores de β y compara las recompensas implícitas.
Analizar	Compara DPO y RLHF en cinco dimensiones: coste, estabilidad, exploración, auditabilidad y datos requeridos.
Evaluar	¿Cuándo elegirías RLHF pese a su complejidad? Da dos escenarios concretos.
Crear	Deriva la pérdida DPO desde el objetivo RLHF con restricción KL, paso a paso, en una página.

16. Autoevaluación

- ¿Qué significa «tu modelo de lenguaje ya es un modelo de recompensa»?
- ¿Por qué se cancela $Z(x)$ y qué pasaría si no se cancelara?
- ¿Qué papel juega π_{ref} y qué ocurre exactamente si se elimina?
- ¿Qué controla β ?
- ¿Qué capacidad de RLHF se pierde con DPO?
- ¿DPO necesita menos datos de preferencia que RLHF?
- ¿Por qué la simplicidad de un método es un argumento técnico y no solo estético?

17. Respuestas esperadas

- Que la recompensa implícita se puede leer como $\beta \cdot \log(\pi/\pi_{\text{ref}})$. La política contiene la información que el modelo de recompensa explícito codificaría, expresada como desviación respecto a la referencia.
- Porque $Z(x)$ depende solo de x y aparece idéntico en las dos ramas de la comparación $r(x, y_w) - r(x, y_l)$. Si no se cancelara, habría que calcular una normalización sobre todas las salidas posibles, que es intratable.
- Es el ancla. Sin ella, la pérdida premia aumentar $p(y_w)$ indefinidamente y la política colapsa a una distribución degenerada que siempre dice lo mismo.
- Cuánto se permite que la política se aleje de la referencia: es el equivalente al coeficiente de la penalización KL en RLHF.
- La exploración on-policy. RLHF genera respuestas nuevas durante el entrenamiento y las evalúa; DPO solo ve los pares del conjunto de datos.
- No. Necesita los mismos datos. Elimina el **modelo** de recompensa, no la anotación.

7. Porque menos piezas significan menos hiperparámetros, menos modos de fallo, menos cómputo y mayor reproducibilidad. Todo eso es medible.

18. Fuentes primarias

- Rafailov, R. et al. (2023). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. **NeurIPS 2023**. arXiv:2305.18290 · consultado 2026-08-16.
- Bradley, R. A. y Terry, M. E. (1952). *Rank Analysis of Incomplete Block Designs*. **Biometrika**, 39(3/4), 324–345. doi.org/10.2307/2334029 · consultado 2026-08-16.
- Azar, M. G. et al. (2023). *A General Theoretical Paradigm to Understand Learning from Human Preferences (IPO)*. arXiv:2310.12036 · consultado 2026-08-16.

Evaluación — P15

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Direct Preference Optimization: Your Language Model is Secretly a Reward Model* (2023, arXiv:2305.18290 · NeurIPS 2023) · **Nivel:** L4

Parte A — Contexto histórico (20 pts)

- (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
- (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

- (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
- (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

- (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

- (10) Ejecuta `P15_dpo.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
- (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
- (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

- (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
- (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P16 — Sistemas agentic contemporáneos

Nodo de frontera, revisable. No es un paper: es un conjunto de trabajos posteriores a ReAct que convierten el bucle en un sistema. Se lee con fecha de consulta y se relee.

Nivel: L5 · **Motor:** `agentic` · **Notebook:** `P16_agentic_systems.ipynb`

[!WARNING] Esta ficha es la más volátil del eje y se aparta deliberadamente del formato de las quince anteriores en un punto: **agrupa varios trabajos** en lugar de analizar uno. Lo hace explícito para no fingir un consenso que no existe. Lo estable aquí son las **preguntas**; lo inestable son los nombres de framework de cada temporada. Última revisión: **2026-08-16**.

1. Identificación

Campo	Valor
Título original	— (nodo compuesto, no es un paper único)
Autoría	Varios equipos independientes
Año	2023 en adelante
Venue	arXiv, NeurIPS, ICLR, UIST y especificaciones abiertas
Fuentes primarias	ver sección 18
Acceso	Abierto
Fecha de consulta	2026-08-16

Trabajos que componen el nodo

Trabajo	Año	Qué añade al bucle
Reflexion (Shinn et al.)	2023	Autocrítica verbal y memoria entre intentos
Generative Agents (Park et al.)	2023	Memoria episódica con recuperación por relevancia, importancia y recencia
Voyager (Wang et al.)	2023	Currículo autónomo y biblioteca de habilidades reutilizables
AutoGen (Wu et al.)	2023	Orquestación conversacional entre varios agentes
Model Context Protocol	2024	Estandarización del acceso a herramientas, recursos y prompts

2. Problema anterior

ReAct demostró el bucle y Toolformer demostró que el uso de herramientas se aprende. Pero un bucle desnudo no sobrevive al contacto con una tarea real:

- **no recuerda** nada de un episodio al siguiente;

- **no se corrige**: si el plan es malo, lo ejecuta hasta el final;
- **no tiene presupuesto**: puede consumir indefinidamente;
- **no sabe parar**: ante un fallo de herramienta, reintenta;
- **no escala a varios agentes** sin un protocolo de coordinación;
- **no tiene una forma estándar** de descubrir e invocar herramientas.

Cada uno de estos huecos generó una línea de trabajo.

3. Propuesta

No hay una propuesta única. Hay una **descomposición en componentes** que la práctica ha ido estabilizando, aunque los nombres varíen:

Componente	Función	Origen representativo
Plan	Descomponer el objetivo en pasos verificables	ReAct, Voyager
Herramientas tipadas	Acciones con contrato de entrada, salida y efectos	Toolformer, MCP
Memoria	Episódica, semántica y de trabajo, con recuperación	Generative Agents
Reflexión	Criticar el propio resultado y reintentar mejor	Reflexion
Presupuesto	Límite de pasos, tokens, coste y tiempo	práctica operativa
Criterio de parada	Cuándo terminar, abortar o escalar	práctica operativa
Orquestación	Varios agentes con roles y protocolo	AutoGen
Permisos y aislamiento	Qué puede tocar el agente y con qué autoridad	seguridad de sistemas

4. Intuición sin fórmulas

Un becario brillante sin instrucciones, sin presupuesto y sin nadie a quien preguntar cuando algo falla. El problema no es su capacidad: es que el **sistema** a su alrededor no existe. Un agente que funciona en la demo y falla en producción casi nunca falla por el modelo.

Dónde deja de funcionar la analogía: el becario sabe cuándo está fuera de su competencia y pregunta. El agente no tiene ese sentido; hay que construirlo explícitamente como criterio de parada y escalamiento.

5. Matemática mínima

No hay una ecuación central. Lo que hay es un **contrato operativo** que sí se puede formalizar:

```
estado_t = (objetivo, plan, memoria_t, presupuesto_restante_t)
```

```
a_t ~  $\pi(\cdot \mid \text{estado}_t)$ 
```

```
o_t = entorno(a_t) ← puede fallar
```

```
memoria_{t+1} = actualizar(memoria_t, a_t, o_t)
```

```
presupuesto_{t+1} = presupuesto_t - coste(a_t)
```

```
PARAR si: objetivo cumplido
```

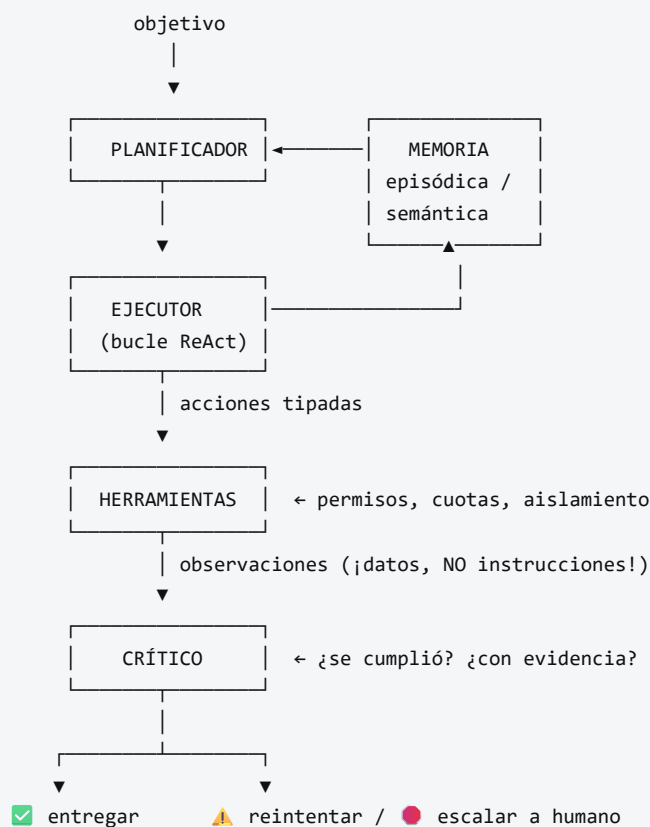
```
✓ presupuesto agotado
```

```
✓ fallo no recuperable → escalar a humano
```

```
✓ estado repetido → detectar bucle
```

La última línea es la que separa un prototipo de un sistema operable.

6. Arquitectura o flujo



Todo el bucle bajo PRESUPUESTO y con TRAZA persistida.

7. Qué observar en el paper original

Al ser un nodo compuesto, lo que hay que observar es **transversal**:

- En **Reflexion**: cómo se convierte un fallo en texto que mejora el siguiente intento, y cuál es el límite de esa mejora.
- En **Generative Agents**: la función de recuperación de memoria (relevancia, recencia, importancia) y la evaluación con humanos, no solo con benchmarks.

- En **Voyager**: la biblioteca de habilidades como forma de memoria **procedimental**, y la generación del currículo.
- En **AutoGen**: qué problemas mejoran con varios agentes y cuáles **no** — la sección más útil y la que menos se cita.
- En **MCP**: la separación entre *tools*, *resources* y *prompts*, y qué implica para permisos.
- **En todos**: cuántas ejecuciones se reportan y con qué varianza. Es la pregunta que más demos derriba.

8. Evidencia y resultados

Aquí hay que ser especialmente cuidadoso. El área tiene una brecha grande entre demostración y evidencia:

- muchos resultados se reportan sobre **pocas ejecuciones**, sin varianza;
- los **benchmarks de agentes** son jóvenes y algunos han mostrado problemas de contaminación o de especificación ambigua;
- la comparación entre frameworks rara vez controla el modelo base, el prompt y el presupuesto;
- el **coste** (llamadas, tokens, latencia) se omite con frecuencia, y es la mitad de la decisión.

Antes de citar cualquier cifra de esta área: comprobar número de ejecuciones, varianza, modelo base, presupuesto y fecha. Si falta alguno, la cifra no es comparable.

La miniatura de este eje no mide capacidad: muestra un agente con presupuesto explícito que se topa con un fallo de herramienta y **escala en lugar de responder igualmente**. Parar es un resultado correcto.

9. Impacto

- Traslada el foco del **prompt** al **sistema**: memoria, presupuesto, permisos, trazas y criterios de parada.
- Convierte la operación de agentes en una disciplina con su propio nombre y sus propias métricas (tasa de éxito, de escalamiento correcto, de respuesta inventada, coste por tarea).
- Hace de la **seguridad** un requisito de diseño: la inyección de prompt indirecta a través de observaciones es un vector real, no teórico.
- Empuja hacia la **estandarización** del acceso a herramientas, con las implicaciones de cadena de suministro que eso conlleva.

10. Limitaciones

1. **Evidencia débil y heterogénea.** Es la limitación principal de todo el nodo.
2. **Sin definición consensuada de «agente».** Cada trabajo usa la palabra para algo distinto.
3. **Fiabilidad compuesta.** Con 90 % de éxito por paso, diez pasos dan ≈ 35 %. La aritmética es implacable y se ignora demasiado a menudo.
4. **Coste y latencia** frecuentemente ausentes del reporte.
5. **Superficie de ataque amplia:** inyección indirecta, herramientas maliciosas, exfiltración de datos a través de acciones legítimas.
6. **Atribución de responsabilidad sin resolver:** quién responde cuando el agente actúa mal.

7. **Los nombres cambian más rápido que las ideas.** Buena parte de la literatura de una temporada queda obsoleta en la siguiente sin haber sido refutada: simplemente se abandona.

11. Errores comunes

Error	Corrección
«Funcionó una vez, está listo»	<code>n=1</code> no es evidencia. Sin distribución, varianza y casos adversarios, es una anécdota.
«Más agentes = mejor»	Añadir agentes multiplica coste, latencia y modos de fallo. Hay que demostrar que aporta.
«El agente es autónomo»	Opera bajo un objetivo, un presupuesto y unos permisos que alguien definió. La autonomía es un rango, no un interruptor.
«La traza demuestra cómo razonó»	Igual que en ReAct: es texto generado, útil para auditar decisiones, no para probar procesos.
«Si el agente responde, hizo su trabajo»	Un sistema que siempre responde está ocultando sus fallos. La abstención es una capacidad.
«El contenido que lee el agente son instrucciones»	Toda observación es dato . Tratarla como instrucción es la vulnerabilidad de inyección indirecta.
«Este es un campo maduro»	Es un campo activo con evidencia joven. Tratarlo como maduro es el error más caro.

12. Relación con trabajos anteriores

- **P13 ReAct (2022)** — el bucle base.
- **P14 Toolformer (2023)** — herramientas aprendidas.
- **P15 DPO (2023)** — alineación accesible del componente de decisión.
- **P11 RAG (2020)** — memoria consultable, antecedente directo de la memoria semántica de un agente.
- **Agentes racionales clásicos** (Russell y Norvig) — la definición de agente como percepción → decisión → acción es muy anterior a los LLM. Ver la clase 004 del programa.

13. Relación con trabajos posteriores

Por definición, esta sección **caduca**. Lo posterior a este nodo no se añade aquí: se registra con fecha y fuente en `frontier/current-topics.yaml`, y solo asciende a `papers/foundational/` cuando se consolida.

Preguntas abiertas que conviene seguir:

- evaluación de agentes con protocolos comparables y coste incluido;
- fiabilidad compuesta: cómo llevar tareas de muchos pasos a tasas de éxito operables;
- defensa contra inyección indirecta con garantías, no con heurísticas;
- memoria a largo plazo que no degrade con el volumen;
- atribución de responsabilidad y trazas verificables por terceros.

14. Notebook asociado

Qué implementa: un agente con plan, herramientas, memoria, presupuesto y criterio de parada que se topa con un fallo de verificación y escala en lugar de reintentar; el mapa componente → riesgo si falta; y el contraste entre un reporte anecdótico ($n=1$) y uno con distribución.

Qué NO implementa: ningún modelo de lenguaje. El agente **ejecuta** un plan, no lo planifica. Un agente real planifica con un LLM y puede equivocarse justo ahí.

```
ai-evolution paper-lab P16 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Enumera los seis componentes del contrato operativo y qué falla si quitas cada uno.
Explicar	Explica la fiabilidad compuesta con un ejemplo numérico de 10 pasos.
Aplicar	Reduce el presupuesto en el notebook y comprueba que el agente aborta limpiamente.
Analizar	Toma un trabajo de la tabla del nodo y analiza qué evidencia aporta y qué deja sin demostrar.
Evaluar	Te presentan un agente con «92 % de éxito». Escribe las siete preguntas que harías antes de aceptarlo.
Crear	Diseña el protocolo de evaluación de un agente de tu dominio: métricas, casos adversarios, presupuesto y criterio de aceptación.

16. Autoevaluación

1. ¿Qué distingue un bucle de un sistema agentic?
2. Con 95 % de éxito por paso, ¿cuál es la tasa esperada en una tarea de 15 pasos?
3. ¿Por qué la abstención es una capacidad y no un fallo?
4. ¿Qué es la inyección de prompt indirecta y qué regla la previene?
5. ¿Por qué añadir agentes puede empeorar un sistema?
6. ¿Qué debe contener el reporte de evaluación de un agente para ser aceptable?
7. ¿Por qué esta ficha lleva fecha de consulta destacada y las anteriores no tanto?

17. Respuestas esperadas

1. Los componentes que rodean al bucle: memoria, presupuesto, criterio de parada, permisos, trazas y escalamiento. El bucle es el motor; el sistema es el vehículo.
2. $0,95^{15} \approx 0,46$. Menos de la mitad. Es el argumento para acortar cadenas, verificar pasos intermedios y diseñar puntos de control.
3. Porque un sistema que siempre responde convierte sus fallos en respuestas plausibles. Poder decir «no lo sé» o «necesito ayuda» es lo que hace operable un sistema en producción.
4. Que contenido leído por el agente (una página, un documento, un correo) contenga texto dirigido a él. La regla: **toda observación es dato, nunca instrucción**; las instrucciones solo vienen del usuario por su canal.

5. Porque cada agente añade coste, latencia, puntos de fallo y ambigüedad de coordinación. El beneficio debe demostrarse contra una línea base de un solo agente bien construido.
6. Número de ejecuciones, varianza, tasa de éxito, de escalamiento correcto y de respuesta inventada, coste medio y percentil alto, modelo base, presupuesto, casos adversarios probados y fecha.
7. Porque agrupa trabajos recientes cuya evidencia aún se está consolidando. Las quince fichas anteriores describen resultados asentados y replicados; esta describe un frente activo.

18. Fuentes primarias

- Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K. y Yao, S. (2023). *Reflexion: Language Agents with Verbal Reinforcement Learning*. **NeurIPS 2023**. [arXiv:2303.11366](https://arxiv.org/abs/2303.11366) · consultado 2026-08-16.
- Park, J. S. et al. (2023). *Generative Agents: Interactive Simulacra of Human Behavior*. **UIST 2023**. [arXiv:2304.03442](https://arxiv.org/abs/2304.03442) · consultado 2026-08-16.
- Wang, G. et al. (2023). *Voyager: An Open-Ended Embodied Agent with Large Language Models*. [arXiv:2305.16291](https://arxiv.org/abs/2305.16291) · consultado 2026-08-16.
- Wu, Q. et al. (2023). *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. [arXiv:2308.08155](https://arxiv.org/abs/2308.08155) · consultado 2026-08-16.
- *Model Context Protocol* — especificación abierta. modelcontextprotocol.io · consultado 2026-08-16.

Evaluación — P16

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Sistemas agentic contemporáneos (nodo de frontera, revisable)* (2023, Conjunto de trabajos posteriores a ReAct — releer con fecha de consulta) · **Nivel:** L5

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P16_agentic_systems.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).